第4章: Next.js の基礎

この章のゴールと地図

この章は、Next.js（ネクストジェイエス）で書籍管理アプリを作るための「土台の知識」をまとめた、本書でいちばん大事な章の1つです。とても長いので、最初に「どこへ向かうのか」を地図で確認しておきましょう。旅行の前に路線図を見ておくと、途中で「今どこ？」と迷わずに済むのと同じです。

この章を読み終えると、次のことができるようになります。

1. `app/` フォルダにファイルを置いて、URL とページを対応させられる（`page.tsx`・`layout.tsx`・`loading.tsx`・`error.tsx`・`[id]`）
2. あるコンポーネントが「サーバーで動くのか、ブラウザで動くのか」を説明でき、`"use client"` を必要な部品にだけ付けられる
3. 1回のアクセスで「サーバーで HTML を作る → ブラウザに届く → ハイドレーションでボタンが動き出す」流れを、図を使って説明できる
4. `<Link>` と `router.push` を使い分けて、ページを移動できる
5. Server Component でデータを取得し、`loading.tsx` / `error.tsx` で「読み込み中」「失敗した」画面を出せる
6. Server Action でフォームを処理し、`revalidatePath` で一覧を最新の状態にできる
7. `NEXT_PUBLIC_` の有無で、環境変数がどこから見えるか（どこに漏れるか）を説明できる

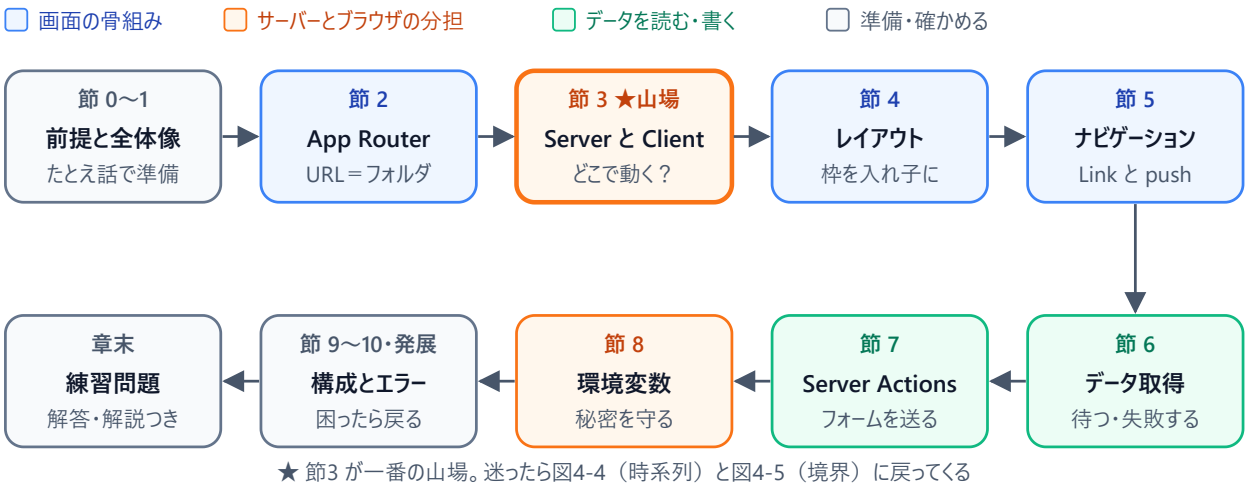


図4-1: この章の地図（矢印の順に読み進める。色は話題のまとまり）

読み進め方のヒント:

- 0〜1節は、たとえ話中心の準備運動です。「サーバー」「レンダリング」という言葉にピンと来ない人は、ここを飛ばさずに読んでください。

- 各サンプルは「**▼ このコードがやること（日本語）→ コード → ▼ 実行結果 → 1つずつ分解した解説**」の順に並んでいます。最初はコードを細かく読まず、日本語の説明と実行結果だけを読んで「何が起きるのか」をつかんでから、解説を読むと理解しやすくなります。
- **つまずきポイント** は、初心者が実際によくハマる失敗を「❌ NG（ダメな書き方）→ ✅ OK（直し方）」で並べたものです。エラーが出たときに見比べられるよう、実際のエラー文も載せています。
- 章末に「練習問題（解答・解説つき）」があります。読み終えたら、まず答えを見ずに解いてみてください。

> **この章の「▼ 実行結果」について:** 実行結果は、Next.js 16.4.1（2026年10月時点の最新安定版）・React 19・TypeScript 5 で、本書と同じ手順で作ったプロジェクトを実際に動かして記録したものです（個人のフォルダ名などは伏せています）。第6章の案内どおり、`create-next-app` で **Cache Components を「No」** にしたプロジェクトを前提にしています。「Yes」にした場合の違いは、6.1 節の最後の「注意」でまとめて説明します。Next.js 15 でも、ほとんどのコードはそのまま動きます。

0. 前提知識: URL・ルーティング・サーバーレンダリング

Next.js（読み: ネクストジェイエス）を理解するには「URL（ユーアールエル：Web上の住所）」「ルーティング（routing：URLとページを結びつける仕組み）」「レンダリング（rendering：HTMLを組み立てて画面に出す処理）」の3つの基本概念を押さえる必要があります。ここから始めましょう。

> **この章で前提となる用語の超ざっくり予習:**
>
> - **サーバー**：インターネットの向こうにあるコンピューター。リクエストを受けて返事をする。
> - **クライアント／ブラウザ**：ユーザーの手元のコンピューターで動くChromeやSafariなどのアプリ。
> - **HTML**：画面に出る文字や画像のレイアウト情報を書いた文書。
> - **JavaScript**：ブラウザの中で動くプログラム。React や Next.js のコードもこれにコンパイルされる。

0.1 URL の構造

```
https://example.com:443/products/123?sort=price&page=2#reviews
```

`https`	**プロトコル（通信方式）**。`https` は暗号化された通信、`http` は暗号化されていない通信。今は基本 `https` を使う。
`example.com`	**ホスト名（ドメイン）**。サーバーの住所。`example.com` のような人間に読める形と、実際の IP アドレスが DNS で結ばれている。
`:443`	**ポート番号**（省略時は https なら 443、http なら 80）。普段は省略されているので意識しないことが多い。
`/products/123`	**パス**（path: ページの場所）。Next.js ではこの部分が `app/` フォルダの構造とそのまま対応する。
`?sort=price&page=2`	**クエリパラメータ**（? 以降）。`sort=price&page=2` のように「キー=値」を `&` で繋いで送る追加情報。
`#reviews`	**フラグメント**（ページ内位置）。`#reviews` のように `#` の後に書く部分。ページ内の ID 指定に使う。

Next.js で書く各ページは、この**パス**（`/products/123` の部分）に対応します。つまり「URL のパスをどう設計するか＝どんなフォルダ構造にするか」という関係が成り立ちます。

0.2 ルーティングって何？

「URL を受け取って、どのページを表示するか決める仕組み」が **ルーティング**（routing：道案内のこと。URLに対応するページを「探して呼び出す」処理）です。Next.js の **App Router**（アップルーター：Next.js 13.4以降で正式採用された新しいルーティング方式。`app/` フォルダ配下の構造で URL を決める）では、`app/` フォルダの中のフォルダ構成がそのまま URL になります。これを **ファイルベースルーティング**（File-based Routing：ファイルやフォルダの配置自体がそのままルート定義になる方式）と呼びます。

app/		この `app/` フォルダが Next.js の出発点
├ page.tsx	→ /	トップページ。`page.tsx` があるフォルダ＝その URL でアクセスできる
├ about/		フォルダ名が URL のセグメント（区切り）になる
│ └ page.tsx	→ /about	`app/about/page.tsx` は `/about` のページ本体
└ books/		`/books` 以下のグループ
├ page.tsx	→ /books	`/books` の一覧ページ
└ [id]/		角括弧 `[id]` は「動的セグメント」: 数字でも文字でも何でも受ける
└ page.tsx	→ /books/123	例えば `/books/42` でアクセスすると `id="42"` として受け取れる（`[id]` は動的な値）

> **ファイル＝ページ**: 「`app/about/page.tsx` を作っただけで `/about` というURLでそのページが見られるようになる」というのが Next.js のキモです。第3章までの React だけでは、`react-router-dom` などのライブラリを別途インストールし、自分で `<Route path="/about" element={...} />` のようにルーティング設定を書く必要がありました。Next.js ではその設定作業が「ファイルを置く」という直感的な操作に置き換わります。

0.3 レンダリング3兄弟（CSR / SSR / SSG）

Webページを「画面に出るHTMLにする」処理を **レンダリング**（rendering：データやテンプレートから最終的な見た目のHTMLを組み立てること）と呼びます。これが**いつ・どこで**行われるかで3種類あります。

種類	フルネーム	いつHTMLができる？	どこで？	メリット
CSR	Client Side Rendering（クライアントサイドレンダリング：ブラウザ側でJSを動かしてHTMLを組み立てる方式）	アクセス時	ブラウザ	動的、軽量サーバー
SSR	Server Side Rendering（サーバーサイドレンダリング：サーバー側で完成形HTMLを作って送る方式）	アクセス時	サーバー	SEO良、初回表示速い
SSG	Static Site Generation（スタティックサイトジェネレーション：ビルド時にHTMLを事前生成しておく方式）	ビルド時に1回	サーバー	最速、安価

このほか **ISR**（Incremental Static Regeneration：インクリメンタル・スタティック・リジェネレーション。SSGをベースに「指定秒ごとに再生成」する仕組み）、**RSC**（React Server Components：リアクト・サーバー・コンポーネンツ。サーバーでだけ動く新しい種類のReactコンポーネント）、**ストリーミング**（Streaming：完成した部分から順次ブラウザに送る方式）、**ハイドレーション**（Hydration：サーバーで出来上がったHTMLにブラウザ側のJSが「水を与える」ように後付けで動きを付ける処理）という言葉もよく出てくるので、本章で順番に扱います。

> **本書での使い分け:** Next.js の App Router では、`page.tsx` をシンプルに書くと**サーバーで実行される（SSR/SSG相当）** のがデフォルトです。インタラクティブな部品（ボタンや入力欄など）だけ `"use client"` を付けて Client Component（クライアントコンポーネント：ブラウザ側で動くコンポーネント）にし、CSR的な動きを実現します。

はじめに

この章では、React ベースの**フルスタックフレームワーク**（Full-stack Framework：フロントエンド=画面もバックエンド=サーバー処理も両方カバーするフレームワーク）である **Next.js**（ネクストジェイエス） の基礎を学びます。

Next.js を使うことで、React 単体では実現が難しい**サーバーサイドレンダリング**（SSR：Server Side Rendering = サーバー側でHTMLを生成してからブラウザに送る方式。ページの初回表示が速くなる）、**静的サイト生成**（SSG：Static Site Generation = ビルド時にHTMLを事前生成する方式。最も高速）、**ファイルベースルーティング**（ファイルを配置するだけでURLとページが自動的に対応する仕組み）などを簡単に実装できます。

なぜ React だけではなく Next.js を使うのか

React は「UIの部品を作るライブラリ」であり、それ以外の機能（ページのURL管理、サーバーとの通信、SEO対策など）は自分で用意する必要があります。Next.js はこれらを**最初から備えている**ため、開発者はアプリのロジックに集中できます。

機能	React のみ	Next.js
ページのURL管理（ルーティング）	別途ライブラリ（react-router等）が必要	ファイルを置くだけで自動対応
サーバーでのデータ取得	自分でAPIサーバーを構築	Server Components で直接取得
SEO（検索エンジン対策）	追加設定が必要	標準で対応
ページ読み込み速度	初回が遅い（CSR）	高速（SSR/SSG）
デプロイ（公開）	別途設定が必要	Vercel で数クリック

この章で学ぶこと:

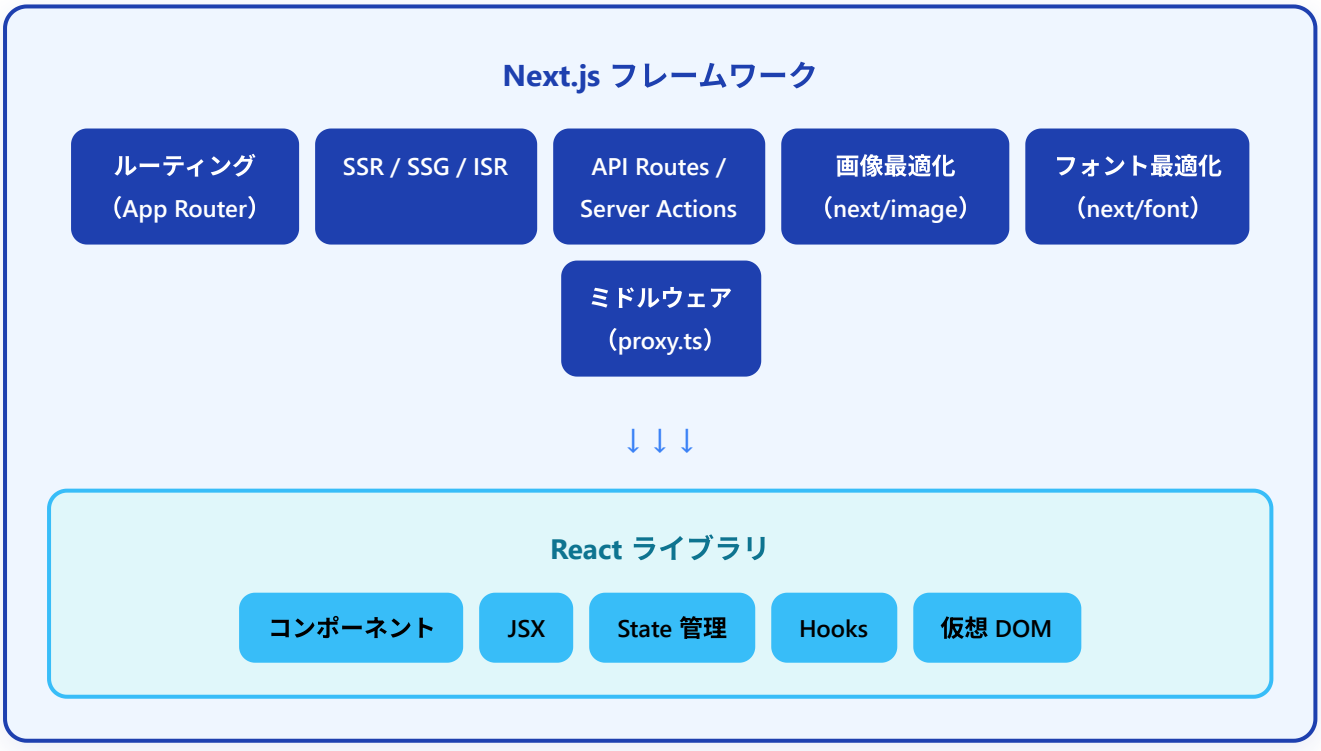
- Next.js の概念と React との関係
- **App Router**（アップルーター：Next.js 13以降の新しいルーティング方式）によるルーティング
- **Server Components**（サーバー上で実行されるコンポーネント）と **Client Components**（ブラウザ上で実行されるコンポーネント）の使い分け
- レイアウト、ナビゲーション、**データフェッチ**（Data Fetch：サーバーやAPIからデータを取得すること）の基本
- **Server Actions**（サーバーアクション：フォーム送信などのサーバー処理をシンプルに書ける仕組み）によるサーバー処理
- プロジェクト構成の**ベストプラクティス**（Best Practice：最も良いとされるやり方・慣習）

---

1. Next.js とは

1.1 React との関係

Next.js は **Vercel 社** が開発・メンテナンスしている React ベースのフレームワークです。React はあくまで「UIライブラリ」であり、ルーティングやサーバーサイド処理などは含まれていません。Next.js はその React の上に、Web アプリケーション開発に必要な機能を包括的に提供します。



たとえ話で理解する:

- **React** = エンジン（車を動かす核心部分）

- **Next.js** = 完成した車（エンジンに加え、ハンドル、ブレーキ、ナビなど全部入り）

React 単体でアプリを作ることもできますが、ルーティングやデータ取得の仕組みを自分で選定・構築する必要があります。Next.js を使えば、それらが最初から統合されています。

1.2 SSR, SSG, CSR の違い

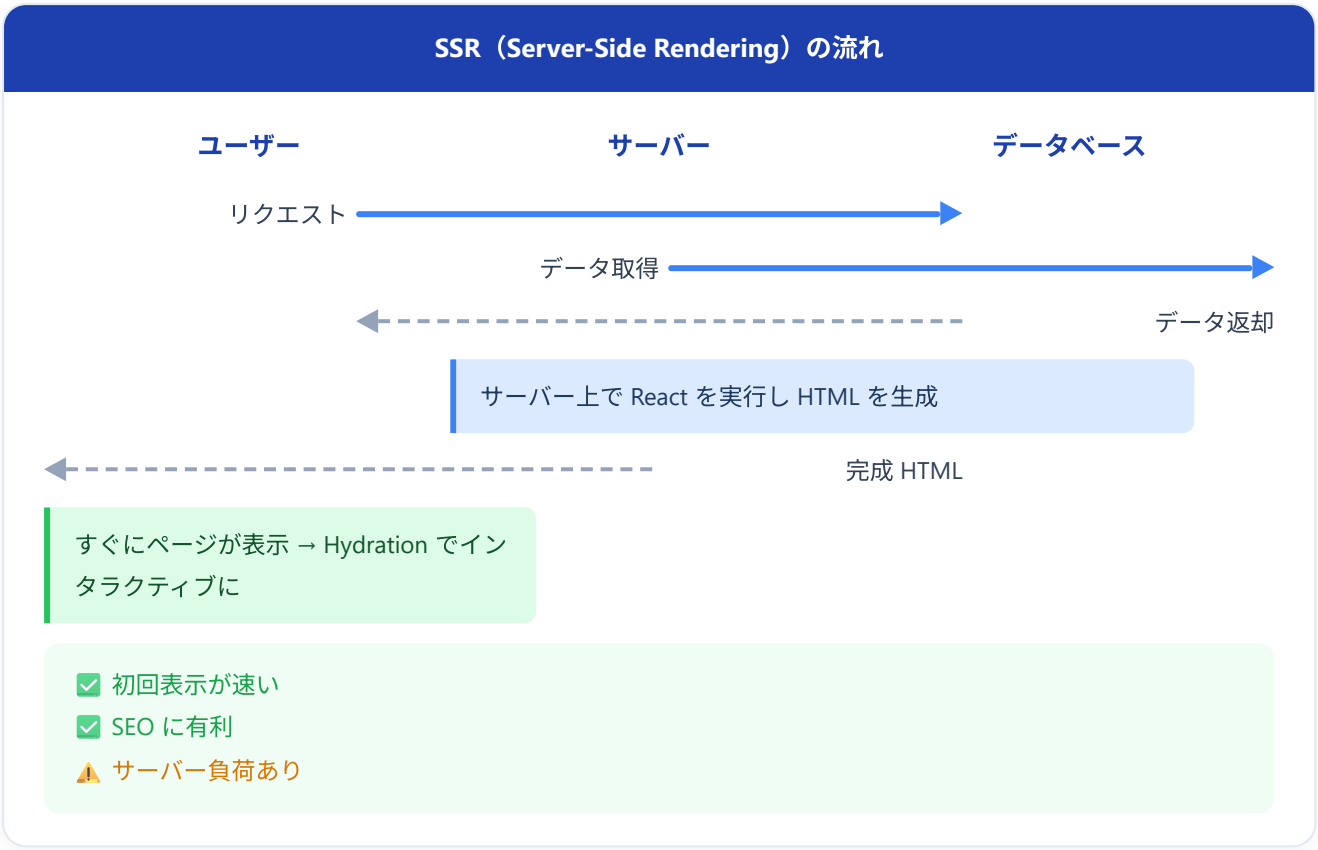
Web ページのレンダリング方法は大きく3つあります。Next.js はこれらすべてに対応しています。

CSR（Client-Side Rendering）- クライアントサイドレンダリング



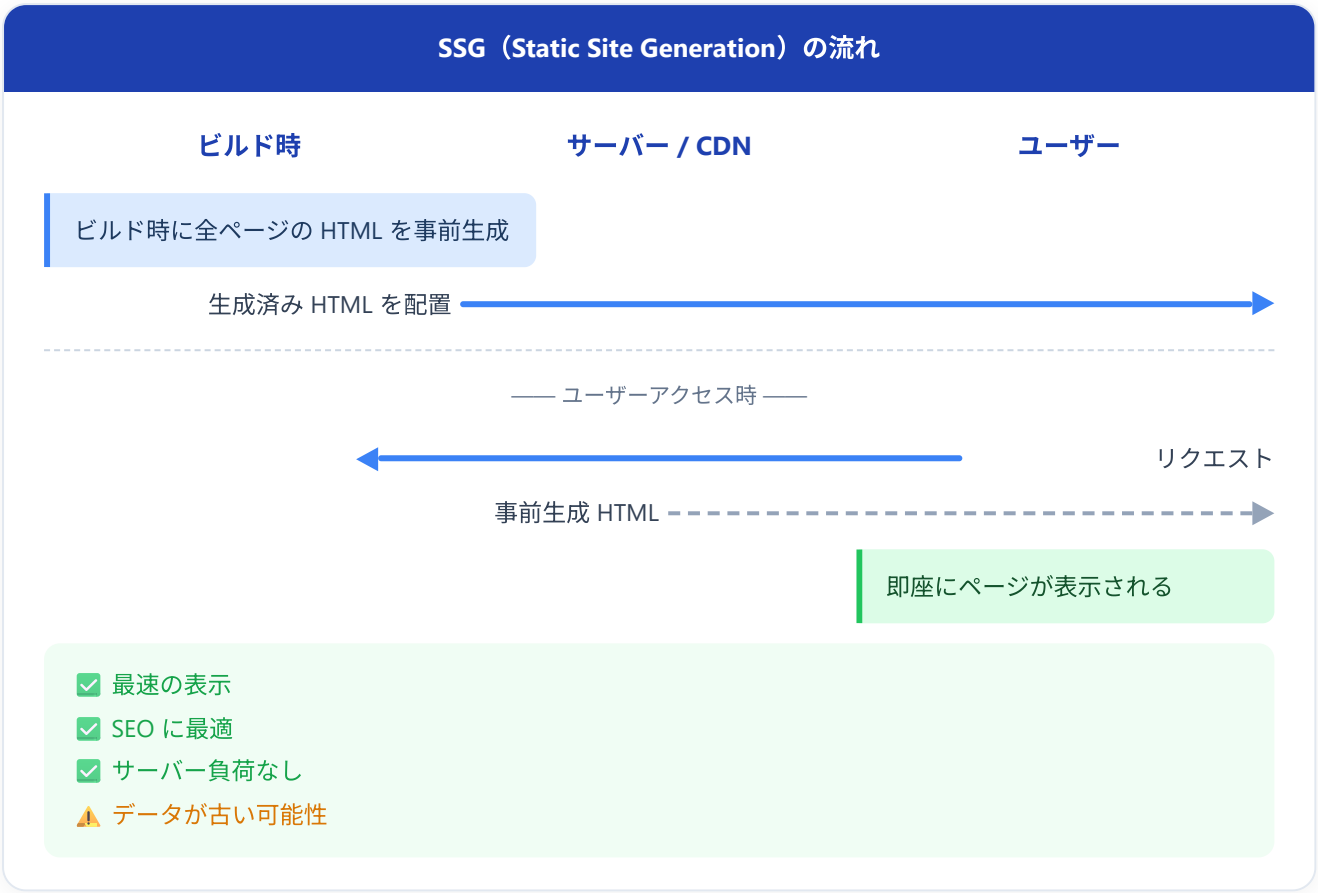
画面にはこう表示される: 最初に一瞬だけ白い画面やローディングスピナーが表示され、JavaScript の処理が完了してからコンテンツが表示されます。素の React（Create React App）はデフォルトでこの方式です。

SSR（Server-Side Rendering）- サーバーサイドレンダリング



画面にはこう表示される: リクエストのたびにサーバーで HTML が生成されるため、最新のデータが反映された完成形のページがすぐに表示されます。ブラウザが JavaScript を読み込むと、ボタンクリックなどのインタラクションが可能になります（この過程を **Hydration** と呼びます）。

SSG（Static Site Generation）- 静的サイト生成



画面にはこう表示される: ビルド時に生成済みの HTML がそのまま返されるため、表示速度は最速です。ただし、ビルド後にデータが変わっても、再ビルドするまで古いデータが表示され続けます。ブログや企業サイトなど、更新頻度の低いページに最適です。

3つのレンダリング方式の比較

特性	CSR	SSR	SSG
初回表示速度	遅い	速い	最速
SEO	不利	有利	最も有利
データの鮮度	常に最新	常に最新	ビルド時点
サーバー負荷	低い	高い	最も低い
使用例	ダッシュボード	EC サイト商品ページ	ブログ記事

1.3 なぜ Next.js を使うのか（素の React との比較）

機能	素の React	Next.js
ルーティング	react-router 等を別途インストール	App Router が組み込み
SSR / SSG	自分で構築が必要（非常に複雑）	設定なしで利用可能
コード分割	手動で React.lazy 等を使う	自動で最適化
画像最適化	自分で実装	`next/image` が自動最適化
フォント最適化	自分で実装	`next/font` が自動最適化
API エンドポイント	Express 等の別サーバーが必要	Route Handlers / Server Actions
TypeScript	設定が必要	初期設定済み
ESLint	設定が必要	初期設定済み
環境変数	dotenv 等を別途設定	`.env.local` が組み込み
デプロイ	ホスティング先の選定・設定が必要	Vercel にワンクリックデプロイ

> **結論:** 2024年以降、新規の React プロジェクトを始める場合は Next.js（App Router）を使うのがスタンダードです。React の公式ドキュメントでも Next.js が推奨フレームワークの1つとして紹介されています。

---

2. App Router

2.1 App Router vs Pages Router

Next.js には2つのルーティングシステムがあります。

特性	Pages Router（従来）	App Router（推奨）
ディレクトリ	`pages/`	`app/`
導入バージョン	Next.js 初期〜	Next.js 13.4〜（安定版）
デフォルトコンポーネント	Client Component	**Server Component**
レイアウト	`_app.tsx`, `_document.tsx`	`layout.tsx`（ネスト可能）
データ取得	`getServerSideProps`, `getStaticProps`	**async/await で直接**
ローディング UI	自分で実装	`loading.tsx`
エラー UI	`_error.tsx`（グローバル）	`error.tsx`（ルートごと）
Server Actions	非対応	**対応**
Streaming	非対応	**対応**

> **本チュートリアルでは App Router のみを使用します。** Pages Router（ページズルーター： `pages/` フォルダを使う Next.js の初期からあるルーティング方式）は旧バージョンとの互換性のために残されていますが、新規プロジェクトでは App Router を選択してください。Pages Router では「コンポーネントは原則 Client、データ取得は `getServerSideProps` などの専用関数で外側から差し込む」という設計でしたが、App Router では「コンポーネント自体を async にして直接 await する」という根本的に新しい設計になっています。

> **なぜデフォルトが Server Component なのか:** App Router は React Server Components（RSC：サーバーで実行されブラウザに HTML だけが送られる新種のコンポーネント）を基盤にしています。ブラウザに送る JS の量を減らし、機密情報の漏えいを防ぎ、初回表示を速くするために「まずは全部サーバー側で動かす」のがデフォルトなのです。クライアントでの動きが必要な箇所だけを `"use client"` で明示的に切り出していきます。

2.2 ファイルベースルーティングの仕組み

Next.js の App Router では、`app/` ディレクトリ内のフォルダ構造がそのまま URL のパスに対応します。これが **ファイルベースルーティング** です。



重要なルール: - **フォルダ** がURLのセグメント（パスの一部）になる - `page.tsx` があるフォルダだけがアクセス可能なルートになる - `page.tsx` がないフォルダは URL として公開されない（コンポーネントの整理用に使える）

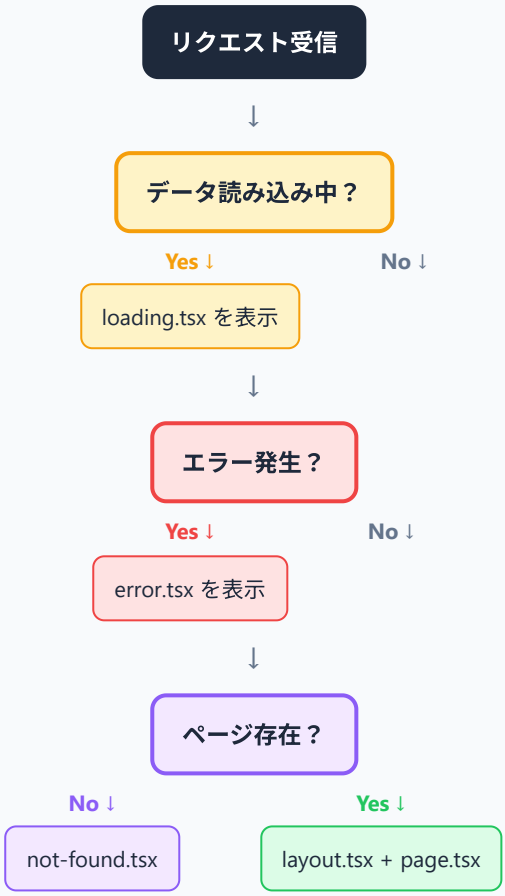
2.3 page.tsx, layout.tsx, loading.tsx, error.tsx の役割

App Router では、特別な名前を持つファイルにそれぞれ役割があります。

app/books/ ディレクトリ

- layout.tsx（共通レイアウト）
- page.tsx（ページ本体）
- loading.tsx（読み込み中の表示）
- error.tsx（エラー時の表示）
- not-found.tsx（404の表示）

レンダリングの流れ



page.tsx - ページ本体

`page.tsx`（ページ・ティーエスエックス）は **Next.js が「これがこのフォルダのページ本体」と認識する予約名のファイル** です。そのルート（URL）にアクセスしたときに表示されるメインコンテンツを書きます。`page.tsx` がないフォルダは URL として公開されない、というルールも合わせて覚えておきましょう。

> **▼ このコードがやること（先に日本語で）：** `app/page.tsx` を作って「トップページ（URL は `/`）の中身」を定義します。コンポーネント（画面の部品）は「JSX を返す関数」で、`export default` を付けて「このファイルの主役」として外に公開するのがポイントです。見出しと段落を表示するだけのシンプルなページです。詳しい1行ずつの意味はコード内コメントを見てください。

```tsx
// ファイルパス。`app/` 直下なので URL は `/`（ルート）になる
// app/page.tsx
// ブラウザでアクセスする URL の説明コメント
// URL: /

// `export default` でこのコンポーネントを「このファイルの主役」として外に公開
// 関数名 `HomePage` は何でもよい（慣習的にページ名を付ける）。Next.js は名前ではなく default export を見ている
export default function HomePage() {
  // JSX（HTMLに似たReactの記法）を返す
  return (
    // `<div>`：レイアウト用の汎用ブロック要素。中身を1つにまとめるために使う
    <div>
      {/* `<h1>`：ページの一番大きな見出し。1ページに1つだけ置くのが推奨 */}
      <h1>書籍管理アプリへようこそ</h1>
      {/* `<p>`：段落（paragraph）。普通の文章ブロック */}
      <p>あなたの読書記録を管理しましょう。</p>
    {/* 閉じタグ。開いた `<div>` と必ず対応させる */}
    </div>
  // return の終わり
  );
// 関数の終わり
}
```

画面にはこう表示される: ブラウザで `http://localhost:3000/` にアクセスすると、「書籍管理アプリへようこそ」という見出しと「あなたの読書記録を管理しましょう。」という段落が表示されます。

> **補足: なぜデフォルトが Server Component なのか？** App Router では `page.tsx` の中身はサーバーで実行され、結果のHTMLだけがブラウザに送られます。これは「JavaScript バンドル（ブラウザに送られるJSの塊）を小さく保つ」「DB接続情報など秘密の値をブラウザに漏らさない」「初回表示を速くする」という3つの利点があるからです。ボタンクリックなどユーザー操作が必要になったときだけ `"use client"` で Client Component に切り替えます。

layout.tsx - 共通レイアウト

複数のページで共有される UI（ヘッダー、サイドバーなど）を定義します。ページ遷移してもレイアウトは再レンダリングされず、状態が保持されます。

> **▼ このコードがやること（先に日本語で）:** すべてのページを「共通ヘッダー＋本文＋フッター」という枠で包む、アプリの一番外側の土台を作ります。`children`（チルドレン）という名前の引数に「各ページの中身」が自動で差し込まれる仕組みがキモで、ページを移動してもこの枠は作り直されません。`<html>` と `<body>` タグはこのルートレイアウトにだけ書く、という決まりも押さえてください。

```tsx
// ============================================================================
// ファイルパス: app/layout.tsx
// 役割       : すべてのページを「ヘッダー＋本文＋フッター」の枠で包む
//              ＝ Next.jsアプリの「一番外側の共通レイアウト」
// ----------------------------------------------------------------------------
// このファイルは Next.js が自動的に認識する「予約名」のひとつ。
// app/ フォルダ直下に layout.tsx が必ず1つ必要。
// ============================================================================

// `Metadata` 型を import する（Next.js 標準の型）。
// `import type` は「型だけ取り込む」記法。実行時のJSには残らないので軽量。
import type { Metadata } from "next";

// ──────────────────────────────────────────────────────────────────
// (1) メタデータの定義
// ──────────────────────────────────────────────────────────────────
// metadata という名前で export すると、Next.js が <head> 内の <title> や
// <meta name="description" ...> を自動生成してくれる。
// 自分で <head> を書く必要がない。
export const metadata: Metadata = {
  // ブラウザタブの文字
  title: "書籍管理アプリ",
  // SEO・SNS共有時の説明文
  description: "あなたの読書記録を管理するアプリケーション",
};

// ──────────────────────────────────────────────────────────────────
// (2) ルートレイアウトのコンポーネント本体
// ──────────────────────────────────────────────────────────────────
// `RootLayout` は予約名ではないが、慣習的にこの名前を付ける。
//
// 引数（Props）:
//   - children: このレイアウトの中に差し込まれる「各ページの中身」
//     React.ReactNode 型は「JSX/文字列/数値/null/...などReactで扱える何でも」を表す。
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    // ── 必須: <html> と <body> はルートレイアウトでだけ書く ──
    // lang="ja" は「このページは日本語ですよ」という指定。
    // スクリーンリーダーや翻訳ツールがこれを参照する。
    <html lang="ja">
      <body>

        {/* ── 共通ヘッダー（全ページに表示される） ── */}
        <header>
          <nav>
```

```
            <h1>📚 書籍管理アプリ</h1>
          </nav>
        </header>

        {/*
          ── ページ本体が差し込まれる場所 ──
          各 page.tsx の return 値が、ここに自動で入る。
          /books にアクセスしたら books/page.tsx の内容が、
          /books/123 にアクセスしたら books/[id]/page.tsx の内容がここに入る。
        */}
        <main>{children}</main>

        {/* ── 共通フッター（全ページに表示される） ── */}
        <footer>
          <p>&copy; 2024 書籍管理アプリ</p>
        </footer>

      </body>
    </html>
  );
}
```

▼ ブラウザでの見た目（どのページでも上下にこの枠が出る）：

```
┌──────────────────────────────────────┐
│ 📚 書籍管理アプリ                    │  ← <header> 内の <h1>
├──────────────────────────────────────┤
│                                      │
│   ← ここに各ページ（{children}）の   │
│      中身が差し込まれる              │
│                                      │
├──────────────────────────────────────┤
│ © 2024 書籍管理アプリ                │  ← <footer>
└──────────────────────────────────────┘
```

▼ 動作の仕組み:

URL	layout.tsx	中央 ({children})
`/`	表示される	`app/page.tsx` の中身
`/books`	表示される	`app/books/page.tsx` の中身
`/books/123`	表示される	`app/books/[id]/page.tsx` の中身

ページ遷移しても**ヘッダーとフッターは再描画されない**（=スクロール位置や状態が保たれる）のが Next.js のレイアウト機能のメリットです。

▼ コードを1つずつ分解して解説

上のレイアウトには「メタデータの export」「children を受け取る型」「`<html>` / `<body>` の特別扱い」という、初心者がつまずきやすいポイントが入っています。順番に見ていきましょう。

解説1: `metadata` を export するだけで `<head>` が自動生成される

```
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "あなたの読書記録を管理するアプリケーション",
};
```

- `metadata`（メタデータ）は「ページに関する情報」のことです。ブラウザのタブに出る**タイトル**や、検索エンジン・SNS共有で使われる**説明文**などがこれにあたります。
- ポイントは「`metadata` という**決まった名前**で `export` するだけ」という点です。これだけで Next.js が自動的に `<head>` の中の `<title>` や `<meta name="description">` を作ってくれます。**自分で `<head>` を書く必要がありません。**
- `: Metadata` の部分は「このオブジェクトは `Metadata` という型ですよ」とTypeScriptに教える型注釈です（`Metadata` 型はファイル冒頭で `import type { Metadata } from "next"` して取り込んでいます）。

> **用語: メタデータ（metadata）** … 「データについてのデータ」という意味。ページ本体の中身ではなく「このページは何か」を説明する付帯情報（タイトル・説明文など）を指します。

解説2: `children` を受け取って「各ページの中身」を差し込む

```
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
```

- `RootLayout` は、すべてのページを包む一番外側の枠（レイアウト）です。`export default` を付けて「このファイルの主役」として公開しています。
- 引数の `{ children }` は**分割代入**で、props（親から渡される値）の中から `children` だけを取り出しています。`children`（チルドレン＝子）には「**このレイアウトの内側に入る、各ページの中身**」が自動で入ってきます。
- `: { children: React.ReactNode }` は引数の型注釈です。`React.ReactNode`（リアクト・リアクトノード）は「JSX・文字列・数値・`null` など、Reactが画面に描画できるものなら何でも」を表す便利な型です。

> **用語: children（チルドレン）** … React で「あるコンポーネントのタグの内側に書かれた中身」を指す特別な props 名。Next.js のレイアウトでは「各ページの内容」がここに渡されます。

解説3: `<html>` と `<body>` はルートレイアウトにだけ書く

```
  <html lang="ja">
    <body>

      <header>
        <nav>
          <h1>📚 書籍管理アプリ</h1>
        </nav>
      </header>

      <main>{children}</main>

      <footer>
        <p>&copy; 2024 書籍管理アプリ</p>
      </footer>

    </body>
  </html>
```

- 普通の React コンポーネントでは `<html>` や `<body>` を書きませんが、**Next.js のルートレイアウトだけは例外**で、ここに `<html>` と `<body>` を書きます（ページ全体の最も外側の枠だからです）。
- `lang="ja"` は「このページは日本語ですよ」という指定で、翻訳ツールやスクリーンリーダー（読み上げソフト）がこれを参照します。
- `<header>`（ヘッダー）と `<footer>`（フッター）は全ページ共通で表示される枠です。その間の `<main>{children}</main>` の `{children}` の位置に、**各ページの中身がはめ込まれます**。`/books` を開けば books のページが、`/` を開けばトップページの中身が、この同じ場所に入れ替わりで表示されます。

> **用語: `<header>` / `<main>` / `<footer>`** … HTMLの「セマンティック要素」（意味を持つタグ）。それぞれ「上部の見出し領域」「主要な本文領域」「下部の補足領域」を表し、検索エンジンや支援技術がページ構造を理解する助けになります。

---

loading.tsx - 読み込み中の表示

`loading.tsx` は **Next.js が「データ取得中に表示するUI」と認識する予約名のファイル** です。データの読み込み中に自動的に表示される UI を書きます。仕組みとしては内部で React の `<Suspense>`（サスペンス：データ待ちの間に代

わりのUIを出す機能）を使っており、対応する `page.tsx` の Server Component が `await` でデータを待っている間、この `loading.tsx` の中身が代わりに描画されます。

> **▼ このコードがやること（先に日本語で）**：データ取得中に「読み込み中」を伝える画面（ローディングUI）を作ります。`loading.tsx` という予約名のファイルを置くだけで、Next.js が自動で「ページの準備ができるまでの間」にこれを表示してくれます。回転するスピナーと案内メッセージを出すだけのシンプルな部品です。詳細はコード内コメントを参照してください。

```tsx
// ファイルパス。`/books` のロード中UIになる
// app/books/loading.tsx
// フォルダごとに置けるので、ページ単位で異なるローディング表現が可能

// default export された関数がそのまま「ローディングUI」として使われる
// 関数名は何でもよいが、慣習的に `XxxLoading` と付ける
export default function BooksLoading() {
  // 返す JSX
  return (
    // `className` は HTML の `class` 属性のJSX版（`class` はJSの予約語なので別名）
    <div className="loading-container">
      {/* CSSアニメーションで作る回転するスピナー（自己終了タグ `/>` で書ける） */}
      <div className="spinner" />
      {/* ユーザーへの説明文 */}
      <p>書籍データを読み込んでいます...</p>
    {/* ラッパーdivの閉じタグ */}
    </div>
  );
// 関数の終わり
}
```

画面にはこう表示される: `/books` にアクセスした直後、データベースからデータを取得している間、スピナーアニメーションと「書籍データを読み込んでいます...」というメッセージが表示されます。データの取得が完了すると、自動的に `page.tsx` の内容に切り替わります。これは内部的に「サーバーがHTMLを少しずつ送る＝ストリーミング」と呼ばれる挙動で実現されています。

error.tsx - エラー時の表示

`error.tsx` は **Next.js が「そのフォルダ内でエラーが起きたときに表示するUI」と認識する予約名のファイル** です。ページの描画中に例外が `throw` された場合などに表示される UI を書きます。`"use client"` **が必須** です（React の Error Boundary（エラーバウンダリ：エラーをキャッチして代替UIを出す仕組み）はクラスコンポーネントを内部で使うため、Client Component でないと動かないため）。

▼ **このコードがやること（先に日本語で）**：ページ表示中にエラー（不具合）が起きたとき、白い画面で固まる代わりに「エラーが発生しました」という案内と「もう一度試す」ボタンを出す画面を作ります。`error.tsx` には必ず先頭に `"use client"` を書く必要があり、Next.js が `error`（起きたエラー情報）と `reset`（再挑戦用の関数）を自動で渡してくれます。ボタンを押すと `reset()` が呼ばれてその部分の再表示を試みます。詳細はコード内コメントを参照してください。

```
// `/books` 配下でエラーが起きたときに自動で表示される
// app/books/error.tsx
// この1行でこのファイルを「ブラウザ側で動くコンポーネント」に切り替える
// error.tsx ではこれを必ず先頭に書く（ファイルの一番上、importより前）
"use client";

// default export 関数。Next.js が自動で呼び出す
export default function BooksError({
  // props.error: 発生した Error オブジェクト
  error,
  // props.reset: 「もう一度試す」用に Next.js が渡してくれる関数
  reset,
// ↓ ここから TypeScript の型注釈
}: {
  // 標準 Error 型に、Next.js が独自に付ける `digest`（任意）を合成した型
  // digest はサーバーログとの紐付け用ID（本番環境で詳細メッセージを隠したいとき使う）
  error: Error & { digest?: string };
  // 引数なし・戻り値なしの関数型
  reset: () => void;
}) {
  return (
    // エラー表示の枠
    <div className="error-container">
      {/* タイトル */}
      <h2>エラーが発生しました</h2>
      {/* `error.message`：例外が持つメッセージ文字列を表示 */}
      {/* `{}` 内はJSの式。文字列補間に使う */}
      <p>{error.message}</p>
      {/* ボタン押下で `reset()` を呼ぶ → Next.js が該当セグメントを再描画してくれる */}
      {/* アロー関数で包んでいるのは「呼び出し時の引数（event）を渡さないため」 */}
      <button onClick={() => reset()}>もう一度試す</button>
    </div>
  );
}
```

画面にはこう表示される: 書籍データの取得中にエラーが発生すると、「エラーが発生しました」という見出しとエラーメッセージ、そして「もう一度試す」ボタンが表示されます。ボタンを押すと、そのセグメントの再レンダリングが試みられます。

> **補足: `not-found.tsx` もある:** `notFound()` 関数（後述）が呼ばれたときや404相当の状態のときに表示するUIとして `not-found.tsx` という予約ファイルも置けます。`error.tsx` と兄弟関係にあります。

> **注意（Next.js 16.3 以降の `retry`）:** Next.js 16.3 から、`error.tsx` には `reset` のほかに **`retry`** という関数も渡されるようになり、公式ドキュメントでは「たいていは `retry` を使う」と案内されています。違いは次のとおりです。
>
> - `retry()` … データを**取り直してから**、その部分を描き直す
> - `reset()` … データは取り直さず、エラー表示を消して描き直すだけ
>
> サーバーでのデータ取得に失敗したときは、取り直さないと同じ失敗を繰り返しやすいので `retry` が向いています。上のコードの `reset` を `retry` に置き換える（型も `retry: () => void` にする）だけで使えます。

`page.tsx`・`loading.tsx`・`error.tsx` が「いつ」画面に出てくるのかを、`/books` を開いたときの画面のイメージで見てみましょう。ポイントは、**入れ替わるのは中央の部分だけ**で、ヘッダーや左のメニュー（`layout.tsx` が描いている部分）はずっと同じまま、ということです。



上のヘッダーと左のメニュー（layout.tsx）は3つとも同じ。入れ替わるのは中央だけ

図4-2: 画面イメージ — loading.tsx・page.tsx・error.tsx が表示される瞬間

- ① データを取りに行っている間は、`loading.tsx` の中身が中央に表示されます。何も出ない真っ白な画面より、「今読み込み中です」と分かるほうが利用者は安心します。
- ② データが届くと、Next.js が自動で中央を `page.tsx` の中身に差し替えます。自分で「切り替える」コードを書く必要はありません。
- ③ データ取得の途中でエラーが `throw` されると、②の代わりに `error.tsx` が表示されます。ヘッダーやメニューは残っているので、利用者はほかのページへ移動したり、「もう一度試す」を押したりできます。

この「待つ」動きは 6.3 節で、ストリーミング（届いた順に少しずつ送る仕組み）として実際に時間を測って確かめます。

2.4 動的ルーティング（[id]）

URL の一部をパラメータ（parameter：動的に変わる値）として受け取りたい場合、フォルダ名を角括弧 `[...]` で囲みます。これを **動的セグメント**（Dynamic Segment：URLの中で変化する部分）と呼びます。

```
app/                                  # アプリのルートフォルダ
  books/                              # `/books` 以下のグループ
    [id]/                             # 角括弧で囲むと「ここは可変」を意味する。フォルダ名の `id`
が変数名になる
      page.tsx    → /books/1, /books/2, /books/abc などにマッチ    # 数字でも文字列でも何でも
受ける
```

> **▼ このコードがやること（先に日本語で）**：URL の一部（例: `/books/42` の「42」）をプログラムの中で受け取って画面に表示します。フォルダ名を `[id]` のように角括弧で囲むと、その部分が変化する値（パラメータ）になり、`params` という引数で受け取れます。Next.js 15 以降では `params` は `await` で取り出す必要がある点に注意してください（受け取った値は数値ではなく文字列で来ます）。詳細はコード内コメントを参照してください。

```tsx
// ファイルパス。フォルダ名 [id] が動的セグメント
// app/books/[id]/page.tsx

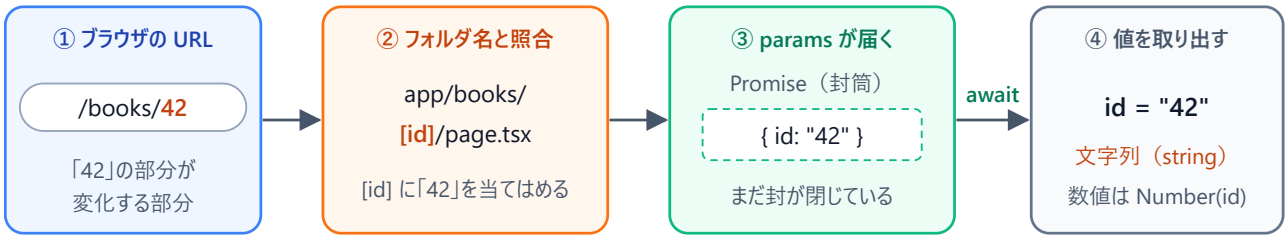
// params はオブジェクトとして渡される
// Next.js 15 では params は Promise として渡されるため await が必要
// `async` 関数にすると本体で `await` が使える
// この `async function` をコンポーネントに使えるのは Server Component の特権
export default async function BookDetailPage({
  // props.params: 動的セグメントの値が入ったオブジェクト
  params,
// ↓ TypeScript 型注釈
}: {
  // Next.js 15+ では params は Promise でラップされて渡される
  // `{ id: string }` は「id という文字列プロパティを持つオブジェクト」型
  params: Promise<{ id: string }>;
}) {
  // `await` で Promise を解決し、分割代入で `id` だけ取り出す
  // 例: URLが `/books/42` なら id は "42"（数値ではなく文字列！）
  const { id } = await params;

  return (
    // ページ全体のラッパー
    <div>
      {/* 見出し */}
      <h1>書籍詳細</h1>
      {/* 取り出した id を JSX に埋め込んで表示 */}
      <p>書籍 ID: {id}</p>
      {/* JSX の中のコメントは、この行のように波かっこ付きのブロックコメントで書く */}
      {/* 例: const book = await getBookById(id); */}
      {/* 実際にはここで id を使ってデータベースから書籍情報を取得する */}
    </div>
  );
}
```

画面にはこう表示される: - `/books/1` にアクセスすると「書籍 ID: 1」と表示されます。- `/books/42` にアクセスすると「書籍 ID: 42」と表示されます。- `/books/abc` にアクセスすると「書籍 ID: abc」と表示されます。

URL のその部分がそのまま `id` パラメータとして使えるわけです。

URL の「42」が `id` という変数に入るまでの道のりを、図で追ってみましょう。郵便物にたとえると、`params` は「中身の入った封筒（Promise）」で、`await` が「封を開ける」動作にあたります。



const { id } = await params; の1行が ③ → ④ の「封を開けて取り出す」にあたる

図4-3: URL の「42」が id という変数に入るまで

▼ コードを1つずつ分解して解説

この動的ルートのコードには「`async` コンポーネント」「`params` が Promise で来る」「`await` で取り出す」という3つの新しい考え方が詰まっています。1つずつ見ていきましょう。

解説1: コンポーネント関数に `async` を付けられるのは Server Component の特権

```
export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
```

- 関数名の前に付いている `async`（エイシンク＝非同期）は、「**この関数の中で `await`（待つ）が使えるようにする**」キーワードです。
- 普通の React（Client Component）では、コンポーネント関数を `async` にできません。**関数を `async` にできるのは Server Component だけの特権**です（このファイルは `"use client"` を書いていないので Server Component です）。
- `params`（パラムス＝パラメータの複数形）は、URLの動的セグメント（`[id]` の部分）の値が入った箱です。引数の分割代入で受け取っています。

> **用語: async / await** ...「時間がかかる処理（データ取得など）の結果を待つ」ための仕組み。`async` を付けた関数の中でだけ `await` が使え、`await ○○` は「○○ の結果が出るまでここで待つ」という意味になります。

解説2: Next.js 15 では `params` は「Promise」で渡される

```
  const { id } = await params;
```

- `params: Promise<{ id: string }>` の **`Promise`**（**プロミス＝約束**）は「**今すぐではなく、少し後に値が届く**」ことを表す型です。Next.js 15 以降では `params` がこの Promise に包まれて渡されます。
- そのため、中身を取り出すには `await params` のように **`await` で「値が届くのを待つ」**必要があります。届いたオブジェクト（`{ id: "42" }` のような形）から、分割代入で `id` だけを取り出しています。
- **注意点：取り出した `id` は数値ではなく文字列です。**URLが `/books/42` でも、`id` は数値の `42` ではなく文字列の `"42"` になります。数値として計算したいときは `Number(id)` で変換します。

> **用語: Promise（プロミス）** …「将来この値を渡します」という約束を表すオブジェクト。`await` を付けると、その約束が果たされる（値が届く）まで待ってから次の行へ進みます。

> **つまずきポイント（`await` を忘れて `params.id` と書く）**: Next.js 14 以前の解説記事やサンプルでは、`params.id` のように直接読み出していました。Next.js 15 から `params` は Promise（封筒）になり、16 では封を開けずに読む書き方は完全に使えなくなりました。古い書き方をそのまま写すと、次の2通りのどちらかになります。

型を `Promise<...>` と正しく書いた上で `params.id` と読むと、型チェック（`npx tsc --noEmit`）がエラーで教えてくれます。

```
app/books/[id]/page.tsx(7,25): error TS2339: Property 'id' does not exist on type
'Promise<{ id: string; }>'.
```

もっと怖いのは、型まで古い書き方（`params: { id: string }`）にしてしまった場合です。このときは型チェックも `next build` も**通ってしまい**、画面には id が空のまま「書籍 の詳細」と表示されます。開発サーバーのターミナルにだけ、次のエラーが出ます（`npm run dev` で `/books/42` を開いたときの実際の出力）。

```
Error: Route "/books/[id]" used `params.id`. `params` is a Promise and must be
unwrapped with `await` or `React.use()` before accessing its properties. Learn more:
https://nextjs.org/docs/messages/sync-dynamic-apis
    at BookDetailPage (app\books\[id]\page.tsx:7:25)
  5 |   params: { id: string };
  6 | }) {
> 7 |   return <h1>書籍 {params.id} の詳細</h1>;
    |                        ^
  8 | }
  9 |
```

```tsx
// ❌ NG: 型が古い書き方で、params を await せずに読んでいる（id が空になる）
export default function BookDetailPage({
  params,
}: {
  params: { id: string };
}) {
  return <h1>書籍 {params.id} の詳細</h1>;
}
```

```tsx
// ✅ OK: 型は Promise<...>、関数は async にして await で封を開けてから読む
export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  return <h1>書籍 {id} の詳細</h1>;
}
```

> エラー文に `Promise` や `must be unwrapped with await` という言葉が出てきたら、「`await` を忘れているかも」と思い出してください。インターネットで見つけた古いサンプルを使うときは、`params` と `searchParams` の書き方を必ず確認しましょう。

複数の動的セグメント

```
app/
  users/
    [userId]/
      books/
        [bookId]/
          page.tsx    → /users/5/books/12 にマッチ
```

> **▼ このコードがやること（先に日本語で）**：URL に変化する部分が2つある場合（例: `/users/5/books/12` ）に、その両方の値（ `userId` と `bookId` ）を同時に取り出して表示します。動的セグメントは2段以上ネストでき、`params` の中に複数のキーとして入ってきます。`await` で取り出してから分割代入で2つの変数に展開するのがポイントです。詳細はコード内コメントを参照してください。

```tsx
// ============================================================================
// ファイルパス: app/users/[userId]/books/[bookId]/page.tsx
// 役割       : URL の中の userId と bookId を取り出して表示するページ
// ----------------------------------------------------------------------------
// このファイルは Next.js が「動的セグメントを含むページ」と認識する。
// /users/5/books/12 にアクセスされると userId="5", bookId="12" になる。
// ============================================================================

// (1) 関数本体に async を付けると、内部で await が使える。
//     コンポーネント関数を async にできるのは Server Component だけの特権。
//
// (2) 引数の Props 型は { params: Promise<{ ... }> }
//     Next.js 15 以降では params が Promise でラップされて渡される。
//     なので await で「中身の値」を取り出してから使う。
export default async function UserBookPage({
  params,
}: {
  params: Promise<{ userId: string; bookId: string }>;
}) {
  // (3) Promise を await してオブジェクトを取り出し、
  //     さらに分割代入で userId と bookId に展開する。
  //     URL が /users/5/books/12 なら userId = "5", bookId = "12" となる。
  //     ※ 文字列で来ることに注意。数値が必要なら Number(userId) で変換する。
  const { userId, bookId } = await params;

  // (4) 取り出した値を JSX 内に埋め込んで表示
  return (
    <div>
      <p>ユーザー ID: {userId}</p>
      <p>書籍 ID: {bookId}</p>
    </div>
  );
}
// ▼ /users/5/books/12 にアクセスしたときの画面表示:
//   ユーザー ID: 5
//   書籍 ID: 12
```

キャッチオールセグメント

「何階層あっても全部受け取りたい」という場合は `[...slug]` のように `...`（スプレッド構文と同じ記号）を付けます。これを **Catch-all Segment**（キャッチオールセグメント：何階層でも一括で受け取る動的セグメント）と呼びます。

```
app/                                                    # ルート
  docs/                                                 # `/docs` 以下のグループ
    [...slug]/                                          # `...` 付きの動的セグメント。slug は配列
で渡る
      page.tsx   → /docs/a, /docs/a/b, /docs/a/b/c などにマッチ   # 何階層でもOK
```

▼ このコードがやること（先に日本語で）： `/docs/a/b/c` のように何階層続くか分からない URL を、まとめて1つのページで受け取ります。フォルダ名を `[...slug]` のように `...` 付きにすると、それ以降のパスが「文字列の配列」として渡ってきます（例: `["react", "hooks", "useEffect"]` ）。受け取った配列を `join(" / ")` でつなげて表示する例です。詳細はコード内コメントを参照してください。

```tsx
// ============================================================================
// ファイルパス: app/docs/[...slug]/page.tsx
// 役割       : スラッシュ区切りで何階層でもマッチする「キャッチオール」ページ
// ----------------------------------------------------------------------------
// [...slug] のように ... を付けると、それ以降のすべてのパスが配列として渡る。
// /docs/react           → slug = ["react"]
// /docs/react/hooks     → slug = ["react", "hooks"]
// /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]
// ============================================================================

export default async function DocsPage({
  params,
}: {
  // (1) slug は文字列の配列として渡る
  params: Promise<{ slug: string[] }>;
}) {
  // (2) Promise から slug 配列を取り出す
  const { slug } = await params;
  // 例: /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]

  // (3) Array.prototype.join(" / ") で配列を文字列にする
  //     ["react", "hooks", "useEffect"].join(" / ") → "react / hooks / useEffect"
  return (
    <div>
      <p>パス: {slug.join(" / ")}</p>
    </div>
  );
}
// ▼ /docs/react/hooks/useEffect にアクセスしたときの画面表示:
//   パス: react / hooks / useEffect
```

クエリパラメータ（`?q=...`）を page.tsx で受け取る

URL には、`/search?q=react&page=2` のように `?` の後ろに「キー=値」を付けることもできます（0.1 節のクエリパラメータ）。検索ワードやページ番号のように「同じページだけど、表示の条件が違う」ときに使います。フォルダ名は増やさず、`page.tsx` の引数 `searchParams` で受け取ります。

> **▼ このコードがやること（先に日本語で）：** `/search?q=react&page=2` の `?` より後ろの部分を、ページの中で受け取って表示します。`[id]` のときの `params` と同じく、`searchParams` という引数に Promise（封筒）で届くので、`await` で開けてから使います。届く値はすべて**文字列**なので、ページ番号の計算には `Number()` で数値に変えます。

```tsx
// app/search/page.tsx
// URL の例: /search?q=react&page=2
export default async function SearchPage({
  searchParams,
}: {
  searchParams: Promise<{ q?: string; page?: string }>;
}) {
  // ? 以降の値を取り出す。無いときは右側の値（デフォルト値）を使う
  const { q = "", page = "1" } = await searchParams;
  // URL から来る値は文字列なので、計算に使う前に数値へ変換する
  const pageNumber = Number(page);

  return (
    <div>
      <h1>「{q}」の検索結果</h1>
      <p>
        {pageNumber} ページ目を表示中（次は {pageNumber + 1} ページ目）
      </p>
    </div>
  );
}
```

▼ 実行結果（`npm run dev` で起動し、ブラウザで開いたときにできあがった HTML。開発者ツールの Elements タブで見たもの）

`http://localhost:3000/search?q=react&page=2` を開いたとき:

```
<div><h1>「react」の検索結果</h1><p>2 ページ目を表示中（次は 3 ページ目）</p></div>
```

`?` 以降を付けずに `http://localhost:3000/search` を開いたとき:

```
<div><h1>「」の検索結果</h1><p>1 ページ目を表示中（次は 2 ページ目）</p></div>
```

▼ コードを1つずつ分解して解説

解説1: `searchParams` も Promise で届く

```
export default async function SearchPage({
  searchParams,
}: {
  searchParams: Promise<{ q?: string; page?: string }>;
}) {
```

- `searchParams`（サーチパラムス）は、URL の `?` より後ろを「キーと値のオブジェクト」にしたものです。`?q=react&page=2` なら `{ q: "react", page: "2" }` が届きます。
- `params` と同じく Next.js 15 以降は Promise で届くので、関数に `async` を付けて `await` します。
- `q?: string` の `?` は「**無いこともある**」という意味です。利用者が `?q=` を付けずにアクセスすることもあるので、省略可能にしておきます。

> **用語: クエリパラメータとパス** … `/books/42` の「42」のように URL の**道筋（パス）**の一部になっている値は `[id]` フォルダと `params` で、`?q=react` のように **`?` の後ろに付け足す値**は `searchParams` で受け取ります。「どのページか」はパス、「そのページをどう表示するか（絞り込み・並び順・ページ番号）」はクエリ、と覚えると使い分けやすくなります。

解説2: 分割代入のデフォルト値で「無いとき」に備える

```
  const { q = "", page = "1" } = await searchParams;
```

- `{ q = "", page = "1" }` は、取り出すと同時に「値が無かったら `""` や `"1"` を使う」と決めておく書き方です（分割代入のデフォルト値）。
- 実行結果の2つ目のように、`/search` だけを開いても `q` は空文字、`page` は `"1"` になるので、エラーにならずに表示できます。

解説3: 文字列のまま足し算しない

```
  const pageNumber = Number(page);
```

- URL から来る値は**いつも文字列**です。`page` が `"2"` のまま `page + 1` とすると、文字列の連結になって `"21"` になってしまいます。
- `Number(page)` で数値の `2` に変えてから `+ 1` しているので、「次は 3 ページ目」と正しく表示されます。

> **ヒント:** ブラウザ側（Client Component）で今の URL のクエリを読みたいときは、5.2.1 で紹介する `useSearchParams` フックを使います。`page.tsx`（Server Component）では引数の `searchParams`、Client Component では `useSearchParams`、と覚えておきましょう。

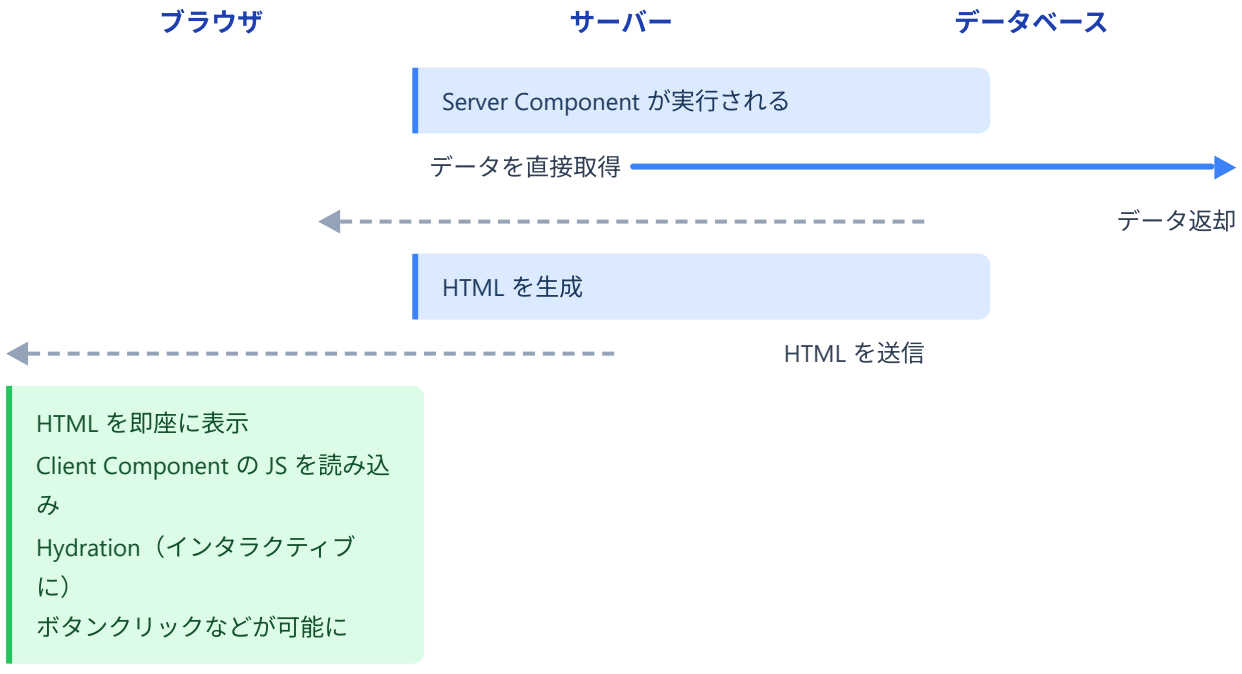
3. Server Components vs Client Components

3.1 概念の説明

Next.js App Router の最大の特徴は、**デフォルトですべてのコンポーネントが Server Component** であるということです。

Server Components（デフォルト）	Client Components（'use client'）
✅ サーバー上で実行される	🌐 ブラウザ上で実行される
✅ データベースに直接アクセス可能	🌐 API 経由でデータ取得
✅ JS バンドルに含まれない	🌐 JS バンドルに含まれる
❌ useState, useEffect 使用不可	✅ useState, useEffect 使用可能
❌ onClick 等のイベント処理不可	✅ onClick 等のイベント処理可能
❌ ブラウザ API 使用不可	✅ ブラウザ API 使用可能

Server Component の実行フロー



3.1.1 1回のアクセスで何が起きているか（時系列で見る）

上の図は「Server Component がサーバーで動く」ことを表していますが、初心者がいちばん混乱するのは「**Client Component はいつ・どこで動くのか**」です。レストランにたとえてみましょう。

- 厨房（サーバー）で料理（HTML）を作り、テーブル（ブラウザ）に運びます。テーブルに届いた瞬間から、料理は**見える**し写真も撮れます。
- ところが、テーブルの「呼び出しボタン」（Client Component のボタン）は、電池（JavaScript）を入れるまで押しても鳴りません。この**電池を入れる作業がハイドレーション**（Hydration：水を与える＝動きを吹き込む）です。
- 呼び出しボタンも、**見た目だけは厨房で一緒に作られて**運ばれてきます。つまり Client Component も、最初の1回はサーバーで描画されるのです。

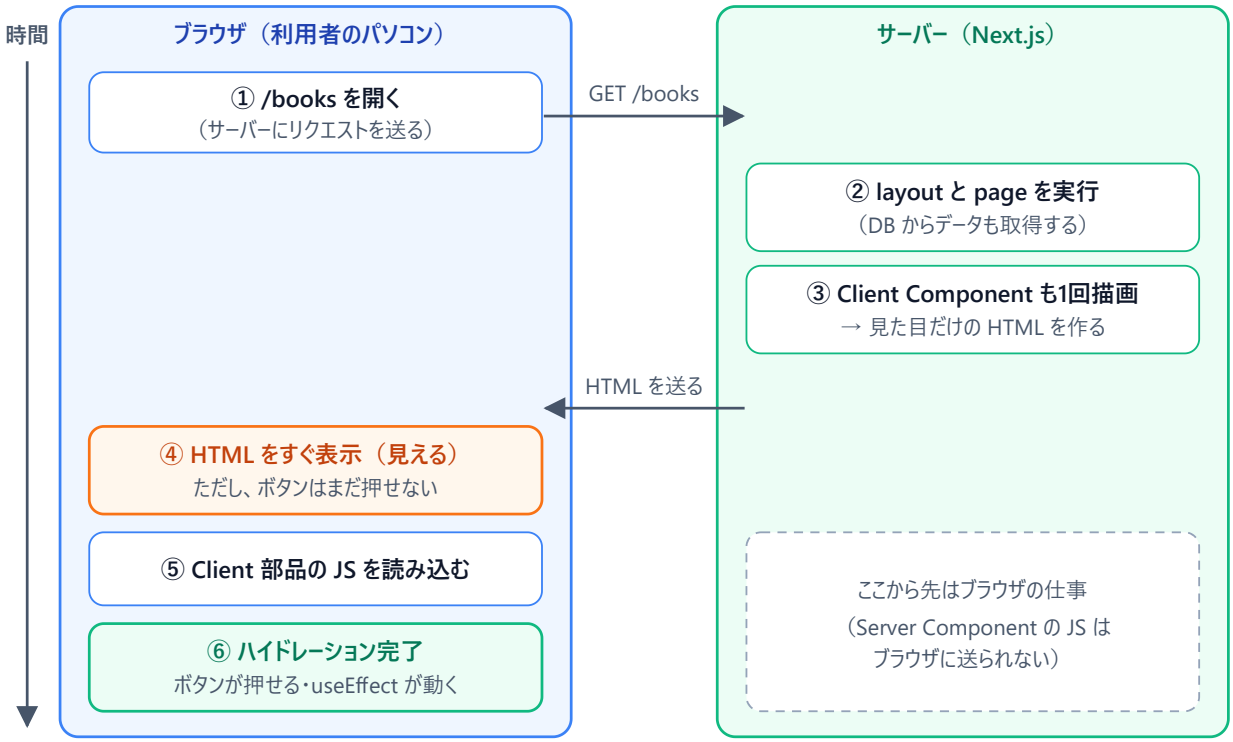


図4-4: 1回のアクセスの時系列 ― 「見える」のは④、「押せる」のは⑥

ポイントは次の3つです。

- **④で、もう画面は見えている。**サーバーで完成した HTML が届くので、データの入った画面がすぐ表示されます（SEO に強く、初回表示が速い理由）。
- **③で、Client Component もサーバーで1回動いている。**`"use client"` は「ブラウザ**だけ**で動く」という意味ではなく、「ブラウザ**でも**動く（ブラウザでの動きを担当する）」という意味です。だから Client Component の中で、いきなり `window`（ブラウザにしかない物）を触るとエラーになります（3.2 の最後の「つまずきポイント」）。
- **⑥まで、ボタンは押せない。**回線が遅いと④と⑥の間が長くなり、「表示されているのに押しても反応しない」時間ができます。

どこで動くのか、ログで確かめる

> **▼ このコードがやること（先に日本語で）**：Server Component（ページ）と Client Component（ボタン）の両方に `console.log` を書いて、ログが**サーバーのターミナル**と**ブラウザのコンソール**のどちらに出るかを確かめます。Client Component では `typeof window`（window があるかどうか）を調べて「今どこで動いているか」も表示します。図4-4 の③と⑥が本当に起きていることを、自分の目で確認するための実験です。

```tsx
// app/where/page.tsx（"use client" が無いので Server Component）
import { ClientLogger } from "./ClientLogger";

export default function WherePage() {
  // このログはどこに出る？
  console.log("[WherePage] Server Component を実行しました");

  return (
    <div>
      <h1>どこで動く？実験</h1>
      {/* ここに Client Component を埋め込む */}
      <ClientLogger />
    </div>
  );
}
```

```tsx
// app/where/ClientLogger.tsx（Client Component）
"use client";

import { useEffect, useState } from "react";

export function ClientLogger() {
  const [count, setCount] = useState(0);

  // window が存在するかどうかで「今どこで動いているか」を判定する
  const place = typeof window === "undefined" ? "サーバー" : "ブラウザ";
  // 描画（関数の実行）のたびにログを出す
  console.log(`[ClientLogger] 描画しました（実行場所: ${place}）`);

  // useEffect の中身は「ブラウザで画面に出た後」にだけ実行される
  useEffect(() => {
    console.log("[ClientLogger] useEffect を実行しました");
  }, []);

  return (
    <button onClick={() => setCount(count + 1)}>押した回数: {count}</button>
  );
}
```

▼ **実行結果**（`npm run dev` で起動し、ブラウザで `http://localhost:3000/where` を開いたとき）

開発サーバーを起動したターミナルには、次のように出ます（1行目はルートレイアウトに書いたログです）。

```
[RootLayout] サーバーで描画しました
[WherePage] Server Component を実行しました
[ClientLogger] 描画しました（実行場所: サーバー）
 GET /where 200 in 42ms (next.js: 1.8ms, application-code: 40ms)
```

ブラウザの開発者ツール（F12）の Console タブには、次のように出ます（React DevTools の案内と `[HMR] connected` の行は省略。行頭の `Server` は、灰色の小さな札として表示されます）。

```
Server  [RootLayout] サーバーで描画しました
Server  [WherePage] Server Component を実行しました
[ClientLogger] 描画しました（実行場所: ブラウザ）
[ClientLogger] 描画しました（実行場所: ブラウザ）
[ClientLogger] useEffect を実行しました
[ClientLogger] useEffect を実行しました
```

▼ コードを1つずつ分解して解説

解説1: Server Component のログは「サーバーのターミナル」に出る

```
  console.log("[WherePage] Server Component を実行しました");
```

- `WherePage` には `"use client"` が無いので Server Component です。この関数はサーバー（`npm run dev` を実行しているパソコンの Node.js）で動くので、ログは**ターミナル**に出ます。
- ブラウザの Console にも `Server` という札付きで同じ文が出ていますが、これは**開発中だけ**、Next.js がサーバーのログを写して見せてくれているものです。本番ではブラウザには出ません。ブラウザでこのコードが動いたわけではない点に注意しましょう。

解説2: Client Component も、最初の1回はサーバーで動く

```
  const place = typeof window === "undefined" ? "サーバー" : "ブラウザ";
  console.log(`[ClientLogger] 描画しました（実行場所: ${place}）`);
```

- `typeof window === "undefined"` は「`window`（ブラウザだけにある物）が存在しない＝今はサーバーで動いている」という判定です。
- ターミナルに「実行場所: サーバー」と出ているのが図4-4 の③です。Client Component も、最初の HTML を作るためにサーバーで1回描画されます。
- ブラウザの Console に「実行場所: ブラウザ」と出ているのが⑤〜⑥です。同じ関数が、**サーバーとブラウザの両方で**実行されたことが分かります。

> **用語: ハイドレーション（Hydration）** … サーバーから届いた「見た目だけの HTML」に、ブラウザ側で JavaScript を結び付け、ボタンのクリックなどに反応できる状態にする処理。サーバーで作った見た目と、ブラウザで作り直した見た目が**一致している**ことが前提で、食い違うと「Hydration エラー」になります（10 節のエラー6）。

解説3: ブラウザでは2回ずつ出る（開発中だけ）・useEffect はブラウザだけ

```
useEffect(() => {
  console.log("[ClientLogger] useEffect を実行しました");
}, []);
```

- ブラウザの Console で同じログが2回ずつ出ているのは、開発中だけ有効な React の **Strict Mode**（ストリクトモード：不具合を見つけやすくするために、描画や useEffect をわざと2回実行する仕組み）のためです。本番ビルドでは1回です。
- `useEffect` のログは**ターミナルには出ていません。** `useEffect` は「ブラウザで画面に表示された後」にだけ実行されるからです。ブラウザでしかできないこと（`window` を使う、`localStorage` を読むなど）は、`useEffect` の中に書くのが基本です。

3.2 "use client" ディレクティブ

Client Component にするには、ファイルの **先頭**（import 文よりも上）に `"use client"` と記述します。これを **ディレクティブ**（directive：コンパイラやランタイムに対する命令文）と呼びます。

> **`'use client'` の伝播ルール（重要）：**
>
> - `"use client"` を書いたファイルは、その時点で「Client Component の境界（ボーダー）」になります。
> - その境界より下流（そのファイルが import する他のファイル）は、明示しなくても自動的に Client Component として扱われます。
> - つまり「Client から import される Server Component」は実質作れません。
> - 逆に「Server Component から import される Client Component」はOKです。これが普通の組み合わせ。
> - 1つのページ内で「Server の骨組みの中に Client 部品を埋め込む」という構造が App Router の基本パターンです。

> **▼ このコードがやること（先に日本語で）：**「 `"use client"` を書かないと自動的に Server Component（サーバー側で動く部品）になる」というデフォルトの形を確認します。Server Component は関数を `async` にしてサーバー上で直接データを取りに行ける反面、`useState` やボタンのクリック処理などブラウザ側の機能は使えません。ここではデータ取得は省略し、文字を表示するだけの最小例です。詳細はコード内コメントを参照してください。

```
// ============================================================================
// Server Component の例（"use client" を書かない＝デフォルト）
// ============================================================================
// このファイルはサーバーでだけ実行される。
//   ✅ 関数を async にして await でDBやAPIにアクセスできる
//   ✅ 機密情報（APIキーなど）を扱える（ブラウザに送られない）
//   ❌ useState / useEffect / onClick などのインタラクティブ機能は使えない
//   ❌ window / document などブラウザ専用のAPIは使えない

export default function BookList() {
  // (1) 通常はここで await fetch(...) や DBクライアントを呼んでデータを取得する
  //     例: const books = await db.query("SELECT * FROM books");
  return <div>書籍一覧（サーバーで描画）</div>;
}
```

▼ このコードがやること（先に日本語で）：ファイル先頭に `"use client"` を書いて、ブラウザ側で動く Client Component（クライアントコンポーネント）にする例です。これにより `useState`（状態を覚えておく仕組み）やユーザー入力への反応が使えるようになります。入力欄に文字を打つたびに状態が更新され、画面の表示もそれに追従する「検索バー」を作ります。詳細はコード内コメントを参照してください。

```tsx
// ============================================================================
// Client Component の例（"use client" をファイル先頭に書く）
// ============================================================================
// このディレクティブが書かれた瞬間から、このファイルとそこから import される
// すべてのコンポーネントはブラウザで動く扱いになる。
"use client";

// (1) Client Component ではブラウザ側で動く React のフックが使える
import { useState } from "react";

export default function SearchBar() {
  // (2) useState で「検索クエリ文字列」を状態として管理する
  //     [現在の値, 値を変える関数] = useState(初期値)
  //     初期値が "" なので最初の query は空文字。
  const [query, setQuery] = useState("");

  // (3) JSX で <input> を描画
  //     ・value={query}      : 入力欄に現在の状態を反映する（制御コンポーネント）
  //     ・onChange={(e)=>...}: ユーザーが文字を打つたびに呼ばれる関数
  //     ・e.target.value     : 入力欄の現在の値（input要素のテキスト）
  //     ・setQuery(...)      : 状態を更新 → React が再描画 → value も更新される
  return (
    <input
      type="text"
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />
  );
}
```

▼ 動作:

- 検索バー（テキスト入力欄）が表示される
- キーボードで「Re」と打つと query が "" → "R" → "Re" に更新され、画面の入力欄もそれに追従する
- これは `useState` がブラウザのメモリに値を保持し、変更ごとに React が再描画するため
- サーバーはこの入力プロセスにまったく関与しない

重要: `"use client"` はそのファイルとそこから import されるすべてのモジュールを Client Component の境界として宣言します。つまり、Client Component の子コンポーネントも自動的に Client Component になります。

▼ コードを1つずつ分解して解説

この Client Component には「`"use client"` の宣言」「`useState` での状態管理」「制御コンポーネント」という3つの大事な要素があります。順に見ていきましょう。

解説1: ファイル先頭の `"use client"` で「ブラウザ側で動く部品」にする

```
"use client";

import { useState } from "react";
```

- `"use client"`（ユーズ・クライアント）は、ファイルの**一番上**（import より前）に書く「おまじない（ディレクティブ）」です。これを書くと、このファイルは「**ブラウザ側で動く Client Component**」になります。
- App Router ではデフォルトが Server Component（サーバー側で動く）なので、`useState` やボタンのクリック処理など「ブラウザでの操作」を使いたいときは、この1行で明示的に切り替える必要があります。
- 切り替えた後は、`react` から `useState`（状態を覚えておく仕組み）などのフックを import して使えるようになります。

> **用語: ディレクティブ（directive）** … プログラムに対する「指示書き」。`"use client"` は「このファイルはクライアント側で動かして」という Next.js への指示です。

解説2: `useState` で「入力された文字」を覚えておく

```
  const [query, setQuery] = useState("");
```

- `useState`（ユーズ・ステート）は「**変化する値を覚えておく箱**」を作るフックです。`const [今の値, 値を変える関数] = useState(初期値)` の形で使います。
- ここでは「今の値」を `query`（クエリ＝検索文字列）、「値を変える関数」を `setQuery` という名前にしています。初期値は `""`（空文字）なので、最初の `query` は空です。
- ユーザーが文字を打つたびに `setQuery(新しい文字)` を呼ぶと、`query` が更新され、Reactが画面を描き直します。

> **用語: state（ステート＝状態）** … コンポーネントが内部で持つ「変化するデータ」。state を更新すると、Reactが自動で画面を最新の状態に描き直します。

解説3: `value` と `onChange` をセットにする「制御コンポーネント」

```
  return (
    <input
      type="text"
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />
  );
```

- `value={query}` は「入力欄の表示内容を、state（`query`）と一致させる」指定です。
- `onChange={(e) => setQuery(e.target.value)}` は「**入力欄の中身が変わるたびに呼ばれる関数**」です。`e.target.value`（イー・ターゲット・バリュー）が「今入力されている文字」で、それを `setQuery` に渡して state を更新しています。
- この「`value` で表示し、`onChange` で更新する」というセットの仕組みを **制御コンポーネント（controlled component）** と呼びます。入力欄の中身を常にReactの state が握っている状態です。
- `placeholder`（プレースホルダー）は、何も入力していないときに薄く表示される案内文です。

> **用語: 制御コンポーネント（controlled component）** … 入力欄の値をReactのstateで管理する方式。「画面の表示」と「stateの値」が常に一致し、入力内容をプログラムから自由に読み書きできます。

> **つまずきポイント（Client Component の中でいきなり `window` を使う）:** 「`"use client"` を付けたからブラウザで動くはず」と考えて、関数の本体で `window.innerWidth`（ブラウザの画面幅）を読むと、ページが表示されずにエラーになります。図4-4 の③のとおり、Client Component も**最初の1回はサーバーで描画される**ので、サーバーには無い `window` を読もうとして失敗するのです。`npm run dev` で開いたときのターミナルの実際の出力です（実験用に `app/ng-window/` フォルダに置いた例）。

```
⨯ ReferenceError: window is not defined
    at WindowWidth (app\ng-window\WindowWidth.tsx:6:17)
  4 | export function WindowWidth() {
  5 |   // ブラウザの画面幅を読む
> 6 |   const width = window.innerWidth;
    |                 ^
  7 |   return <p>画面の幅は {width}px です</p>;
  8 | }
  9 | {
  digest: '477784383'
}
 GET /ng-window 500 in 60ms (next.js: 2ms, application-code: 58ms)
```

```tsx
// ❌ NG: app/ng-window/WindowWidth.tsx（描画中に window を読んでいる）
"use client";

export function WindowWidth() {
  // ブラウザの画面幅を読む
  const width = window.innerWidth;
  return <p>画面の幅は {width}px です</p>;
}
```

```tsx
// ✅ OK: app/ok-window/WindowWidth.tsx（window は useEffect の中で読む）
"use client";

import { useEffect, useState } from "react";

export function WindowWidth() {
  // 最初は「まだ分からない」を表す null にしておく（サーバーでも同じ値になる）
  const [width, setWidth] = useState<number | null>(null);

  // useEffect の中はブラウザでだけ実行されるので window を安全に使える
  useEffect(() => {
    setWidth(window.innerWidth);
  }, []);

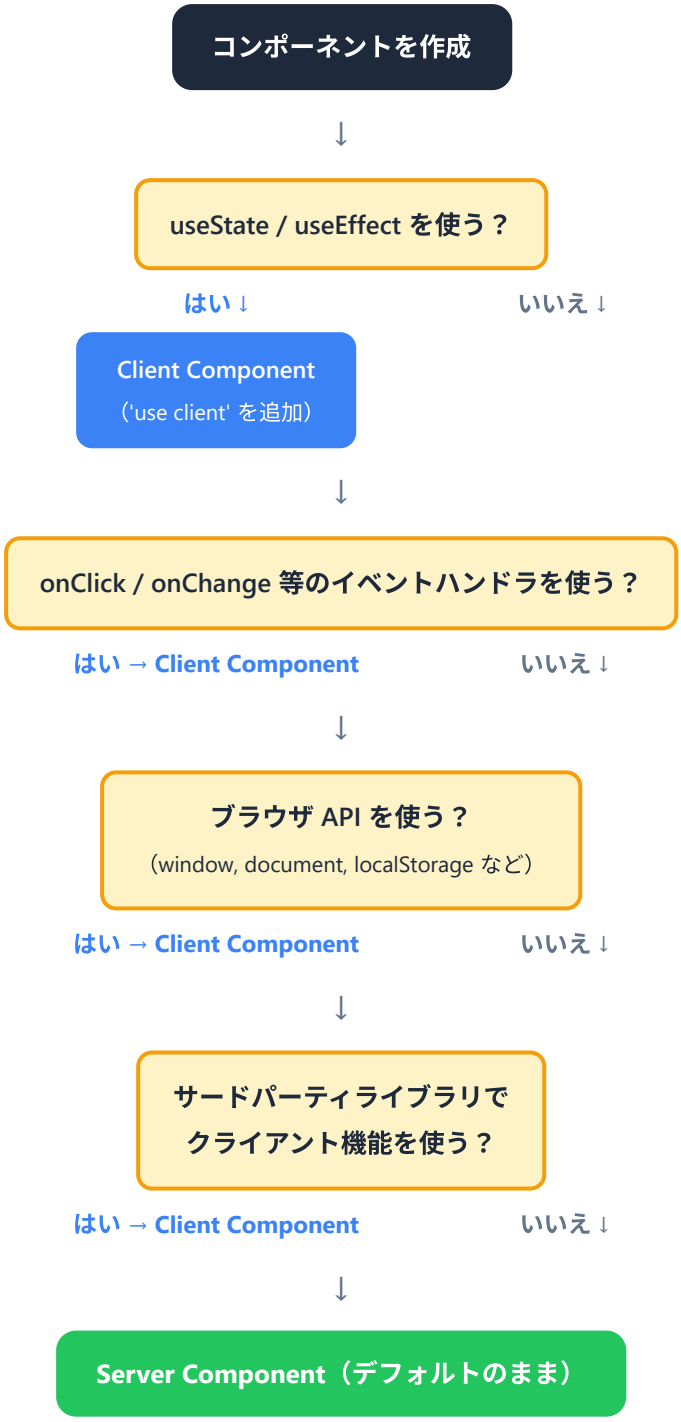
  if (width === null) {
    return <p>画面の幅を調べています...</p>;
  }
  return <p>画面の幅は {width}px です</p>;
}
```

OK 版を幅 900px のブラウザで開くと、表示は次のようになります（最初の一瞬だけ「調べています...」が出て、すぐ切り替わります）。

```
<p>画面の幅は 900px です</p>
```

> `window`・`document`・`localStorage`・`navigator` など「ブラウザにしか無い物」は、**`useEffect` の中か、クリックなどのイベントの中**で使う、と覚えておきましょう。どちらもブラウザでしか実行されない場所です。

3.3 いつどちらを使うか（判断フローチャート）



原則: できるだけ Server Component を使い、インタラクティブ性が必要な部分だけを Client Component にします。

用途	Server Component	Client Component
データの取得・表示	**推奨**	
データベースへの直接アクセス	**推奨**	不可
機密情報（APIキーなど）へのアクセス	**推奨**	不可
静的な UI の描画	**推奨**	
フォーム入力		**必須**
クリックイベント		**必須**
useState / useEffect		**必須**
ブラウザ API（localStorage 等）	不可	**必須**

つまずきポイント（エラーが出たから、とページ全体に `"use client"` を付ける）：ボタンを1つ置きたいだけなのに、`useState` のエラーが出たからといってページ（ `page.tsx` ）の先頭に `"use client"` を付けてしまうのは、よくある遠回りです。ページ全体が Client Component になると、① `async` にしてデータを直接取得できなくなる（エラー2）、② `metadata` を書けなくなる（エラー4）、③ページ全体の JavaScript がブラウザに送られて重くなる、と損ばかりです。`"use client"` は**木の葉っぱ（いちばん末端の小さな部品）にだけ**付けるのがコツです。

```tsx
// ❌ NG: app/books/page.tsx（ボタン1つのためにページ全体を Client Component にした）
"use client";

import { useState } from "react";

export default function BooksPage() {
  const [likes, setLikes] = useState(0);
  // Client Component なので、ここで await を使ったデータ取得はできない
  return (
    <div>
      <h1>書籍一覧</h1>
      <button onClick={() => setLikes(likes + 1)}>いいね {likes}</button>
    </div>
  );
}
```

```tsx
// ✅ OK: app/books/page.tsx（ページは Server Component のまま）
import { LikeButton } from "./LikeButton";

export default async function BooksPage() {
  // Server Component なので、ここでデータ取得もできる
  return (
    <div>
      <h1>書籍一覧</h1>
      {/* 動きが必要な部分だけを Client Component に切り出す */}
      <LikeButton />
    </div>
  );
}
```

```tsx
// ✅ OK: app/books/LikeButton.tsx（"use client" はこの小さな部品にだけ付ける）
"use client";

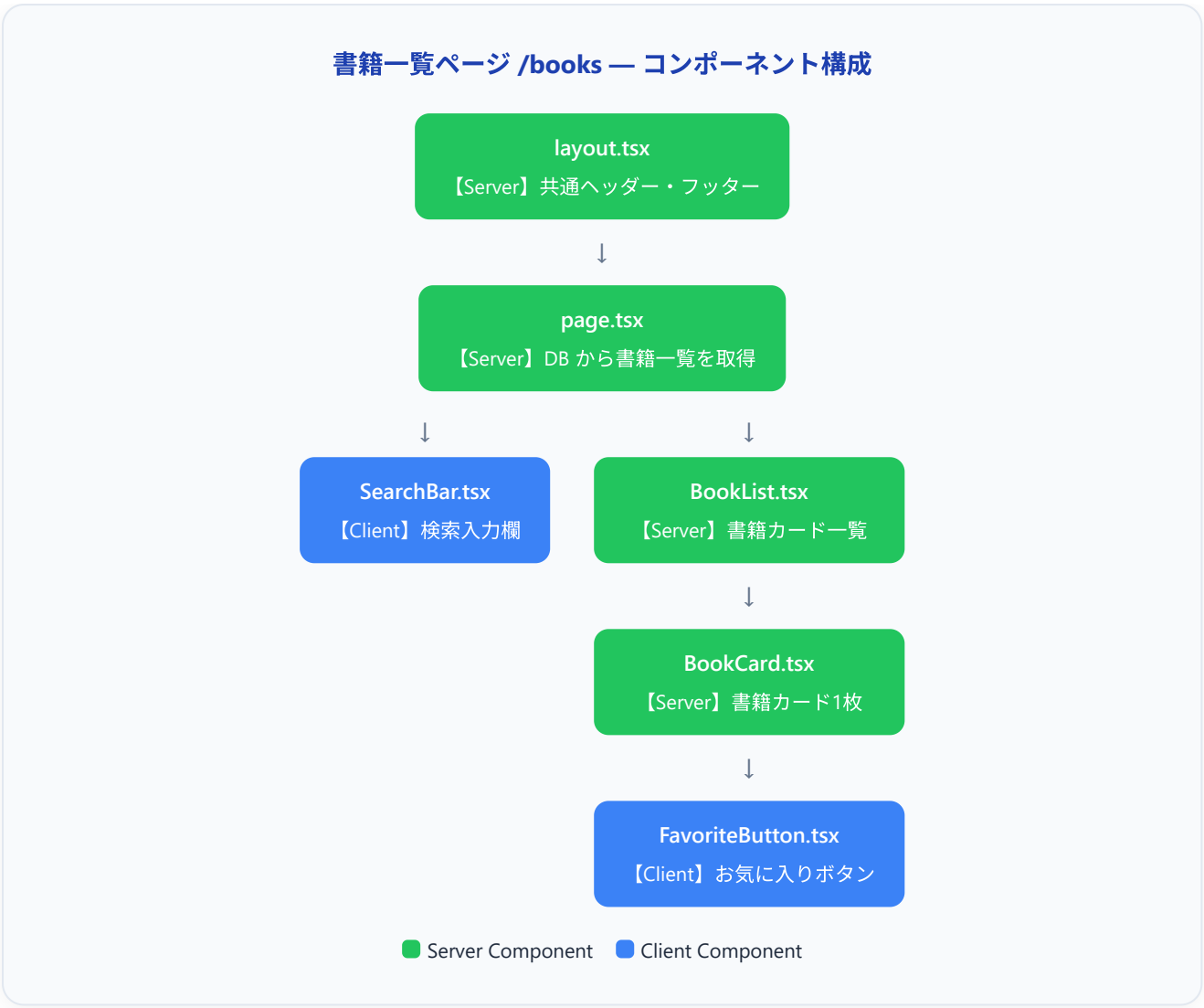
import { useState } from "react";

export function LikeButton() {
  const [likes, setLikes] = useState(0);
  return <button onClick={() => setLikes(likes + 1)}>いいね {likes}</button>;
}
```

> 見た目はどちらも同じですが、OK 版ではブラウザに送られる JavaScript が `LikeButton` の分だけで済み、ページ本体ではデータ取得や `metadata` がそのまま使えます。

3.4 書籍管理アプリでの使い分け例

書籍管理アプリの具体的なページで、どのようにコンポーネントを分割するか見てみましょう。



▼ **このコードがやること（先に日本語で）**：書籍一覧ページ本体を Server Component として作り、サーバー上で書籍データを取得してから画面を組み立てます。ポイントは「Server の骨組み（このページ）の中に、Client の部品（検索バー）と Server の部品（一覧表示）を組み合わせる」という App Router の基本パターンです。取得した `books` を子コンポーネントに props（受け渡しデータ）として渡します。詳細はコード内コメントを参照してください。

```tsx
// `/books` のページ本体。"use client" なしなので Server Component
// app/books/page.tsx（Server Component）
// データベースから直接書籍一覧を取得する

// `@/` はプロジェクトルートを示すエイリアス
// ../../ のような相対パス地獄を避けるための設定（tsconfig.json で定義）
import { SearchBar } from "@/components/SearchBar";
// 書籍リストを描画する別の自作コンポーネントを取り込む
import { BookList } from "@/components/BookList";

// `async` 関数なので本体で `await` が使える
// この関数自体が Server Component
export default async function BooksPage() {
  // Server Component なので、サーバー上で直接データ取得ができる
  // この処理はブラウザには送信されない
  // `fetch` は Web標準のHTTP通信API
  // Next.js は fetch を拡張してキャッシュ機能を追加している
  const response = await fetch("https://api.example.com/books", {
    // 常に最新データを取得（SSR）
    // `"no-store"` でキャッシュを使わず毎回サーバーからデータ取得
    // 他に "force-cache"（SSG的）、`next: { revalidate: N }`（ISR的）がある
    cache: "no-store",
  });
  // `response.json()` で本文JSONをJSオブジェクトに変換
  const books = await response.json();

  return (
    <div>
      {/* ページタイトル */}
      <h1>書籍一覧</h1>
      {/* JSX 内のコメント記法 */}
      {/* Client Component: ユーザーの入力を受け付ける */}
      {/* 検索バー（Client）。ここから下は子コンポーネント */}
      <SearchBar />
      {/* Server Component: 取得したデータを表示する */}
      {/* 取得した books 配列を props として渡す */}
      {/* `books={books}` は「booksというpropに変数booksの値を渡す」の意 */}
      <BookList books={books} />
    </div>
  );
}
```

▼ このコードがやること（先に日本語で）：入力した文字でページを検索結果に切り替える検索バーを作ります。Client Component なので入力状態を `useState` で覚えつつ、`useRouter`（ページ移動用の道具）で検索キーワード付きの URL に移動します。日本語や空白は URL にそのまま使えないため `encodeURIComponent` で変換する点も押さえてください。詳細はコード内コメントを参照してください。

```tsx
// 再利用される検索バー部品
// components/SearchBar.tsx（Client Component）
// この行がある＝このファイルは Client Component
// ブラウザ側で動き、useState やイベントが使える
"use client";

// React の状態管理フック
import { useState } from "react";
// App Router 用のルーターフック
// ※ `next/router` ではない！（あちらは旧 Pages Router 用）
import { useRouter } from "next/navigation";

// 名前付き export。`{ SearchBar }` で import される
export function SearchBar() {
  // `useState` で「現在の検索文字列」を管理
  // 配列の分割代入: [現在値, 更新関数] = useState(初期値)
  const [query, setQuery] = useState("");
  // ルーター操作用オブジェクトを取得（push/replace/back/refresh等が使える）
  const router = useRouter();

  // フォーム送信時のハンドラ。型は React.FormEvent
  const handleSearch = (e: React.FormEvent) => {
    // ブラウザ標準のフォーム送信（ページ全体リロード）を止める
    e.preventDefault();
    // 検索クエリ付きで遷移
    // `router.push` で別URLに移動（履歴に追加）
    // `encodeURIComponent` はURLに使えない文字（空白や日本語）を %エンコード
    router.push(`/books?q=${encodeURIComponent(query)}`);
  };

  return (
    // submit イベントを上のハンドラに紐付け
    <form onSubmit={handleSearch}>
      <input
        // テキスト入力欄
        type="text"
        // 制御コンポーネント：表示値を React state と同期
        value={query}
        // 入力のたびに state を更新（再描画される）
        onChange={(e) => setQuery(e.target.value)}
        // 空欄時の薄いガイドテキスト
        placeholder="タイトルで検索..."
      />
      {/* submit ボタン。クリックで form の onSubmit が走る */}
      <button type="submit">検索</button>
    </form>
  );
}
```

▼ コードを1つずつ分解して解説

この検索バーには「`useRouter` でのプログラム的なページ移動」「`encodeURIComponent` でのURL変換」という、Next.js特有の要素があります。順に見ていきましょう。

解説1: `useRouter` を取り込むのは `next/navigation` から

```
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
```

- `useRouter`（ユーズ・ルーター）は「**プログラムからページを移動する**」ための道具（フック）です。ボタンを押した後やフォーム送信後に、コードで「○○ページへ飛べ」と指示できます。
- **取り込み元が `next/navigation` である点が最重要です。**よく似た `next/router` は古い Pages Router 用で、App Router では使えません（使うとエラーになります）。
- `useRouter` はブラウザ側の機能なので、ファイル先頭に `"use client"` が必須です。

> **用語: フック（hook）** … React で `use○○` という名前の特別な関数の総称。`useState`（状態）、`useRouter`（ページ移動）のように、コンポーネントに機能を「引っ掛けて」使います。

解説2: フォーム送信を受け取って検索結果ページへ移動する

```
  const router = useRouter();

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    router.push(`/books?q=${encodeURIComponent(query)}`);
  };
```

- `const router = useRouter()` で、ページ移動に使う「ルーター」を取り出します。
- `e.preventDefault()`（プリベント・デフォルト）は、フォームの「送信するとページ全体が再読み込みされる」という昔ながらの動作を**止める**おまじないです。これを書かないと画面がリロードされてしまいます。
- `router.push(...)` で、指定したURLへ移動します。`push` は「ブラウザの履歴に1つ追加しながら移動」なので、移動後に「戻る」ボタンで元のページに戻れます。

> **用語: preventDefault** … イベントの「ブラウザ標準の振る舞い」を打ち消すメソッド。フォーム送信では「ページのリロードを止める」ために、ほぼ必ず最初に書きます。

解説3: `encodeURIComponent` でURLに使えない文字を変換する

```
router.push(`/books?q=${encodeURIComponent(query)}`);
```

- バッククォート（ `` ` `` ）で囲んだ文字列は **テンプレートリテラル**で、`${...}` の中に変数や式を埋め込めます。ここでは `/books?q=検索語` というURLを組み立てています。
- `encodeURIComponent(query)` （エンコード・ユーアールアイ・コンポーネント）は、「**URLにそのまま使えない文字（空白・日本語・記号など）を、安全な形に変換する**」関数です。例えば空白は `%20` 、日本語は `%E3%...` のような形に変換されます。
- これを通さないと、検索語に空白や日本語が含まれたときにURLが壊れてしまいます。`?q=` の後ろのような「URLに値を載せる」場面では必ず通す習慣をつけましょう。

> **用語: クエリパラメータ** … URLの `?` 以降に「キー=値」の形で付ける追加情報（例: `?q=react` ）。検索条件やページ番号などを次のページへ渡すのに使います。

> **▼ このコードがやること（先に日本語で）**：書籍1冊分の「カード」表示を Server Component で作ります。タイトルや著者など動かない部分はサーバーで描画し、クリックが必要な「お気に入りボタン」だけを Client Component として中に埋め込みます。`type Book = {...}` は「このオブジェクトはこういう形」と決める TypeScript の型定義で、入力ミスを防いでくれます。詳細はコード内コメントを参照してください。

```tsx
// 書籍1冊分のカード。静的表示部分は Server で
// components/BookCard.tsx（Server Component）

// ※同じフォルダから import するときは "./" で始める
// ここで Client Component を取り込んでも、子だけが Client になる
import { FavoriteButton } from "./FavoriteButton";

// TypeScript の型定義（=この形のオブジェクト）
type Book = {
  // 書籍ID（文字列）
  id: string;
  // タイトル
  title: string;
  // 著者
  author: string;
};

// props を分割代入で受け取り、型は `{ book: Book }`
export function BookCard({ book }: { book: Book }) {
  return (
    // カード枠（CSSでスタイル付け）
    <div className="book-card">
      {/* タイトル表示 */}
      <h3>{book.title}</h3>
      {/* 著者表示 */}
      <p>{book.author}</p>
      {/* インタラクティブな部分だけ Client Component */}
      {/* お気に入りボタンには id だけ渡す（最小限のpropsで境界を作る） */}
      <FavoriteButton bookId={book.id} />
    </div>
  );
}
```

▼ このコードがやること（先に日本語で）：クリックで「お気に入り」の ON/OFF を切り替えるボタンを作ります。Client Component なので、状態を `useState` で覚え、クリックされたらまず画面表示をすぐ切り替え（楽観的更新）、その後サーバーに `fetch` で保存依頼を送ります。ボタンの文字も状態に応じて「☆」と「★」で切り替わります。詳細はコード内コメントを参照してください。

```tsx
// クリック可能なお気に入りボタン
// components/FavoriteButton.tsx（Client Component）
// この1行で Client Component 化
"use client";

// 状態管理用のフック
import { useState } from "react";

// props は `{ bookId: string }`
export function FavoriteButton({ bookId }: { bookId: string }) {
  // 「お気に入りか？」の真偽値state。初期値は false
  const [isFavorite, setIsFavorite] = useState(false);

  // クリック時のハンドラ。async で await が使える
  const handleClick = async () => {
    // まず画面上の状態をトグル（楽観的更新）
    // `!isFavorite` は「現在の値の反対」
    setIsFavorite(!isFavorite);
    // API を呼び出してお気に入り状態を保存
    // テンプレートリテラル（バッククォート ``）で URL を組み立てる
    await fetch(`/api/books/${bookId}/favorite`, {
      // HTTP メソッド: POST（新しい状態を送る）
      method: "POST",
      // JSオブジェクトをJSON文字列に変換してリクエストボディに乗せる
      body: JSON.stringify({ favorite: !isFavorite }),
    });
  };

  return (
    // ボタンクリックで handleClick を発動
    <button onClick={handleClick}>
      {/* 三項演算子: state によって表示を切り替える */}
      {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}
    </button>
  );
}
```

▼ コードを1つずつ分解して解説

このお気に入りボタンには「楽観的更新」「`fetch` でのサーバー保存」「三項演算子での表示切替」という、よく使うパターンが詰まっています。順に見ていきましょう。

解説1: クリックされたら「まず画面を切り替えてから」サーバーに送る

```
const [isFavorite, setIsFavorite] = useState(false);

const handleClick = async () => {
  setIsFavorite(!isFavorite);
  // API を呼び出してお気に入り状態を保存
  await fetch(`/api/books/${bookId}/favorite`, {
    method: "POST",
    body: JSON.stringify({ favorite: !isFavorite }),
  });
};
```

- `isFavorite` は「お気に入りかどうか」を表す `true` / `false` の状態です。初期値は `false` 。
- `setIsFavorite(!isFavorite)` の `!` （ビックリマーク）は「**反対の値にする**」記号です。`true` なら `false` に、`false` なら `true` に切り替わります。
- 注目してほしいのは順番です。**まず先に `setIsFavorite` で画面の見た目を切り替え、その後で `await fetch(...)` でサーバーに保存依頼を送っています。**こうすると、サーバーの返事を待たずにボタンの見た目が即座に変わるので、ユーザーは「サクサク反応する」と感じます。これを **楽観的更新**（optimistic update） と呼びます。

> **用語: 楽観的更新（optimistic update）** ...「サーバー処理はどうせ成功するだろう」と楽観的に考え、サーバーの返事を待たずに先に画面を更新する手法。体感速度が上がります（失敗時は元に戻す処理を足すこともあります）。

解説2: `fetch` でサーバーに「お気に入り状態」を送る

```
  await fetch(`/api/books/${bookId}/favorite`, {
    method: "POST",
    body: JSON.stringify({ favorite: !isFavorite }),
  });
```

- `fetch` （フェッチ）は「**サーバーと通信する**」ためのブラウザ標準の関数です。第1引数が送り先のURL、第2引数が送り方の設定です。
- URLはテンプレートリテラル（ `` ` `` ）で組み立てており、`${bookId}` の部分に、props で受け取った書籍IDが入ります。
- `method: "POST"` は「データを**送って保存する**ときの通信方法」です（単に取得するだけなら `GET` ）。
- `body: JSON.stringify({ favorite: !isFavorite })` は、送る中身（本体）です。`JSON.stringify(...)` は「**JavaScriptのオブジェクトを、通信で送れる文字列（JSON）に変換する**」関数です。

> **用語: JSON.stringify** … JavaScriptのオブジェクトや配列を「JSON形式の文字列」に変換するメソッド。サーバーへデータを送るときは、この文字列の形にしてから送ります。

解説3: 三項演算子でボタンの文字を切り替える

```
    <button onClick={handleClick}>
      {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}
    </button>
```

- `onClick={handleClick}` で、ボタンがクリックされたら上の `handleClick` 関数が呼ばれるように紐付けています。
- `{isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}` は **三項演算子**です。「`条件 ? 真のときの値 : 偽のときの値`」の形で、`isFavorite` が `true` なら「★ お気に入り済み」、`false` なら「☆ お気に入り」を表示します。
- `isFavorite` の値が切り替わるたびにReactが再描画するので、ボタンの文字も自動で「☆」と「★」が入れ替わります。

> **用語: 三項演算子（ternary operator）** … `条件 ? A : B` という形で「条件に応じてAかBを返す」式。`if` 文と違って「値を返す式」なので、JSXの `{ }` の中に直接書けます。

画面にはこう表示される: ページ上部に検索バー、その下に書籍カードが並んで表示されます。各カードには書籍タイトル、著者名、お気に入りボタンがあります。検索バーに文字を入力するとリアルタイムに反映され、お気に入りボタンをクリックすると「☆ お気に入り」が「★ お気に入り済み」に切り替わります。

3.5 サーバーからクライアントへ渡せるもの（props の境界）

3.4 の例では、Server Component の `BookCard` が Client Component の `FavoriteButton` に `bookId={book.id}`（文字列）を渡していました。このように、サーバー側からブラウザ側の部品へ props を渡すときには、1つ大事な決まりがあります。**渡せるのは「文字にして運べるもの」だけ**です。

たとえるなら、`"use client"` の境目は**国境の関所（税関）**です。サーバーで作った props は、いったん文字の形（シリアライズ：serialize＝運べる形に変換すること）にしてブラウザへ送られます。手紙やメモ（文字列・数値・データの表）は通れますが、「その場で動く機械」（ふつうの関数）は文字にできないので通れません。

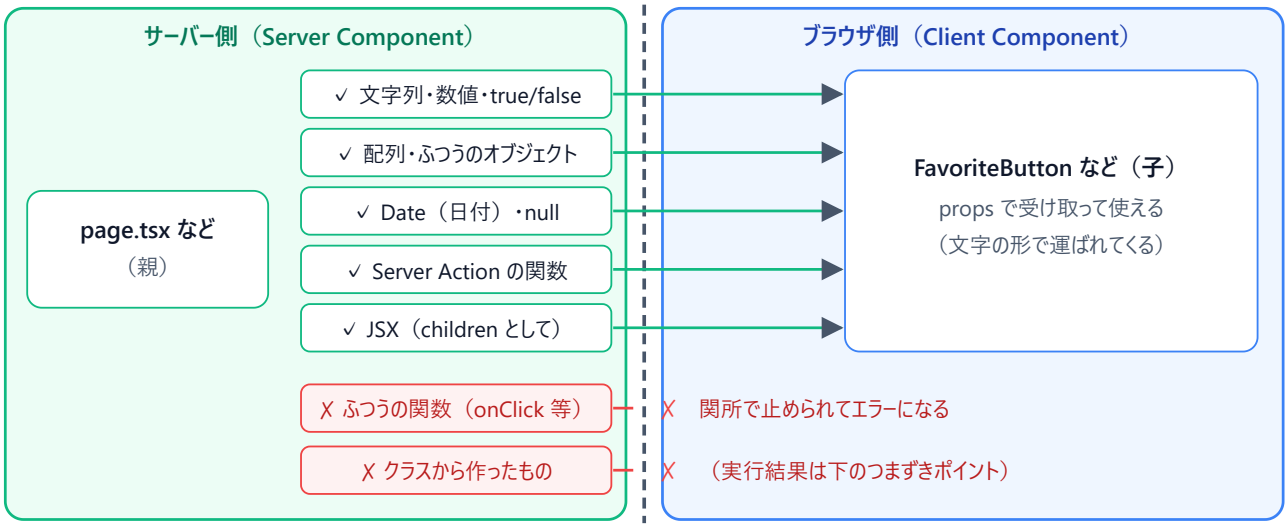


図4-5: サーバーからクライアントへ props で渡せるもの・渡せないもの

- **文字列・数値・配列・ふつうのオブジェクト・`Date` などは、そのまま渡せます。**データベースから取ってきた書籍データ（`{ id, title, author }` のようなオブジェクト）を渡すのは、いちばんよくある使い方です。
- **Server Action**（7節で学ぶ `"use server"` の関数）は、関数ですが特別に渡せます。中身はサーバーに残ったまま、「呼び出すための番号札」だけがブラウザへ運ばれるからです。
- **JSX を `children` として渡す**こともできます。たとえば Client Component の開閉パネルの中に、サーバーで作った一覧を入れられます（サーバーで作り終えた「見た目」として運ばれます）。
- **ふつうの関数（`onClick={() => ...}` など）は渡せません。**クリックされたときの処理は、Client Component **の中で**作ります。

> **注意:** 渡せるからといって、データベースの接続情報や秘密のキーを props で Client Component に渡してはいけません。props はブラウザに送られるので、開発者ツールで誰でも見られます（8節）。

> **つまずきポイント（Server Component から Client Component に関数を渡す）:** React に慣れていると、親から子へ `onDelete={() => ...}` のように関数を渡したくなります。親が Server Component で子が Client Component の場合、これは関所を通れないので、ページを開いた瞬間にエラーになります。`npm run dev` で開いたときのターミナルの実際の出力です。

```
× Error: Event handlers cannot be passed to Client Component props.
  <... title=... onDelete={function onDelete}>
                          ^^^^^^^^^^^^^^^^^^^
If you need interactivity, consider converting part of this to a Client Component.
    at ignore-listed frames {
  digest: '1517905263'
}
 GET /ng-func 500 in 64ms (next.js: 2ms, application-code: 62ms)
```

```tsx
// ❌ NG: app/ng-func/page.tsx（Server Component から関数を渡している）
import { DeleteButton } from "./DeleteButton";

export default function NgFuncPage() {
  return (
    <DeleteButton
      title="リーダブルコード"
      onDelete={() => console.log("削除しました")}
    />
  );
}
```

```tsx
// ❌ NG: app/ng-func/DeleteButton.tsx（関数を props で受け取るつもりの子）
"use client";

export function DeleteButton({
  title,
  onDelete,
}: {
  title: string;
  onDelete: () => void;
}) {
  return <button onClick={onDelete}>「{title}」を削除</button>;
}
```

OK 版では、親から渡すのは文字列だけにして、クリックしたときの処理は子（Client Component）の中で作ります。サーバーでやりたい処理（データベースからの削除など）は Server Action にして、子から呼び出します。

```tsx
// ✅ OK: app/ok-func/actions.ts
"use server";

export async function deleteBook(title: string) {
  // 本物のアプリならここでデータベースから削除する
  console.log(`[deleteBook] 「${title}」を削除しました`);
}
```

```tsx
// ✅ OK: app/ok-func/page.tsx（Server Component）
import { DeleteButton } from "./DeleteButton";

export default function OkFuncPage() {
  // 渡すのは「文字列」だけ。関数は渡さない
  return <DeleteButton title="リーダブルコード" />;
}
```

```tsx
// ✅ OK: app/ok-func/DeleteButton.tsx（Client Component）
"use client";

import { deleteBook } from "./actions";

export function DeleteButton({ title }: { title: string }) {
  // クリックされたときの処理は Client Component の中で作る
  const handleClick = async () => {
    await deleteBook(title);
  };

  return <button onClick={handleClick}>「{title}」を削除</button>;
}
```

OK 版でボタンを押したときの、ターミナルの実際の出力です。最後の行は、Next.js 16 の開発サーバーが「どの Server Action が、どんな引数で呼ばれたか」を表示してくれているものです。

```
[deleteBook] 「リーダブルコード」を削除しました
 POST /ok-func 200 in 14ms (next.js: 1.8ms, application-code: 12ms)
  └─ ƒ deleteBook("リーダブルコード") in 1ms app/ok-func/actions.ts
```

> エラー文の `Event handlers cannot be passed to Client Component props`（イベント処理の関数は Client Component の props に渡せない）は、まさに「関所で止められた」という意味です。`onClick` のような名前でなくても、ふつうの関数を渡すと `Functions cannot be passed directly to Client Components ...` という似たエラーになります。また、`class Book { ... }` のようなクラスから `new` で作ったものを渡すと、`Only plain objects, and a few built-ins, can be passed to Client Components from Server Components.`（ふつうのオブジェクトと一部の組み込みの型しか渡せない）というエラーになります。どちらも実際に試して確認したエラー文です。

---

4. レイアウトとテンプレート

4.1 ルートレイアウト

`app/layout.tsx` は **ルートレイアウト** と呼ばれ、アプリケーション全体に適用されます。`<html>` タグと `<body>` タグを含む **唯一の** レイアウトです。

▼ **このコードがやること（先に日本語で）**：アプリ全体の一番外側の枠（ヘッダー・本文・フッター）と、ブラウザタブのタイトルなどの基本情報（ `metadata` ）をまとめて定義します。 `metadata` を export すると Next.js が `<head>` を自動で組み立ててくれるので、自分で `<title>` を書く必要がありません。Google Fonts の最適化読み込みも含む、実用的なルートレイアウトの完成形です。詳細はコード内コメントを参照してください。

```tsx
// 予約名 `layout.tsx`。`app/` 直下のものは「ルートレイアウト」と呼ばれる
// app/layout.tsx
// すべてのページを包む一番外側の枠になる（必須）

// `import type` は「型だけ取り込む」記法
// 実行時のJSには残らないのでバンドルが軽くなる
import type { Metadata } from "next";
// Google Fonts の Inter フォントを Next.js 経由で読み込む
// `next/font/google` はビルド時にフォントをセルフホストして高速化してくれる
import { Inter } from "next/font/google";
// 全ページに適用する CSS をインポート
// 値として使わず副作用のためだけに読み込む書き方
import "./globals.css";

// Google Fonts の Inter フォントを最適化して読み込む
// フォントのインスタンスを作る
// `subsets: ["latin"]` は「ラテン文字だけ含む」指定でサイズ削減
const inter = Inter({ subsets: ["latin"] });

// `metadata` という名前で export すると Next.js が <head> を自動生成
// 自分で <title> や <meta> を書く必要がない
export const metadata: Metadata = {
  title: {
    // 子ページのタイトルに自動で「 | 書籍管理アプリ」を付ける
    // `%s` の部分が子の title に置き換わる
    template: "%s | 書籍管理アプリ",
    // 子ページが title を指定しなかったときのデフォルト
    default: "書籍管理アプリ",
  },
  // <meta name="description"> として出力される。SEOで重要
  description: "あなたの読書記録を管理するアプリケーション",
};

// ルートレイアウトの本体関数。名前は慣習で `RootLayout`
export default function RootLayout({
  // children: このレイアウトに包まれる「各ページのJSX」
  children,
}: {
  // `React.ReactNode` は「JSX/文字列/数値/null など描画できる何でも」を表す型
  children: React.ReactNode;
}) {
  return (
    // `<html>` はルートレイアウトでだけ書く
    // lang="ja" は日本語ページであることをブラウザや支援技術に伝える
    <html lang="ja">
      {/* `<body>` もルートレイアウト専用 */}
      {/* `inter.className` でフォント用 CSS クラスを適用 */}
      <body className={inter.className}>
        {/* ここにヘッダーを配置 */}
```

```
        {/* セマンティック要素 `<header>`：ページ上部のヘッダー領域 */}
        <header className="site-header">
          {/* `<nav>`：ナビゲーション領域を示すセマンティック要素 */}
          <nav>
            {/* 普通のアンカー。ロゴ的リンク（後で `<Link>` に置き換えるのがベター） */}
            <a href="/">書籍管理アプリ</a>
          </nav>
        </header>

        {/* children に各ページの内容が入る */}
        {/* `<main>`：ページの主要コンテンツ領域 */}
        {/* ここに `app/page.tsx` や `app/books/page.tsx` 等の中身が差し込まれる */}
        <main className="site-main">{children}</main>

        {/* ここにフッターを配置 */}
        {/* `<footer>`：ページ下部のフッター領域 */}
        <footer className="site-footer">
          {/* `&copy;` は © の HTML エンティティ（特殊文字） */}
          <p>&copy; 2024 書籍管理アプリ</p>
        </footer>
      </body>
    </html>
  );
}
```

ポイント:

- ルートレイアウトは **必須** です。削除するとエラーになります。
- `<html>` と `<body>` タグはルートレイアウトにのみ記述します。
- `metadata` オブジェクトで SEO 用のタイトル・説明を設定できます。
- `template: "%s | 書籍管理アプリ"` により、子ページのタイトルが「ページ名 | 書籍管理アプリ」の形式になります。

4.2 ネストされたレイアウト

各ルートセグメント（フォルダ）にも `layout.tsx` を置くことができ、そのセグメント以下のページに適用されます。



▼ このコードがやること（先に日本語で）： `/books` 以下のページだけに共通で付くサイドバー付きの枠（ネストされたレイアウト）を作ります。ルートレイアウトの内側にさらに重ねられるのがポイントで、トップページには出ず `/books` 配下でだけサイドバーが表示されます。ルート以外のレイアウトでは `<html>` / `<body>` は書かない点に注意してください。詳細はコード内コメントを参照してください。

```tsx
// `app/books/` フォルダの中の `layout.tsx`
// app/books/layout.tsx
// `/books`, `/books/new`, `/books/[id]` 等で共通の枠になる
// /books 以下のすべてのページに適用されるレイアウト

// 型だけ import（Metadata 型を使うため）
import type { Metadata } from "next";

// この階層用のメタデータ
export const metadata: Metadata = {
  // 親（RootLayout）の template と組み合わさり「書籍管理 | 書籍管理アプリ」になる
  title: "書籍管理",
};

// ネストされたレイアウトの本体関数
export default function BooksLayout({
  // children: この階層の page.tsx またはさらに深い layout.tsx の中身
  children,
}: {
  // 描画可能な何でも、を意味する型
  children: React.ReactNode;
}) {
  return (
    // ※ルート以外の layout.tsx は <html>/<body> を書かない
    <div className="books-layout">
      {/* サイドバー */}
      {/* `<aside>`：補足コンテンツ用のセマンティック要素 */}
      <aside className="sidebar">
        {/* ナビゲーション領域 */}
        <nav>
          {/* 箇条書きリスト */}
          <ul>
            {/* ※ 後ほど `<Link>` への置き換えを学ぶ */}
            <li><a href="/books">書籍一覧</a></li>
            <li><a href="/books/new">書籍を追加</a></li>
            <li><a href="/books/favorites">お気に入り</a></li>
          </ul>
        </nav>
      </aside>

      {/* メインコンテンツ */}
      {/* `<section>`：意味的にひとまとまりの領域 */}
      <section className="content">
        {/* ここに /books/page.tsx や /books/[id]/page.tsx の内容が入る */}
        {children}
      </section>
    </div>
```

```
  );
}
```

画面にはこう表示される:



`/books` や `/books/123` にアクセスすると、ルートレイアウトのヘッダー・フッターに加えて、書籍セクション専用のサイドバーが表示されます。トップページ（`/`）にはサイドバーは表示されません。

レイアウトは「**箱の中に箱を入れる**」ように重なります。ロシアの人形マトリョーシカのように、外側の箱（ルートレイアウト）の中に、`/books` 用の箱（books のレイアウト）が入り、そのさらに中にページ本体が入ります。そして、ページを移動しても外側の箱は**作り直されません**。

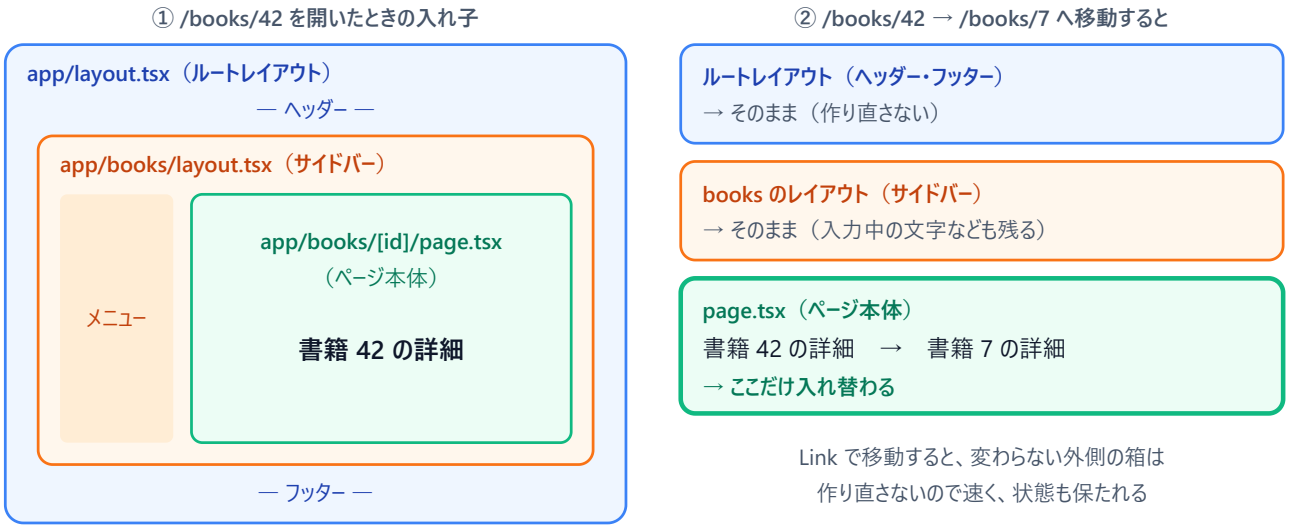


図4-6: レイアウトは入れ子の箱。ページ移動で入れ替わるのは中身だけ

レイアウトの入れ子をログと HTML で確かめる

> **▼ このコードがやること（先に日本語で）**：ルートレイアウト・`/books` のレイアウト・詳細ページの3つに `console.log` を書いて、`/books/42` を開いたときに**どの順番で、どれが実行されるか**をターミナルで確かめます。さらに、ブラウザに届いた HTML を見て、3つの部品が本当に「箱の中に箱」の形に組み合わさっていることを確認します。

```
// app/layout.tsx（ルートレイアウト。実験用に console.log を足したもの）
import type { Metadata } from "next";
import Link from "next/link";
import "./globals.css";

export const metadata: Metadata = {
  title: "実験用アプリ",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  console.log("[RootLayout] サーバーで描画しました");
  return (
    <html lang="ja">
      <body>
        <header>
          <Link href="/">ホーム</Link>
        </header>
        <main>{children}</main>
      </body>
    </html>
  );
}
```

```tsx
// app/books/layout.tsx（/books 以下の全ページを包む枠）
export default function BooksLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  console.log("[BooksLayout] サーバーで描画しました");

  return (
    <div className="books-layout">
      <aside>書籍メニュー</aside>
      {/* children の位置に page.tsx の中身が入る */}
      <section>{children}</section>
    </div>
  );
}
```

```tsx
// app/books/[id]/page.tsx（/books/42 などで表示されるページ）
export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  console.log(`[BookDetailPage] id=${id} のページを描画しました`);

  return <h1>書籍 {id} の詳細</h1>;
}
```

▼ **実行結果**（`npm run dev` で起動し、`http://localhost:3000/books/42` を開いたとき）

ターミナルの出力:

```
[RootLayout] サーバーで描画しました
[BooksLayout] サーバーで描画しました
[BookDetailPage] id=42 のページを描画しました
 GET /books/42 200 in 46ms (next.js: 12ms, application-code: 35ms)
```

ブラウザに表示された HTML（開発者ツールの Elements タブで `<body>` の中を見たもの。Next.js が内部で使う `<script>` などは省略）:

```
<header><a href="/">ホーム</a></header><main><div class="books-layout"><aside>書籍メニュー
</aside><section><h1>書籍 42 の詳細</h1></section></div></main>
```

1行だと読みにくいので、改行と字下げを入れて整えると次のとおりです。

```
<header><a href="/">ホーム</a></header>
<main>
  <div class="books-layout">
    <aside>書籍メニュー</aside>
    <section>
      <h1>書籍 42 の詳細</h1>
    </section>
  </div>
</main>
```

▼ コードを1つずつ分解して解説

解説1: 外側から順に、全部サーバーで実行される

- ターミナルに3行のログが「ルート → books → ページ」の順で出ています。どれも Server Component なので、**サーバーで**実行されています（ブラウザには HTML だけが届きます）。
- `/books/42` という1つの URL を開いただけで、URL に関係する `layout.tsx` が**全部**使われることが分かります。`app/layout.tsx` は全ページ、`app/books/layout.tsx` は `/books` 以下のページで必ず使われます。

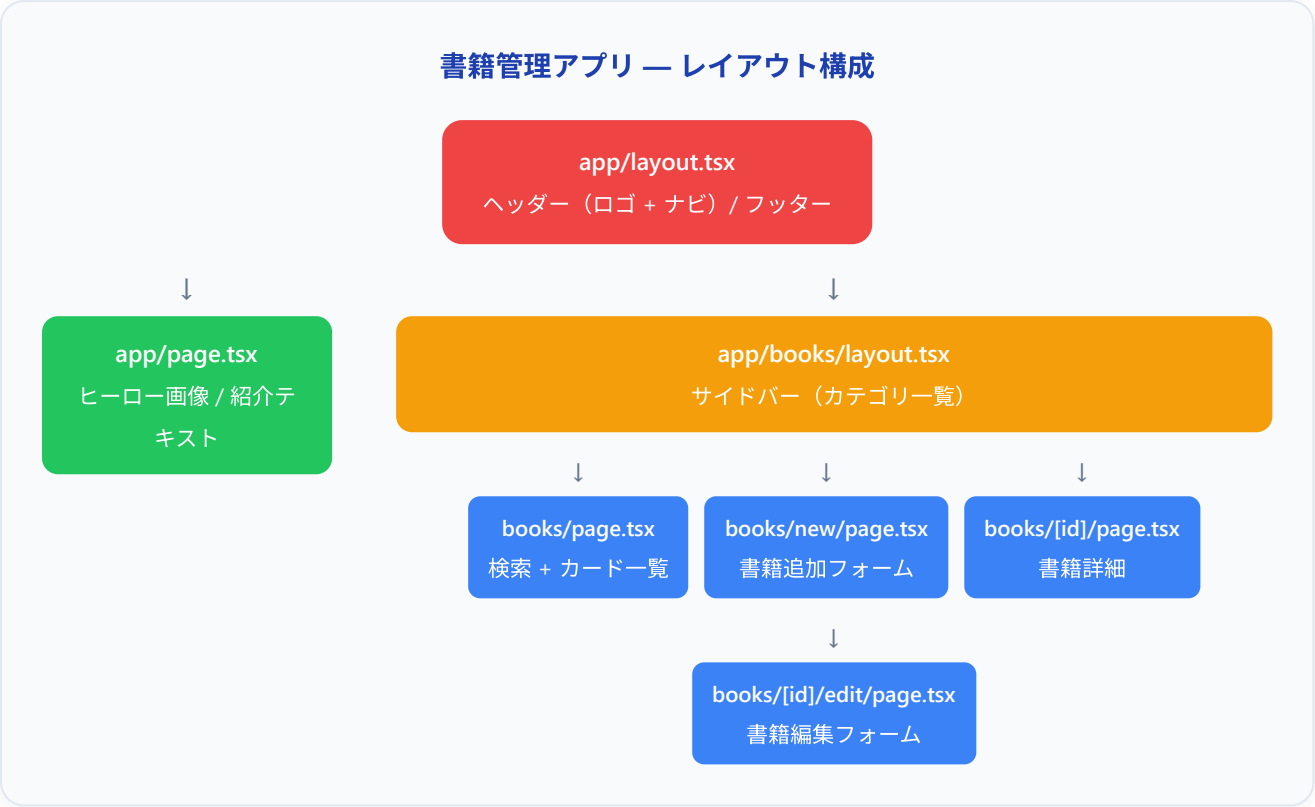
解説2: `{children}` の位置に、内側の箱がはめ込まれる

```
      <section>{children}</section>
```

- HTML を見ると、`<main>`（ルートレイアウトの `{children}` の場所）の中に books のレイアウトの `<div class="books-layout">` があり、さらにその `<section>`（books レイアウトの `{children}` の場所）の中にページの `<h1>` が入っています。
- つまり `children` は「**この場所に、1つ内側の箱を入れてください**」という差し込み口です。図4-6 の①が、そのまま HTML になっていることが確かめられました。

> **ヒント:** `<Link>` を使って `/books/42` から `/books/7` へ移動すると、Next.js は変わらない外側の箱（2つのレイアウト）を作り直さず、ページ本体だけを取り替えます（図4-6 の②）。だから、サイドバーの検索欄に入力した文字やスクロール位置が消えません。逆に、ページを移動するたびにレイアウトの中身も作り直したい場合は、`layout.tsx` の代わりに `template.tsx` という予約ファイルを使います。

4.3 書籍管理アプリのレイアウト構成



> **注意（第6章以降で作るアプリとの違い）**：上の図は「レイアウトが入れ子になる」ことを説明するための構成例です。第6章から実際に作る書籍管理アプリでは、話を簡単にするため、**書籍一覧はトップページ `/`（`app/page.tsx`）**に置き、新規登録は `/books/new`、詳細は `/books/[id]`、編集は `/books/[id]/edit` とします（`app/books/layout.tsx` のサイドバーは作りません）。「一覧に戻る」リンクの行き先も `/` になります。

---

5. ナビゲーション

5.1 Link コンポーネント

Next.js では、ページ間の遷移に HTML の `<a>` タグではなく、`next/link` モジュールが提供する `<Link>` コンポーネントを使います。`<Link>` は内部的に `<a>` を描画しますが、クリック時にページ全体をリロードせず、必要な部分だけを差し替える **クライアントサイド遷移**（client-side navigation：ブラウザでURLだけ書き換えて描画を切り替える方式）を行います。

> **▼ このコードがやること（先に日本語で）**：ページ間を移動するためのナビゲーション（メニュー）を、`next/link` の `<Link>` を使って作ります。`<a>` タグの代わりに `<Link>` を使うと、クリック時にページ全体を読み込み直さず必要な部分だけ差し替える「高速なページ移動」になります。`href` に行き先 URL を指定する書き方は `<a>` とよく似ています。詳細はコード内コメントを参照してください。

```
// ナビゲーション専用コンポーネント
// components/Navigation.tsx
// Next.js の Link コンポーネントを取り込む（default export）
import Link from "next/link";

// 名前付き export（中括弧で import される）
export function Navigation() {
  return (
    // セマンティック要素：ナビゲーション領域
    <nav>
      {/* リスト */}
      <ul>
        {/* 1項目 */}
        <li>
          {/* `href` で行き先URLを指定。<a> と似た書き方 */}
          <Link href="/">ホーム</Link>
        </li>
        <li>
          {/* `/books` への高速遷移リンク */}
          <Link href="/books">書籍一覧</Link>
        </li>
        <li>
          {/* `/books/new` への遷移 */}
          <Link href="/books/new">書籍を追加</Link>
        </li>
      </ul>
    </nav>
  );
}
```

`<Link>` と `<a>` の違い:

特性	`<a>` タグ	`<Link>` コンポーネント
ページ遷移	ページ全体をリロード	**クライアント側で遷移（高速）**
プリフェッチ	なし	**ビューポート内のリンクを自動プリフェッチ**
状態の保持	リセットされる	**レイアウトの状態が保持される**
JavaScript	不要	必要

▼ **このコードがやること（先に日本語で）**：書籍ごとに異なる詳細ページ（例: `/books/42` ）へのリンクを、データに応じて動的に組み立てます。`href={ /books/${book.id} }` のようにバッククォート（テンプレートリテラル）で URL に変数を埋め込むのがポイントです。1冊分の `book` を props で受け取り、その `id` からリンク先を作ります。詳細はコード内コメントを参照してください。

```
// 動的セグメント `[id]` を含むURLへのリンク例
// 動的ルートへのリンク
// Link コンポーネントを取り込む
import Link from "next/link";

// 書籍の型を簡易定義
type Book = {
  // 書籍ID
  id: string;
  // タイトル
  title: string;
};

// props として 1冊の book を受け取る
export function BookLink({ book }: { book: Book }) {
  return (
    // テンプレートリテラルで URL を組み立てる
    // 例: book.id が "42" なら "/books/42" が生成される
    <Link href={`/books/${book.id}`}>
      {/* 表示テキストは book.title を埋め込み */}
      {book.title} の詳細を見る
    </Link>
  );
}
```

画面にはこう表示される: 「○○ の詳細を見る」というリンクテキストが表示されます。クリックすると、ページ全体がリロードされることなく、スムーズに書籍詳細ページに遷移します。ヘッダーやサイドバーのレイアウトはそのまま残り、メインコンテンツ部分だけが切り替わります。

プリフェッチについて: `<Link>` コンポーネントはデフォルトで、ユーザーの画面（ビューポート）に表示されているリンク先のページを裏で先読みします。これにより、リンクをクリックした瞬間にページが表示されるような高速な体験が実現します。

▼ コードを1つずつ分解して解説

この動的リンクには「`<Link>` の import」「テンプレートリテラルでのURL組み立て」という2つのポイントがあります。

解説1: `next/link` から `<Link>` を取り込む

```
import Link from "next/link";

type Book = {
  id: string;
  title: string;
};
```

- `<Link>`（リンク）は、Next.js が用意している「**ページ間を高速に移動するための部品**」です。`next/link` から取り込みます。
- `import Link from ...`（中括弧なし）の形なのは、`<Link>` が **default export**（そのファイルの主役として公開）されているためです。
- `type Book = { ... }` は「書籍データの形」を決めるTypeScriptの型定義です。`id`（文字列）と `title`（文字列）を持つオブジェクト、と宣言しています。

> **用語: `<Link>` コンポーネント** … `<a>` タグの代わりに使うNext.jsの移動部品。クリック時にページ全体を読み込み直さず、必要な部分だけ差し替える「高速なページ移動」を実現します。

解説2: テンプレートリテラルで「行き先URL」を動的に組み立てる

```
export function BookLink({ book }: { book: Book }) {
  return (
    <Link href={`/books/${book.id}`}>
      {book.title} の詳細を見る
    </Link>
  );
}
```

- `{ book }` は分割代入で、props から `book`（書籍1冊分のデータ）を取り出しています。型は `{ book: Book }`。
- `href={`/books/${book.id}`}` がこのコードのキモです。バッククォート（`` ` ``）で囲んだ **テンプレートリテラル**を使い、`${book.id}` の部分に書籍IDを埋め込んでいます。`book.id` が `"42"` なら `/books/42` というURLが組み立てられます。
- `<Link>` の**内側**に書いた `{book.title} の詳細を見る` が、画面に表示されるリンクの文字になります。

> **用語: テンプレートリテラル** … バッククォートで囲む文字列で、`${変数}` の形で値を埋め込めます。URLやメッセージを「固定部分＋変化する値」で組み立てるときに便利です。

つまずきポイント（アプリ内のリンクに `<a>` を使う）：HTML に慣れていると、アプリの中のページ同士も `<a href="...">` でつないでしまいがちです。見た目は同じリンクですが、`<a>` だとクリックのたびに**ページ全体が読み込み直し**になり、レイアウトの中の状態（入力中の文字やカウンターの値など）が消えてしまいます。次のレイアウトで、実際に比べてみました。

```
// app/demo/layout.tsx（/demo/a と /demo/b で共通の枠）
import Link from "next/link";
import { Counter } from "./Counter";

export default function DemoLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div>
      {/* レイアウトの中のカウンター（ページを移動しても残ってほしい） */}
      <Counter />
      <nav>
        {/* ❌ NG: ふつうの a タグ */}
        <a href="/demo/b">a タグで B へ</a>
        {/* ✅ OK: Next.js の Link */}
        <Link href="/demo/b">Link で B へ</Link>
        <Link href="/demo/a">Link で A へ</Link>
      </nav>
      {children}
    </div>
  );
}
```

```
// app/demo/Counter.tsx（Client Component）
"use client";

import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>カウント: {count}</button>;
}
```

ブラウザでカウンターを3回押してから移動したときの記録です（ページ A・B の `page.tsx` は見出しを出すだけ）。

```
A で3回押した: ページ A / カウント: 3
a タグで B へ移動: ページ B / カウント: 0
A に戻って3回押した: ページ A / カウント: 3
Link で B へ移動: ページ B / カウント: 3
```

> `<a>` で移動した2行目はカウントが **0 に戻っています**（ページ全体を読み込み直したため）。`<Link>` で移動した4行目は **3 のまま**です（変わらないレイアウトはそのまま残り、ページ本体だけが入れ替わったため。図4-6）。アプリの中のページへは `<Link>`、外部のサイト（`https://...`）へは `<a>`、と使い分けましょう。なお、`<Link>` の先読み（プリフェッチ）は、`npm run build` してから `npm run start` で動かす本番モードで働きます。開発モード（`npm run dev`）では先読みしないので、「本番だと速い」ことは本番モードで確かめてください。

5.2 useRouter フック

プログラムからページ遷移を行いたい場合（ボタンクリック後やフォーム送信後など）は、`useRouter` フックを使います。**Client Component でのみ使用可能です。**

> **▼ このコードがやること（先に日本語で）：** `<Link>` のクリックではなく「処理が終わった後にプログラムから自動でページを移動する」方法を学びます。フォーム送信後に `useRouter` の `router.push("/books")` を呼んで一覧ページへ飛ばす例です。`useRouter` は Client Component でしか使えず、取り込み元は `next/navigation`（旧 `next/router` ではない）である点が要注意です。詳細はコード内コメントを参照してください。

```tsx
// 書籍フォーム部品
// components/BookForm.tsx
// useRouter は Client Component でしか使えないため必須
"use client";

// ⚠️ next/router ではない！
// App Router 用は `next/navigation`、旧 Pages Router 用は `next/router`
import { useRouter } from "next/navigation";

export function BookForm() {
  // ルーターインスタンスを取得
  const router = useRouter();

  // フォーム送信時のハンドラ
  const handleSubmit = async (e: React.FormEvent) => {
    // デフォルトの送信動作（ページリロード）を防ぐ
    e.preventDefault();

    // 書籍を保存する処理（省略）
    // 例: await fetch("/api/books", { method: "POST", ... });

    // 保存後、書籍一覧ページに遷移
    // プログラム的に画面遷移。履歴に「/books」が追加される
    router.push("/books");
  };

  return (
    // submit イベントを上記関数に紐付け
    <form onSubmit={handleSubmit}>
      {/* 実際には input 要素などを並べる */}
      {/* フォームの内容（省略） */}
      {/* submit ボタン */}
      <button type="submit">保存</button>
    </form>
  );
}
```

useRouter の主要メソッド:

▼ このコードがやること（先に日本語で）：useRouter が持つ4つの代表的な機能を、ボタンで呼び分けて比較します。push （履歴を残して移動）、replace （履歴を上書きして移動＝戻れない）、back （1つ前に戻る）、refresh （今のページのデータだけ取り直す）の違いを押さえるのが目的です。それぞれをいつ使うかはこの後の表にもまとまっています。詳細はコード内コメントを参照してください。

```tsx
// Client Component 化
"use client";

// App Router 用のフック
import { useRouter } from "next/navigation";

export function NavigationExample() {
  // ルーターを取得
  const router = useRouter();

  return (
    <div>
      {/* 指定したパスに遷移（履歴に追加） */}
      {/* `push`: 履歴を1つ増やしながら遷移（ブラウザの戻るで前ページに戻れる） */}
      <button onClick={() => router.push("/books")}>
        書籍一覧へ
      </button>

      {/* 指定したパスに遷移（履歴を置換） */}
      {/* `replace`: 現在の履歴エントリを上書きして遷移（戻れない） */}
      <button onClick={() => router.replace("/books")}>
        {/* ログイン後にログインページを履歴に残したくないときに使う */}
        書籍一覧へ（履歴を置換）
      </button>

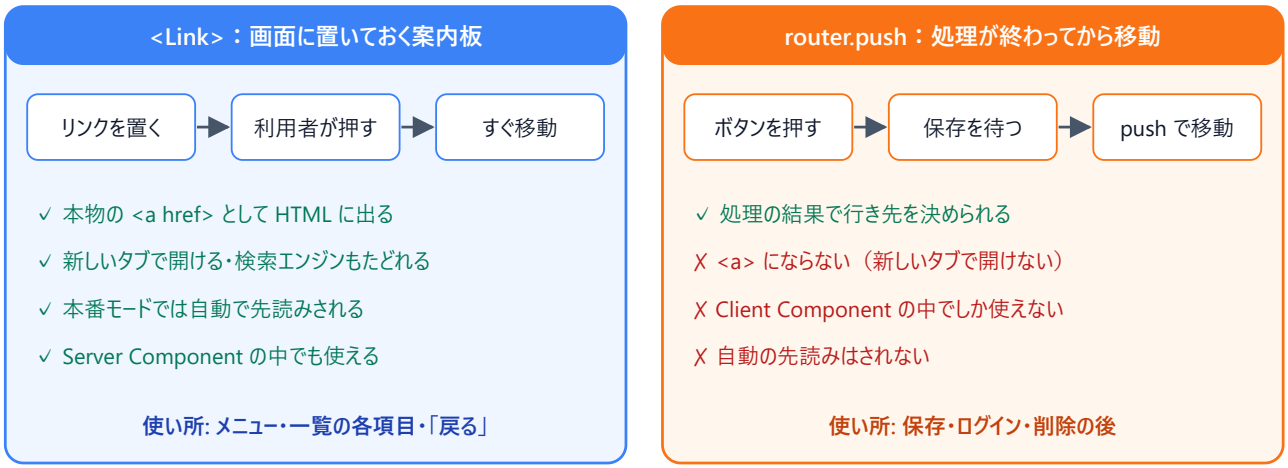
      {/* ブラウザの「戻る」と同じ */}
      {/* `back`: 履歴を1つ戻る */}
      <button onClick={() => router.back()}>
        戻る
      </button>

      {/* ページのデータを再取得（再レンダリング） */}
      {/* `refresh`: 現在のルートの Server Component を再実行・再描画 */}
      <button onClick={() => router.refresh()}>
        {/* ※ ページ全体リロードではない（state は保持される） */}
        更新
      </button>
    </div>
  );
}
```

メソッド	説明	使用例
`router.push(url)`	指定した URL に遷移	フォーム送信後のリダイレクト
`router.replace(url)`	現在の履歴エントリを置換して遷移	ログイン後のリダイレクト
`router.back()`	ブラウザの戻るボタンと同じ	詳細ページから一覧に戻る
`router.refresh()`	現在のページを再取得	データ更新後の再読み込み
`router.prefetch(url)`	指定した URL を先読み	近く遷移しそうなページの準備

Link と router.push を並べて比べる

`<Link>` と `router.push` は、どちらも「ページを移動する」道具ですが、役割が違います。`<Link>` は**道に立てておく案内板**で、利用者が自分で選んで進みます。`router.push` は**タクシーの運転手への指示**で、「荷物を積み終わったら（保存が終わったら）、あそこへ行って」とプログラムが行き先を告げます。



迷ったら Link。「何かが終わってから移動したい」ときだけ router.push を使う

図4-7: Link（案内板）と router.push（行き先の指示）の違い

> **▼ このコードがやること（先に日本語で）**：1つのページに `<Link>` と、`router.push` で移動するボタンを並べて置き、ブラウザに届いた HTML がどう違うかを確かめます。ボタンは「保存の代わりに 0.5 秒待ってから」移動するようにして、`router.push` が「処理が終わってから移動する」道具であることを体験します。

```tsx
// app/nav-demo/page.tsx（Server Component）
import Link from "next/link";
import { SaveButton } from "./SaveButton";

export default function NavDemoPage() {
  return (
    <div>
      {/* Link は「ここへ行けます」と画面に置いておくリンク */}
      <Link href="/books/1">1冊目の詳細へ（Link）</Link>
      {/* router.push は「処理が終わったら移動する」ためのボタン */}
      <SaveButton />
    </div>
  );
}
```

```tsx
// app/nav-demo/SaveButton.tsx（Client Component）
"use client";

import { useRouter } from "next/navigation";

export function SaveButton() {
  const router = useRouter();

  const handleClick = async () => {
    // 本当はここで保存処理をする（今回は 0.5 秒待つだけ）
    await new Promise((resolve) => setTimeout(resolve, 500));
    // 保存が終わったら、プログラムから詳細ページへ移動する
    router.push("/books/1");
  };

  return <button onClick={handleClick}>保存して詳細へ（router.push）</button>;
}
```

▼ 実行結果（`npm run dev` で `http://localhost:3000/nav-demo` を開いたときの HTML と、ボタンを押したときの記録）

```
<div><a href="/books/1">1冊目の詳細へ（Link）</a><button>保存して詳細へ（router.push）
</button></div>
```

```
ボタンを押してから 0.7 秒後に移動: /books/1
```

▼ コードを1つずつ分解して解説

解説1: `<Link>` は本物の `<a href>` になる

- HTML を見ると、`<Link>` は `<a href="/books/1">` という**ふつうのリンク**として出力されています。だから、右クリックで「新しいタブで開く」ができ、検索エンジンもこのリンクをたどってページを見つけられます。
- その上で、クリックされたときは Next.js がページ全体の読み込み直しを止めて、必要な部分だけを差し替えます（5.1 の比較表）。「ふつうのリンクの良さ」と「速い移動」の両方を持っているのが `<Link>` です。
- `<Link>` はただリンクを置くだけなので、`page.tsx` のような Server Component の中にそのまま書けます。

解説2: `router.push` は「処理の後で」移動するための道具

```
const handleClick = async () => {
  await new Promise((resolve) => setTimeout(resolve, 500));
  router.push("/books/1");
};
```

- `router.push` は**関数の中から呼ぶ**ので、「保存が成功したら移動」「失敗したら移動しない」のように、処理の結果に合わせて動きを変えられます。記録のとおり、ボタンを押してから約0.7秒後（0.5秒の待ち時間＋移動の時間）に移動しました。
- 一方、HTML の中では `<button>` のままで、行き先の URL はどこにも書かれていません。そのため、新しいタブで開くことも、検索エンジンがたどることもできません。**ただ移動するだけなら `<Link>`、何かが終わってから移動するなら `router.push`** と使い分けます。
- `useRouter` はブラウザで動くフックなので、`SaveButton` には `"use client"` が必要です。

5.2.1 usePathname / useSearchParams（補足）

`next/navigation` には `useRouter` 以外にも、現在の URL に関する情報を取り出すフックがあります。どちらも **Client Component 専用** です。

> **▼ このコードがやること（先に日本語で）:** 「今どの URL にいるか」「URL の `?sort=price` のような追加情報（クエリパラメータ）は何か」をプログラムから読み取ります。`usePathname` で現在のパス、`useSearchParams` でクエリの値（`.get("sort")` で取得）を取り出す例です。`??`（ヌル合体演算子）は「値が無いときの代わりの値」を指定する書き方です。詳細はコード内コメントを参照してください。

```tsx
// Client Component 必須
"use client";

// 2つのフックを取り込む
import { usePathname, useSearchParams } from "next/navigation";

export function CurrentUrlInfo() {
  // 例: "/books/123" のような現在のパス（クエリは含まない）
  const pathname = usePathname();
  // クエリパラメータを扱う ReadOnly な URLSearchParams 風オブジェクト
  const searchParams = useSearchParams();

  // `?sort=price` なら "price" を返す。なければ null
  const sort = searchParams.get("sort");
  // `??` は左辺が null/undefined のとき右辺を使う「ヌル合体演算子」
  const page = searchParams.get("page") ?? "1";

  return (
    <p>
      現在のパス: {pathname}（並び順: {sort ?? "デフォルト"}、ページ: {page}）
    </p>
  );
}
```

5.3 リダイレクト

リダイレクト（redirect：別のURLに自動転送すること）は、Server Component の中で `redirect()` を呼ぶか、後述する `proxy.ts`（Next.js 15 までの `middleware.ts`）で行います。

Server Component でのリダイレクト

> **▼ このコードがやること（先に日本語で）**：書籍詳細ページで「条件によって別ページへ自動転送する」処理を学びます。データが見つからなければ `notFound()` で404ページを、非公開の書籍なら `redirect("/books")` で一覧へ飛ばす例です。どちらも呼ぶとそこで処理が止まる（戻ってこない）ため、`return` の前に置くだけで分岐できます。詳細はコード内コメントを参照してください。

```tsx
// 動的セグメントを含む書籍詳細ページ
// app/books/[id]/page.tsx
// 2つのヘルパー関数を取り込む
// redirect: 任意の URL へ転送
// notFound: 404扱いにして not-found.tsx を表示
import { redirect, notFound } from "next/navigation";

// async Server Component
export default async function BookDetailPage({
  params,
}: {
  // Next.js 15+ では params は Promise でラップ
  params: Promise<{ id: string }>;
}) {
  // await で id を取り出す
  const { id } = await params;
  // 仮想のデータ取得関数
  const book = await fetchBook(id);

  // 書籍が見つからない場合は 404 ページを表示
  // データが null/undefined なら
  if (!book) {
    // この呼び出しで以降の処理は中断され not-found.tsx が表示される
    // 関数の戻り値は never（戻ってこない関数の型）
    notFound();
  }

  // 非公開の書籍は一覧ページにリダイレクト
  // 条件次第で
  if (book.isPrivate) {
    // `/books` に転送（これも never を返す）
    redirect("/books");
  }

  // 上の if を通り抜けた場合だけここに到達
  return (
    <div>
      {/* タイトル表示 */}
      <h1>{book.title}</h1>
    </div>
  );
}

// データ取得を模した関数
async function fetchBook(id: string) {
  // データベースから書籍を取得する処理（仮）
  // 本来は DB クエリや API 呼び出し
  // ダミーデータ
```

```
  return { title: "サンプル書籍", isPrivate: false };
}
```

▼ コードを1つずつ分解して解説

このリダイレクト処理には「`notFound()` で404を出す」「`redirect()` で別ページへ飛ばす」という、Server Component ならではの分岐があります。順に見ていきましょう。

解説1: `redirect` と `notFound` を `next/navigation` から取り込む

```
import { redirect, notFound } from "next/navigation";
```

- `redirect`（リダイレクト）は「**別のURLへ自動転送する**」関数、`notFound`（ノットファウンド）は「**404ページ（見つかりません）を表示する**」関数です。どちらも `next/navigation` から取り込みます。
- これらは Server Component の中で呼ぶことを想定した関数です。ユーザーの操作を待たずに「サーバー側で表示前に転送先を決める」ときに使います。

> **用語: リダイレクト（redirect）** … あるURLにアクセスしたユーザーを、自動的に別のURLへ飛ばすこと。ログインが必要なページや、移動した古いページなどで使われます。

解説2: 見つからなければ404、非公開なら一覧へ飛ばす

```
  const { id } = await params;
  const book = await fetchBook(id);

  if (!book) {
    notFound();
  }

  if (book.isPrivate) {
    redirect("/books");
  }
```

- `await fetchBook(id)` で書籍データを取りに行き、結果を `book` に入れます。
- `if (!book) { notFound(); }` … `!book` は「`book` が空（`null` / `undefined`）なら」という意味です。データが見つからなければ `notFound()` を呼び、404ページ（`not-found.tsx`）を表示します。
- `if (book.isPrivate) { redirect("/books"); }` … 非公開の書籍なら `redirect("/books")` で一覧ページへ転送します。

- **ポイント：** `notFound()` も `redirect()` も「**呼んだらそこで処理が止まり、後ろの行には進まない**」特殊な関数です。だから `else` を書かなくても、下の `return` まで到達するのは「見つかった＆公開されている」場合だけになります。

> **用語: 早期リターン的な中断** … `notFound()` / `redirect()` は戻り値の型が `never`（＝戻ってこない関数）で、呼んだ時点で処理が打ち切られます。そのため `if` で条件を満たしたらすぐ抜ける書き方ができます。

> **つまずきポイント（`redirect()` を `try` の中で呼ぶ）：** `redirect()` は、実は「`NEXT_REDIRECT` という特別なエラーを `throw`（投げる）して処理を止め、Next.js がそれを受け取って転送する」という仕組みで動いています。そのため、`try { ... } catch { ... }` の `try` の中で `redirect()` を呼ぶと、自分の `catch` がそのエラーを**横取り**してしまい、転送されずにページがそのまま表示されます。`npm run dev` で `/ng-redirect` を開いたときの実際の出力です。

```
catch で捕まえたもの: Error: NEXT_REDIRECT
    at NgRedirectPage (app\ng-redirect\page.tsx:7:13)
   5 |   try {
   6 |     // 何かの処理に失敗した、という想定で一覧へ転送したい
>  7 |     redirect("/shelf");
     |             ^
   8 |   } catch (error) {
   9 |     console.log("catch で捕まえたもの:", error);
  10 |   } {
  digest: 'NEXT_REDIRECT;replace;/shelf;307;'
}
 GET /ng-redirect 200 in 36ms (next.js: 1.3ms, application-code: 35ms)
```

転送されず（`200` ＝ ふつうに表示）、画面には `<p>ここが表示されてしまう</p>` が出ました。

```tsx
// ❌ NG: app/ng-redirect/page.tsx（redirect を try の中で呼んでいる）
import { redirect } from "next/navigation";

export default async function NgRedirectPage() {
  try {
    // 何かの処理に失敗した、という想定で一覧へ転送したい
    redirect("/shelf");
  } catch (error) {
    console.log("catch で捕まえたもの:", error);
  }
  return <p>ここが表示されてしまう</p>;
}
```

```tsx
// ✅ OK: app/ok-redirect/page.tsx（try / catch では結果だけ受け取り、redirect は外で呼ぶ）
import { redirect } from "next/navigation";

// 失敗するかもしれない処理のまね（今回は必ず失敗する）
async function loadSomething(): Promise<boolean> {
  throw new Error("読み込みに失敗しました");
}

export default async function OkRedirectPage() {
  let loaded = false;
  try {
    loaded = await loadSomething();
  } catch (error) {
    console.log("catch で捕まえたもの:", (error as Error).message);
  }

  // redirect は try / catch の「外」で呼ぶ
  if (!loaded) {
    redirect("/shelf");
  }
  return <p>読み込みに成功しました</p>;
}
```

OK 版を `curl -i`（応答の見出しも表示するコマンド）で開いたときの出力と、ターミナルの出力です。`307 Temporary Redirect` と `location: /shelf` は「`/shelf` へ移動してください」という転送の合図です。

```
HTTP/1.1 307 Temporary Redirect
location: /shelf
```

```
catch で捕まえたもの: 読み込みに失敗しました
 GET /ok-redirect 307 in 50ms (next.js: 1.7ms, application-code: 49ms)
```

> `notFound()` も同じ仕組み（特別なエラーを投げる）なので、`try` の中で呼ばないようにしましょう。7節の Server Action の中で `redirect()` を使うときも同じです。

---

proxy.ts（旧 middleware.ts）でのリダイレクト

`proxy.ts`（プロキシ：リクエストとページの間に割り込む「関所」の処理）をプロジェクトルートに置くことで、ページが描画される**前の段階**でリダイレクトを行えます。認証チェック、地域別の振り分け、A/Bテストなどに使えます。

注意（Next.js 16 で名前が変わりました）：Next.js 15 までは、このファイルは `middleware.ts`（ミドルウェア）、関数名は `middleware` でした。Next.js 16 で **`proxy.ts`・関数名 `proxy`** に変わり、古い名前は「非推奨（deprecated：まだ動くが、いずれ使えなくなる予定）」になりました。また、`proxy.ts` は Node.js の実行環境で動きます（以前の `middleware.ts` は Edge Runtime という軽量な実行環境で動いていました）。古い解説記事の `middleware.ts` は `proxy.ts` と読み替えてください。なお、`next build` の結果には今も `ƒ Proxy (Middleware)` と表示されます。

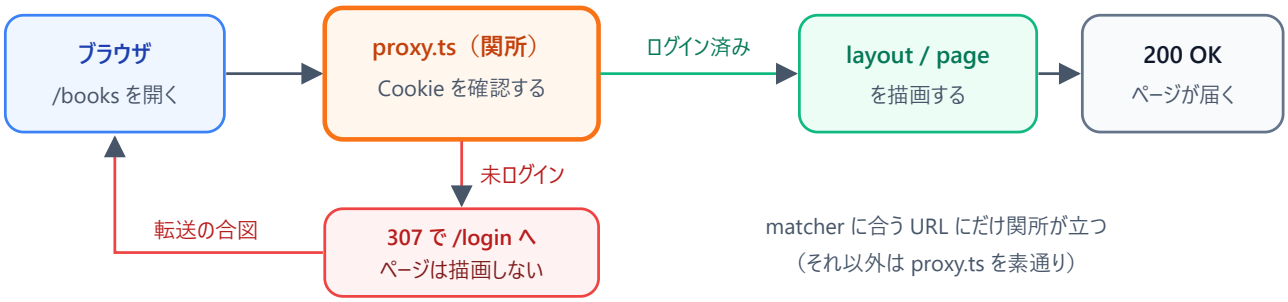


図4-8: proxy.ts は、ページを描画する前に立つ関所

▼ このコードがやること（先に日本語で）：ページが表示される「前の段階」で割り込み、未ログインのユーザーを `/login` に転送する関所（プロキシ）を作ります。リクエストの Cookie からログイン状態を判定し、条件に合えば転送、合わなければそのまま通します。最後の `config.matcher` で「どの URL にこの関所を効かせるか」を指定する点もポイントです。詳細はコード内コメントを参照してください。

```ts
// ファイル名は固定。app/ の外、プロジェクト直下に置く
// proxy.ts（プロジェクトルート。Next.js 15 までの middleware.ts）

// リダイレクトや「通過OK」を返すためのレスポンスヘルパー
import { NextResponse } from "next/server";
// 型だけ取り込み（リクエスト型）
import type { NextRequest } from "next/server";

// `proxy` という名前で export すると Next.js が自動呼び出し
export function proxy(request: NextRequest) {
  // 例: 未ログインユーザーを /books ページからリダイレクト
  // Cookie から `session` の値を取得（あればログイン中とみなす）
  const isLoggedIn = request.cookies.get("session");

  // 未ログイン かつ /books 配下にアクセスしている
  if (!isLoggedIn && request.nextUrl.pathname.startsWith("/books")) {
    // /login に転送するレスポンスを返す
    // `new URL("/login", request.url)` はベースURLを保ったまま絶対URLを作る
    return NextResponse.redirect(new URL("/login", request.url));
  }

  // 条件に合わない場合は普通に次の処理（=ページ描画）へ進める
  return NextResponse.next();
}

// proxy（関所）を適用するパスを指定
// `config` という名前で設定オブジェクトを export
export const config = {
  // この matcher にマッチしたURLでだけ proxy が動く
  // `:path*` は「任意の階層を含む」の意味
  matcher: ["/books/:path*"],
};
```

---

6. データフェッチ

6.1 Server Component でのデータ取得

Server Component ではコンポーネント関数を `async` にして、直接 `await` でデータを取得できます。これが Next.js App Router の最も強力な機能の1つです。

▼ **このコードがやること（先に日本語で）**：Server Component の最大の魅力である「コンポーネント関数を `async` にして、その場で `await` でデータを取得する」書き方を学びます。取得した書籍配列を `map`（1件ずつ変換）でリスト表示する、データ取得の王道パターンです。データが入った状態の HTML がサーバーから届くので表示が速く SEO にも強い、という普通の React との違いも意識してください。詳細はコード内コメントを参照してください。

```tsx
// ============================================================================
// ファイルパス: app/books/page.tsx
// 役割       : 書籍一覧ページ。サーバーで全書籍を取ってきてHTMLにする。
// 種類       : Server Component（"use client" を書いていないので自動でこちら）
// ----------------------------------------------------------------------------
// Server Component の最大の魅力は「コンポーネント関数自体を async にできて、
// 直接 await でデータを取ってこられる」こと。
// そのデータはサーバーで埋め込まれた状態でブラウザに届くので、
// 初回表示が速く、SEOにも有利。
// ============================================================================


// (1) 書籍データ1件分の「形」を定義
//     Book 型を1度ここで決めておくと、map のコールバック内などで
//     book.〇〇 と書いたとき VS Code が補完してくれる。
type Book = {
  // 一意なID
  id: string;
  // 書籍タイトル
  title: string;
  // 著者名
  author: string;
  // 出版年
  publishedYear: number;
};


// (2) APIから書籍配列を取ってくる関数
//     async を付けると、関数の中で await が使えるようになる。
//     戻り値は「Book型の配列が将来届く」を意味する Promise<Book[]>。
async function getBooks(): Promise<Book[]> {

  // fetch は Web標準のHTTP通信API。
  // 第2引数の cache オプションで Next.js のキャッシュ動作を指定する。
  const response = await fetch("https://api.example.com/books", {
    // ← 毎リクエストで最新データを取得する（=SSR動作）
    cache: "no-store",
    // cache: "force-cache",
    // ← ビルド時に1度だけ取得（=SSG動作）
    // next: { revalidate: 60 },
    // ← 60秒ごとに再取得（=ISR動作）
  });

  // (2-1) 通信は成功したけどステータスが 4xx/5xx の場合は失敗扱いにする。
  // response.ok は status が 200〜299 のとき true、それ以外で false。
  if (!response.ok) {
    throw new Error("書籍データの取得に失敗しました");
  }
```

```
  // (2-2) JSON 文字列を JS オブジェクトに変換して返す
  return response.json();
}


// (3) ページコンポーネント本体
//     関数自体に async を付けられるのが Server Component の特権。
//     Client Component（"use client"あり）では関数本体を async にできない。
export default async function BooksPage() {

  // この行はサーバーで実行される。
  // 取得処理が終わるまでブラウザにはHTMLが送られない（その間 loading.tsx が表示される）。
  const books = await getBooks();

  // (4) 取れたデータを使ってJSXを組み立てる
  //     books.length は配列の要素数（書籍が3件なら3）。
  //     books.map((book) => ...) で1件ずつJSXに変換する。
  //     key={book.id} は React がリストの差分検出に使う必須の属性。
  return (
    <div>
      <h1>書籍一覧（{books.length}冊）</h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author}（{book.publishedYear}年）</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

▼ APIが3件のデータを返した場合のブラウザ表示:

```
書籍一覧（3冊）
・ リーダブルコード
  著者: Dustin Boswell（2012年）
・ プロを目指す人のためのTypeScript入門
  著者: 鈴木 僚太（2022年）
・ 達人プログラマー
  著者: David Thomas（2016年）
```

▼ 通信失敗時:

`throw new Error(...)` が走ると、Next.js は同じセグメント内の `error.tsx` を自動で表示します。`error.tsx` を作っていない場合は親のエラーバウンダリにバブリングします。

▼ ブラウザに届くHTMLの様子（要点だけ抜粋）：

```
<div>
  <h1>書籍一覧（3冊）</h1>
  <ul>
    <li><h3>リーダブルコード</h3><p>著者: Dustin Boswell（2012年）</p></li>
    <li><h3>プロを目指す人のためのTypeScript入門</h3><p>著者: 鈴木 僚太（2022年）</p></li>
    <li><h3>達人プログラマー</h3><p>著者: David Thomas（2016年）</p></li>
  </ul>
</div>
```

▼ **普通のReactとの最大の違い:** クライアントサイドの useState/useEffect でデータを取りに行くのと違って、**最初からデータが入ったHTMLが返ってくる**ので、表示がカクカクしない・SEOに強い・JSバンドルも小さく済む。

▼ コードを1つずつ分解して解説

このデータ取得コードには「`async` 関数での `fetch`」「`cache` オプション」「`response.ok` でのエラー判定」「`await` でのページ内データ取得」という要素があります。順に見ていきましょう。

解説1: `async` 関数の中で `fetch` してデータを取りに行く

```
async function getBooks(): Promise<Book[]> {
  const response = await fetch("https://api.example.com/books", {
    cache: "no-store",
  });
```

- `getBooks` は「APIから書籍一覧を取ってくる」関数です。`async` が付いているので、中で `await` が使えます。
- `: Promise<Book[]>` は戻り値の型注釈で、「`Book` **型の配列が、将来（非同期で）届く**」という意味です。`Promise<...>` は「後で値が届く」ことを表します。
- `await fetch(...)` で、指定したURLにデータを取りに行き、返事（`response`）が届くまで待ちます。

> **用語: fetch（フェッチ）** … サーバーとHTTP通信してデータをやり取りするブラウザ標準の関数。Next.js ではこれを拡張して、後述のキャッシュ機能を追加しています。

解説2: `cache` オプションで「いつ取り直すか」を決める

```
const response = await fetch("https://api.example.com/books", {
  // ← 毎リクエストで最新データを取得する（=SSR動作）
  cache: "no-store",
  // cache: "force-cache",
  // ← ビルド時に1度だけ取得（=SSG動作）
  // next: { revalidate: 60 },
  // ← 60秒ごとに再取得（=ISR動作）
});
```

- `fetch` の第2引数の `cache` （キャッシュ）は、Next.js が標準の `fetch` に追加した独自オプションです。「**取得したデータをどれくらい使い回すか**」を決めます。
- `cache: "no-store"` は「使い回さず、毎回サーバーから最新を取る」設定で、常に新しいデータが必要なページ（＝SSR動作）に向きます。
- コメントアウトされた `"force-cache"` は「1度取ったら使い回す」（＝SSG動作）、`next: { revalidate: 60 }` は「60秒ごとに取り直す」（＝ISR動作）です。**この1行を変えるだけで、ページの性質を切り替えられる**のが Next.js の特徴です。

> **用語: キャッシュ（cache）** … 一度取得したデータを保存しておき、次回はそれを再利用する仕組み。通信を減らして高速化できますが、その分データが古くなる可能性があります。

解説3: `response.ok` で通信の成否を確認する

```
if (!response.ok) {
  throw new Error("書籍データの取得に失敗しました");
}

return response.json();
```

- `response.ok` （レスポンス・オーケー）は、通信のステータスが正常（200〜299番台）なら `true` 、それ以外（404や500などのエラー）なら `false` になります。
- `if (!response.ok)` は「正常でなければ」という意味で、`throw new Error(...)` で**エラーを発生させます。** Next.js ではこのエラーが起きると、自動的に同じフォルダの `error.tsx` が表示されます。
- 最後の `response.json()` は「**返ってきた本文（JSON文字列）を、JavaScriptのオブジェクトに変換する**」処理です。これで使える形の書籍配列が返ります。

> **用語: throw（スロー）** … 「エラーを投げる」命令。`throw new Error("メッセージ")` で意図的にエラーを発生させ、エラー処理（ `error.tsx` など）に流れを移せます。

解説4: ページ本体で `await` してデータ入りのJSXを返す

```
export default async function BooksPage() {
  const books = await getBooks();

  return (
    <div>
      <h1>書籍一覧（{books.length}冊）</h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author}（{book.publishedYear}年）</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

- ページコンポーネント自体が `async` なので、本体で `const books = await getBooks()` と書いてデータ取得の完了を待てます。この待っている間、ユーザーには `loading.tsx` が表示されます。
- `books.length` は配列の件数（3冊なら3）です。見出しに「書籍一覧（3冊）」のように埋め込んでいます。
- `books.map((book) => (...))` で、配列の各書籍を1件ずつ `<li>`（リスト項目）のJSXに変換しています。`key={book.id}` は、Reactがリストの各項目を見分けるための必須の目印です。

> **用語: map（マップ）** … 配列の各要素を1つずつ別の形に作り替え、新しい配列を作るメソッド。「データの配列」を「JSX要素の配列」に変換してリスト表示するのが定番の使い方です。

---

fetch のキャッシュ戦略

Next.js の `fetch` には、標準仕様にはない独自のオプションが追加されています。これらを使い分けることで SSR / SSG / ISR（インクリメンタル静的再生成）を切り替えられます。

```
// 毎回サーバーで最新データを取りに行く動作
// SSR - リクエストのたびに最新データを取得
// `cache: "no-store"` でキャッシュを使わない
fetch(url, { cache: "no-store" });

// 1回取得したら保存しておき、以降は同じ結果を返す
// SSG - データをキャッシュ（Next.js 14 まではこれがデフォルトだった）
// `cache: "force-cache"` で強制キャッシュ
fetch(url, { cache: "force-cache" });

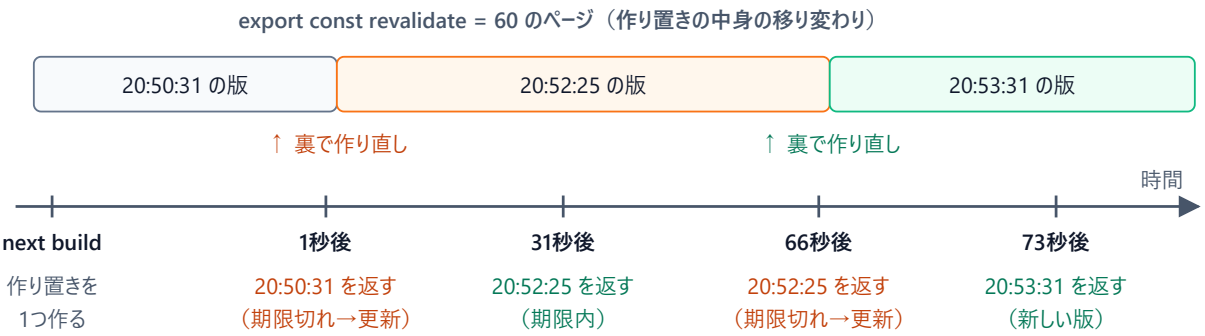
// 一定間隔で再取得する中間方式
// ISR - 60秒ごとにキャッシュを再検証
// `next.revalidate` は Next.js が追加した独自オプション
// 60 は秒数。指定間隔ごとに次のアクセス時に再取得
fetch(url, { next: { revalidate: 60 } });
```

戦略	説明	使用例
`cache: "no-store"`	毎回取得（SSR）	ユーザー固有のデータ
`cache: "force-cache"`	1回取得して使い回す（SSG）	変更の少ないデータ
`next: { revalidate: N }`	N秒ごとに再検証（ISR）	ニュース記事など
何も指定しない	キャッシュしない（ただしページが作り置きされると、ビルド時の1回だけになる。下の「つまずきポイント」）	—

> **注意（Next.js 15 で「何も指定しないとき」の動きが変わりました）**：Next.js 14 までは、`fetch` に何も指定しないと `force-cache`（保存して使い回す）になっていました。Next.js 15 からは逆に、**何も指定しないとキャッシュしません**。古い解説記事の「fetch はデフォルトでキャッシュされる」という説明は、今の Next.js には当てはまらないので注意してください。

キャッシュと再検証のしくみを図で見る

`revalidate`（リバリデート：再検証）は「作り置きの賞味期限」のようなものです。お弁当屋さんにたとえると、`revalidate = 60` は「作ってから60秒たったお弁当は、次のお客さんが来たときに**とりあえずそれを渡しつつ**、裏で新しいお弁当を作り始める」というルールです。お客さんを待たせない代わりに、期限切れ直後の1人だけは少し古いお弁当を受け取ります。下の図は、発展の「ISR」で実際に `revalidate = 60` のページを動かして記録した時刻をそのまま並べたものです。



期限切れ直後の1回は「古い版をすぐ返し、裏で作り直す」。待たせない代わりに、1回だけ古い

図4-9: revalidate = 60 の作り置きが更新されていく様子（実際の記録）

- 「1秒後」のアクセスでは、作り置き（ビルド時の 20:50:31 版）がすでに60秒以上たっていたので、**古い版をすぐ返しつつ**、裏で作り直しが始まりました。
- 「31秒後」のアクセスでは、作り直された 20:52:25 版が返っています。まだ60秒たっていないので、そのまま使い回されます。
- 「66秒後」は 20:52:25 版から60秒を過ぎたので、また古い版を返しながら裏で作り直し、「73秒後」には新しい 20:53:31 版が返りました。

> **つまずきポイント（本番にしたら時刻やデータが変わらなくなった）**：`npm run dev`（開発モード）では、ページはアクセスのたびに作り直されます。ところが `npm run build` → `npm run start`（本番モード）にすると、Next.js は「アクセスのたびに変わる要素が無い」と判断したページを、**ビルドのときに1回だけ作って保存（静的化）**します。すると、時刻やデータが**ビルドした瞬間のまま止まって**しまいます。次の時刻を表示するだけのページで確かめました。

```tsx
// ❌ NG: app/clock/page.tsx（ビルド時に1回だけ作られて、時刻が止まる）
export default function ClockPage() {
  // このコードが「いつ」実行されたかを時刻で確かめる
  const now = new Date().toLocaleTimeString("ja-JP");
  return <p>このページを作った時刻: {now}</p>;
}
```

`npm run build` の結果（関係する行だけ抜粋）。`/clock` の左に `○`（Static：作り置き）が付いています。

```
Route (app)
├ ○ /clock
○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

`npm run start` で起動し、時間をあけて2回 `curl` で開いた結果です（`<!-- -->` は React が文字の区切りに入れる目印で、画面には出ません）。2回とも、ビルドしたときの時刻のままです。

```
<p>このページを作った時刻: <!-- -->20:47:16</p>
<p>このページを作った時刻: <!-- -->20:47:16</p>
```

```tsx
// ✅ OK: app/clock/page.tsx（アクセスのたびにサーバーで作り直す）
import { connection } from "next/server";

export default async function ClockPage() {
  // 「このページはアクセスのたびにサーバーで実行する」と宣言する
  await connection();
  const now = new Date().toLocaleTimeString("ja-JP");
  return <p>このページを作った時刻: {now}</p>;
}
```

OK 版をビルドすると `/clock` は `ƒ /clock`（Dynamic：アクセスのたびに作る）になり、3秒あけて2回開くと時刻が変わりました。

```
<p>このページを作った時刻: <!-- -->11:31:41</p>
<p>このページを作った時刻: <!-- -->11:31:44</p>
```

> `connection()`（コネクション）は `next/server` が用意している関数で、`await connection()` と書くと「この先はリクエストが来てから実行する」という合図になります。データベースからの取得でも同じことが起きるので、「本番だけ一覧が更新されない」ときは、まず `npm run build` の結果で `○` と `ƒ` のどちらになっているかを確認しましょう。`cache: "no-store"` の `fetch`、`cookies()`・`headers()`、`searchParams` を使ったページも、自動で `ƒ` になります。

注意（Cache Components を「Yes」にしたプロジェクトの場合）：第6章では `create-next-app` で Cache Components を **No** にするよう案内しています。もし Yes にすると、`next.config.ts` に `cacheComponents: true` が入り、キャッシュのしくみが Next.js 16 の新しい方式に切り替わります。この章のコードに関係する主な違いは次のとおりです（どれも Next.js 16.4.1 で実際に確かめました）。

- `await params`・`await searchParams`・`await connection()`・`cache: "no-store"` の `fetch` などを使うページに `loading.tsx`（または `<Suspense>`）が無いと、`npm run build` が `Next.js encountered uncached or runtime data during prerendering.` というエラーで止まります。同じフォルダに `loading.tsx` を置けば通ります。
- `export const revalidate = 60` のような書き方は使えず、`Route segment config "revalidate" is not compatible with nextConfig.cacheComponents. Please remove it.` というエラーになります。代わりに `"use cache"` と `cacheLife(...)` を使います（発展の「ISR」で例を紹介します）。
- 上の NG 版の時刻ページは、黙って止まるのではなく `Next.js encountered the unstable value new Date() while prerendering.` というエラーで「`await connection()` を書いてね」と教えてくれます。

新しい方式は強力ですが、ここまでの説明と書き方が変わる部分が多いので、本書では No（従来の方式）で進めます。

6.2 Client Component でのデータ取得

Client Component では、従来の React と同じように `useEffect` と `useState` を使うか、サードパーティのデータ取得ライブラリ（SWR や TanStack Query）を使います。

▼ このコードがやること（先に日本語で）：ブラウザ側でリアルタイムに検索する部品を、従来の React と同じ `useState` + `useEffect` で作ります。入力のたびに毎回通信すると重いので、「入力が止まって300ミリ秒後にだけ検索する」デバウンスという工夫を入れています。読み込み中・エラー・結果という3つの状態をそれぞれ管理する、実戦的な例です。詳細はコード内コメントを参照してください。

```tsx
// ブラウザでリアルタイム検索する部品
// components/BookSearch.tsx
// Client Component。useState/useEffect を使うため必須
"use client";

// 状態管理と副作用管理のフック
import { useState, useEffect } from "react";

// 検索結果1件の型
type Book = {
  // 書籍ID
  id: string;
  // タイトル
  title: string;
  // 著者
  author: string;
};

export function BookSearch() {
  // 検索文字列の state（初期値は空文字）
  const [query, setQuery] = useState("");
  // 検索結果の配列。ジェネリック `<Book[]>` で要素の型を指定
  const [books, setBooks] = useState<Book[]>([]);
  // 読み込み中フラグ
  const [isLoading, setIsLoading] = useState(false);
  // エラーメッセージ（なければ null）
  const [error, setError] = useState<string | null>(null);

  // query が変わるたびに走る副作用
  useEffect(() => {
    // 検索クエリが空なら何もしない
    // `trim()` で前後の空白を取り除いて空かチェック
    if (!query.trim()) {
      // 空の結果に
      setBooks([]);
      // ここで打ち切り
      return;
    }

    // デバウンス: 入力が止まって300ms後に検索を実行
    // 300ms後に走るタイマーを仕掛ける
    // 入力が続いている間はクリーンアップで毎回キャンセルされる（=デバウンス）
    const timer = setTimeout(async () => {
      // 「読み込み中」に切り替え
      setIsLoading(true);
      // エラー表示はリセット
      setError(null);

      // 例外の可能性のある処理を try で囲む
```

```
      try {
        // 内部APIに GET リクエスト
        const response = await fetch(
          // クエリパラメータ q に検索文字列を入れる
          `/api/books/search?q=${encodeURIComponent(query)}`
        );
        // HTTP ステータスが 200番台でなければ
        if (!response.ok) {
          // 例外を投げて catch に飛ばす
          throw new Error("検索に失敗しました");
        }
        // レスポンス本文（JSON）を JS オブジェクトに変換
        const data = await response.json();
        // state に保存 → 再描画
        setBooks(data);
      // 通信失敗や上の throw を捕まえる
      } catch (err) {
        // err が Error 型ならその message を、それ以外なら汎用文を表示
        setError(err instanceof Error ? err.message : "エラーが発生しました");
      // 成功・失敗どちらでも最後に走る
      } finally {
        // 読み込み中フラグを解除
        setIsLoading(false);
      }
    // 300ms = デバウンス時間
    }, 300);

    // クリーンアップ: 次の入力が来たらタイマーをキャンセル
    // useEffect が return する関数は「次のeffect実行前」または「unmount時」に呼ばれる
    return () => clearTimeout(timer);
  // 依存配列。query が変わったときだけ effect を再実行
  }, [query]);

  return (
    <div>
      <input
        // 1行テキスト入力欄
        type="text"
        // 表示値を state と同期（制御コンポーネント）
        value={query}
        // 入力変更時に state を更新
        onChange={(e) => setQuery(e.target.value)}
        // 空欄ガイド
        placeholder="書籍を検索..."
      />

      {/* 短絡評価: isLoading が true のときだけ <p> を描画 */}
      {isLoading && <p>検索中...</p>}
      {/* エラー文字列があるときだけ表示 */}
      {error && <p className="error">{error}</p>}
```

```
      <ul>
        {/* 配列を JSX 要素にマッピング */}
        {books.map((book) => (
          // `key` は React に「どの項目が同じか」を伝える必須属性
          <li key={book.id}>
            {/* タイトル - 著者 の形式で表示 */}
            {book.title} - {book.author}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

画面にはこう表示される: 検索入力欄が表示されます。ユーザーが「村上」と入力すると、入力が止まって300ミリ秒後に「検索中...」というメッセージが一瞬表示され、その後「村上春樹」の著書一覧がリスト形式で表示されます。

▼ コードを1つずつ分解して解説

このクライアント検索には「複数の state 管理」「`useEffect` での副作用」「デバウンス」「`try/catch/finally` での通信処理」という要素が組み合わさっています。順に見ていきましょう。

解説1: 4つの state で「入力・結果・読み込み中・エラー」を管理する

```
  const [query, setQuery] = useState("");
  const [books, setBooks] = useState<Book[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
```

- Client Component で通信を伴う検索を作るには、複数の状態を同時に管理する必要があります。ここでは4つの `useState` を使っています。
- `query`：入力された検索文字列（初期値は空文字）。
- `books`：検索結果の配列。`useState<Book[]>([])` の `<Book[]>` は「`Book` 型の配列を入れる箱」という型指定で、初期値は空配列 `[]`。
- `isLoading`：読み込み中かどうかの `true` / `false`。
- `error`：エラーメッセージ。`<string | null>` は「文字列、またはエラーなしの `null`」という型で、初期値は `null`。

> **用語: ジェネリック（`<...>`）** … `useState<Book[]>(...)` の `<Book[]>` のように、型を `< >` で指定する書き方。「この箱には何の型が入るか」をTypeScriptに明示できます。

解説2: `useEffect` で「`query` が変わるたびに検索する」

```
useEffect(() => {
  if (!query.trim()) {
    setBooks([]);
    return;
  }
  // ...（タイマーの仕掛け）...
}, [query]);
```

- `useEffect`（ユーズ・エフェクト）は「**画面の描画とは別に行う処理（副作用）**」を書くフックです。データ取得はこの代表例です。
- 末尾の `[query]` は **依存配列**で、「`query` が変わったときだけ、この中の処理を実行する」という指定です。つまり入力が変わるたびに検索処理が走ります。
- `if (!query.trim())` は「入力が空（または空白だけ）なら」の判定です。その場合は結果を空にして `return` で打ち切り、無駄な通信を避けます。

> **用語: 副作用（side effect） / 依存配列** … 副作用は「画面を描く以外の処理（通信・タイマーなど）」のこと。依存配列はその副作用を「どの値が変わったら実行し直すか」を指定するリストです。

解説3: デバウンス ―「入力が止まって300ミリ秒後」にだけ検索する

```
  const timer = setTimeout(async () => {
    // ...通信処理...
  }, 300);

  return () => clearTimeout(timer);
```

- `setTimeout(関数, 300)` は「300ミリ秒後にこの関数を実行する」タイマーを仕掛けます。
- 最後の `return () => clearTimeout(timer)` は **クリーンアップ関数**で、「次の入力が来たら（＝effectが再実行される直前に）前のタイマーを取り消す」処理です。
- この「**タイマーを仕掛ける→次の入力で取り消す**」の繰り返しにより、ユーザーが文字を打ち続けている間は検索が走らず、**手が止まって300ミリ秒経ったときだけ**検索が実行されます。これを **デバウンス（debounce）** と呼びます。毎文字ごとに通信する無駄を防げます。

> **用語: デバウンス（debounce）** … 連続して起きるイベント（キー入力など）を間引き、「最後の操作から一定時間経ったとき」だけ処理を実行する手法。検索や入力補完で多用されます。

解説4: `try / catch / finally` で通信の成功・失敗・後始末を分ける

```
try {
  const response = await fetch(
    `/api/books/search?q=${encodeURIComponent(query)}`
  );
  if (!response.ok) {
    throw new Error("検索に失敗しました");
  }
  const data = await response.json();
  setBooks(data);
} catch (err) {
  setError(err instanceof Error ? err.message : "エラーが発生しました");
} finally {
  setIsLoading(false);
}
```

- `try { ... }` には「失敗するかもしれない処理（通信）」を書きます。成功すれば、結果を `setBooks(data)` で state に保存します。
- `catch (err) { ... }` は「`try` の中でエラーが起きたとき」に実行されます。`err instanceof Error ? err.message : "エラーが発生しました"` は「`err` がちゃんとした Error なら、そのメッセージを、そうでなければ汎用メッセージを使う」という三項演算子です。
- `finally { ... }` は「成功しても失敗しても**必ず最後に**実行される」ブロックです。ここで `setIsLoading(false)` を呼び、どちらの場合も「読み込み中」表示を確実に解除しています。

> **用語: try / catch / finally** … エラーが起きうる処理を安全に扱う構文。`try` で試し、`catch` でエラーを受け止め、`finally` で後始末（必ず実行）をします。

Server Component と Client Component のデータ取得の比較:

特性	Server Component	Client Component
コード量	少ない（async/await のみ）	多い（useState + useEffect）
ローディング状態	loading.tsx が自動管理	自分で管理が必要
エラー処理	error.tsx が自動管理	自分で管理が必要
データの安全性	APIキー等が露出しない	APIキーは使えない
SEO	HTML にデータが含まれる	初回は空
リアルタイム更新	ページ再読み込みが必要	可能

6.3 loading.tsx による読み込み状態

`loading.tsx` を配置するだけで、Server Component のデータ取得中に自動的にローディング UI が表示されます。

> **▼ このコードがやること（先に日本語で）**：読み込み中に、実際のコンテンツと同じ形をした灰色の枠（スケルトン UI）を表示するローディング画面を作ります。データが届いた瞬間に画面がガタつかないようにするための工夫です。`Array.from({ length: 6 }).map(...)` で同じカードを6個並べる書き方や、`style` 属性にオブジェクトを渡す JSX 特有の書き方を学べます。詳細はコード内コメントを参照してください。

```tsx
// 予約名 `loading.tsx`。/books のロード中UI
// app/books/loading.tsx

// default export 関数
export default function BooksLoading() {
  return (
    <div className="loading-container">
      {/* スケルトンUI: 実際のコンテンツと同じ形状の灰色の枠 */}
      {/* `style` 属性はオブジェクトを渡す（CSSのキャメルケース表記） */}
      {/* `{{ ... }}` の外側はJSXの式、内側はJSオブジェクト */}
      <h1 className="skeleton" style={{ width: "200px", height: "32px" }} />

      <div className="book-grid">
        {/* 6個のスケルトンカード */}
        {/* `Array.from({ length: 6 })` で要素6個の配列を作る */}
        {/* 第1引数 `_` は使わないので慣習的にアンダースコア */}
        {/* 第2引数 `i` は 0,1,2,...,5 のインデックス */}
        {Array.from({ length: 6 }).map((_, i) => (
          // key にインデックスを使う（並び順が変わらないリストならOK）
          <div key={i} className="skeleton-card">
            <div
              className="skeleton"
              // 表紙画像の代わりの灰色ブロック
              style={{ width: "100%", height: "200px" }}
            />
            <div
              className="skeleton"
              // タイトル代わりの細長い灰色ブロック
              style={{ width: "80%", height: "20px", marginTop: "8px" }}
            />
            <div
              className="skeleton"
              // 著者代わりのさらに小さなブロック
              style={{ width: "60%", height: "16px", marginTop: "4px" }}
            />
          </div>
        ))}
      </div>
    </div>
  );
}
```

画面にはこう表示される: データの読み込み中、見出しの位置に灰色のバーが1つ、その下に灰色のカード状のブロックが6個並びます。これらは実際のコンテンツと同じ大きさ・位置で表示されるため、読み込みが完了するとスムーズに実際のコンテンツに切り替わり、画面がガタつきません（この手法を **スケルトンUI** と呼びます）。



loading.tsx が「先に」届く様子を時間で測る

上の流れ図のとおりなら、ブラウザには「まず `loading.tsx` の中身、データが届いたら `page.tsx` の中身」の順で HTML が届くはずです。本当にそうなっているのかを、届いた時刻を測って確かめてみましょう。

> **▼ このコードがやること（先に日本語で）**：わざと2秒かかるページ（ `app/slow/page.tsx` ）と、その読み込み中に出す `loading.tsx` を作ります。そして、そのページを読み込む小さなプログラム（ `stream-check.mjs` ）で、「何秒目に・どちらの中身が届いたか」を表示します。ページ全体が2秒後にまとめて届くのではなく、`loading.tsx` の部分がすぐ届くことを確かめるのが目的です。

```tsx
// app/slow/page.tsx
import { connection } from "next/server";

// 2秒かかるデータ取得のまね（本物ならデータベースへの問い合わせ）
async function getBooksSlowly() {
  await new Promise((resolve) => setTimeout(resolve, 2000));
  return ["リーダブルコード", "達人プログラマー"];
}

export default async function SlowPage() {
  // 「アクセスのたびにサーバーで実行する」と宣言（ビルド時の作り置きをさせない）
  await connection();
  const books = await getBooksSlowly();

  return (
    <ul>
      {books.map((title) => (
        <li key={title}>{title}</li>
      ))}
    </ul>
  );
}
```

```tsx
// app/slow/loading.tsx（page.tsx の準備ができるまで代わりに表示される）
export default function SlowLoading() {
  return <p>読み込み中です...</p>;
}
```

```tsx
// stream-check.mjs（プロジェクトの外でもよい。ページが「何秒目に・どんな中身で」届くかを調べる）
const start = Date.now();
const response = await fetch("http://localhost:3000/slow");
const decoder = new TextDecoder();

// レスポンスの本文を「届いた塊（チャンク）ごと」に読む
for await (const chunk of response.body) {
  const text = decoder.decode(chunk, { stream: true });
  const seconds = ((Date.now() - start) / 1000).toFixed(2);
  const found = [];
  if (text.includes("読み込み中です")) found.push("loading.tsx の中身");
  if (text.includes("リーダブルコード")) found.push("page.tsx の中身");
  console.log(`${seconds}秒: ${chunk.length}バイト受信 ${found.join(" / ")}`);
}
```

▼ 実行結果（`npm run build` → `npm run start` で起動したあと、別のターミナルで `node stream-check.mjs` を実行）

```
0.17秒: 1659バイト受信 loading.tsx の中身
0.17秒: 4835バイト受信 loading.tsx の中身
2.13秒: 259バイト受信 page.tsx の中身
2.13秒: 1190バイト受信 page.tsx の中身
2.13秒: 14バイト受信
```

▼ コードを1つずつ分解して解説

解説1: 0.17秒で「読み込み中です...」が届いている

- 最初の2つの塊は、アクセスしてから **0.17秒** で届いています。中身は `loading.tsx` の「読み込み中です...」を含むHTML です。データ取得（2秒）を待たずに、まず「枠と読み込み中の表示」だけを先に送っているのです。
- これが図4-2 の①の画面です。利用者は真っ白な画面で待たされず、すぐに「読み込み中」と分かります。

解説2: 2.13秒で、同じ通信の続きとして本体が届く

- 2秒の待ち時間が終わった **2.13秒** の時点で、`page.tsx` の中身（書籍の一覧）が届いています。新しくページを読み直したのではなく、**同じ1回の通信の続き**として後から流れてきている点がポイントです。
- このように、できた部分から順番に送る仕組みを **ストリーミング（Streaming：流し送り）** と呼びます。ブラウザは後から届いた部分で「読み込み中です...」を差し替えます（図4-2 の②）。

> **用語: チャンク（chunk）** ... 通信で届くデータの「ひと塊」。大きなデータは、いくつかの塊に分かれて少しずつ届きます。`for await (const chunk of response.body)` は「塊が届くたびに、中の処理を1回実行する」という書き方です。

6.4 Route Handlers（補足）

App Router で「自前のHTTP API」を作りたいときは、`app/api/.../route.ts` というファイルを置きます。これを **Route Handler**（ルートハンドラ：URLとHTTPメソッドに対応する関数を書くファイル）と呼びます。Server Component から直接DBにアクセスできる App Router では出番は減りましたが、外部から `fetch` で叩かれるエンドポイントを公開したい場合に使います。

> **▼ このコードがやること（先に日本語で）：** `app/api/.../route.ts` という予約名のファイルで、外部から `fetch` で呼べる自前の HTTP API を作ります。`GET` や `POST` という名前の関数を export すると、それぞれが同名の HTTP メソッドの処理になるのがポイントです。`NextResponse.json(...)` で JSON を返す、API の基本形を学べます。詳細はコード内コメントを参照してください。

```ts
// 予約名 `route.ts`。`/api/books` の HTTP API になる
// app/api/books/route.ts
// page.tsx と同じフォルダには置けない（ルートが衝突するため）

// レスポンスを作るヘルパー
import { NextResponse } from "next/server";

// 関数名 `GET` が HTTP メソッドに対応（POST/PUT/DELETE等も同様）
export async function GET() {
  // 仮データ
  const books = [
    { id: "1", title: "リーダブルコード" },
    { id: "2", title: "達人プログラマー" },
  ];
  // 配列を JSON にしてレスポンスとして返す
  return NextResponse.json(books);
}

// POST メソッド用。引数は Web 標準の Request
export async function POST(request: Request) {
  // リクエストボディの JSON を解釈
  const body = await request.json();
  // ここで body を使ってDB保存などを行う
  // 201 Created を返す例
  return NextResponse.json({ created: true, body }, { status: 201 });
}
```

6.5 next/image（補足）

画像は `next/image` の `<Image>` コンポーネントを使うと、自動でサイズ最適化・WebP変換・遅延ロードが行われます。

> **▼ このコードがやること（先に日本語で）**：画像を `<img>` ではなく `next/image` の `<Image>` で表示し、サイズ最適化・WebP変換・遅延読み込みを自動で効かせます。`width` / `height` はレイアウト崩れ防止のために必須、`alt` （代替テキスト）もアクセシビリティのために必須です。最重要画像には `preload` （Next.js 15 までは `priority` ）を付ける、という使い方を学べます。詳細はコード内コメントを参照してください。

```tsx
// default export の Image コンポーネント
import Image from "next/image";

export function BookCover() {
  return (
    <Image
      // public/covers/book.jpg を指す（public 直下は `/` で参照）
      src="/covers/book.jpg"
      // 代替テキスト（必須）
      alt="書籍の表紙"
      // 元画像のアスペクト比計算用
      width={200}
      // 画像のレイアウト崩れを防ぐ
      height={300}
      // LCP（最大コンテンツ描画）に関わる重要画像はこれを付ける
      //  （Next.js 15 までの priority。16 から preload という名前になった）
      preload
    />
  );
}
```

> **`public/` フォルダ**: 画像や favicon などの静的ファイルは `public/` に置きます。URL では `/` から直接参照します（例: `public/logo.png` → `/logo.png` ）。

6.6 next.config.js / next.config.ts（補足）

プロジェクトルートの `next.config.js`（または TypeScript 版 `next.config.ts`）は Next.js 全体の設定ファイルです。画像の許可ドメインや実験的機能の有効化などをここで指定します。

> **▼ このコードがやること（先に日本語で）**: プロジェクト全体の設定ファイル `next.config.ts` の基本形を学びます。ここでは `next/image` で外部サイトの画像を表示してよいドメインを `remotePatterns` で許可する例を示します（許可しないと外部画像は表示されません）。設定オブジェクトを作り `export default` する、という決まった書き方を押さえてください。詳細はコード内コメントを参照してください。

```ts
// 設定ファイル本体
// next.config.ts
// 型を取り込み
import type { NextConfig } from "next";

// 設定オブジェクト
const nextConfig: NextConfig = {
  // next/image の設定
  images: {
    // 外部URLを許可するリスト（明示しないと表示できない）
    remotePatterns: [
      { protocol: "https", hostname: "example.com" },
    ],
  },
  // experimental: { ... }
  // 実験的機能はここで ON にする
};

// default export することで Next.js が読み込む
export default nextConfig;
```

7. Server Actions

7.1 Server Actions とは

Server Actions は、**Client Component から直接サーバー上の関数を呼び出せる** 仕組みです。API ルートを別途作成する必要がなく、フォーム処理やデータの変更（作成・更新・削除）をシンプルに書けます。



Server Actions は `"use server"` ディレクティブで宣言します。`"use client"` がファイル全体を Client Component にするのと同じ要領で、`"use server"` はファイルや関数を「サーバー上で動く関数」として印を付ける役割を持ちます。

7.2 フォーム処理での利用

基本的な Server Action

> **▼ このコードがやること（先に日本語で）**：フォーム送信を処理する関数（Server Action）を作ります。ファイル先頭の `"use server"` が「この中の関数はサーバー上で動く」という宣言で、これにより別途 API を作らなくてもフォームの保存処理が書けます。送信された値を `formData.get("title")` で取り出し、入力チェック（バリデーション）してから保存する流れを学べます。詳細はコード内コメントを参照してください。

```ts
// Server Action 用の専用ファイル
// app/books/new/actions.ts
// ファイル先頭の `"use server"` で「この中の関数は全部サーバー実行」と宣言
"use server";

// この関数はサーバー上でのみ実行される
// クライアントには関数の中身は送信されない
// 引数 `FormData` は Web標準のフォーム値コンテナ
// <form action={createBook}> の送信内容が自動で渡される
export async function createBook(formData: FormData) {
  // `formData.get("title")` で name="title" の値を取り出す
  // 戻り値は `FormDataEntryValue | null` なので `as string` で型を絞り込む
  const title = formData.get("title") as string;
  // 同じく著者
  const author = formData.get("author") as string;
  // 数値に変換（formData の値は基本文字列）
  const publishedYear = Number(formData.get("publishedYear"));

  // バリデーション
  // 必須項目が空ならエラーを返す
  if (!title || !author) {
    // オブジェクトを返すと呼び出し側で state として扱える
    return {
      error: "タイトルと著者は必須です",
    };
  }

  // データベースに保存（例）
  // 本番ではここで Supabase や Prisma で INSERT
  // const book = await db.book.create({
  //   data: { title, author, publishedYear },
  // });

  // サーバーのコンソールに出力（ブラウザには出ない）
  console.log("書籍を追加:", { title, author, publishedYear });

  // 成功したら書籍一覧にリダイレクト
  // redirect("/books");
  // `redirect()` を呼べば一覧ページに自動遷移
  // 成功フラグを返す
  return { success: true };
}
```

▼ コードを1つずつ分解して解説

この Server Action には「`"use server"` 宣言」「`FormData` からの値の取り出し」「バリデーション」という、フォーム処理の基本が詰まっています。順に見ていきましょう。

解説1: ファイル先頭の `"use server"` で「サーバー専用の関数」と宣言する

```
"use server";

export async function createBook(formData: FormData) {
```

- `"use server"`（ユーズ・サーバー）は、ファイルの**一番上**に書くディレクティブです。これを書くと、このファイルの関数は「**サーバー上でだけ実行される**」関数（Server Action）になります。
- `"use client"` がファイルを「ブラウザ側」に切り替えるのと対になる存在です。`"use server"` で印を付けた関数の中身は**ブラウザに送られない**ので、DB接続情報などの機密を安全に扱えます。
- `createBook` は `async` 関数で、引数に `FormData` を1つ受け取ります。この関数を `<form action={createBook}>` のように指定すると、フォーム送信時にサーバーで実行されます。

> **用語: Server Action（サーバーアクション）** … `"use server"` を付けた、サーバー上で実行される関数。別途API（route.ts）を作らなくても、フォーム送信やデータ更新の処理を直接書けます。

解説2: `FormData` から `get()` で入力値を取り出す

```
  const title = formData.get("title") as string;
  const author = formData.get("author") as string;
  const publishedYear = Number(formData.get("publishedYear"));
```

- `formData`（フォームデータ）は、送信されたフォームの入力値が全部入った「Web標準の箱」です。
- `formData.get("title")` は「`name="title"` の入力欄の値を取り出す」という意味です。**取り出すキーは、各 `<input>` の `name` 属性と一致します**（`id` ではない点に注意）。
- `as string` は型アサーションで、「`get()` の戻り値（`文字列 | null` というあいまいな型）を、文字列として扱ってね」とTypeScriptに伝えています。
- `Number(formData.get("publishedYear"))` は、取り出した値を**数値に変換**しています。フォームの値は基本すべて文字列なので、数値が必要なときは `Number(...)` で変換します。

> **用語: FormData** … HTMLフォームの入力値を「キー（name）→値」の形でまとめて持つWeb標準のオブジェクト。`.get("キー名")` でそれぞれの値を取り出します。

解説3: バリデーション（入力チェック）して、ダメなら error を返す

```
if (!title || !author) {
  return {
    error: "タイトルと著者は必須です",
  };
}
```

- `if (!title || !author)` は「タイトルが空、**または**著者が空なら」という条件です。`!title` は「`title` が空（falsy）なら true」、`||` は「または」です。
- 必須項目が欠けていれば、`{ error: "..." }` というオブジェクトを `return` します。このオブジェクトは、呼び出し側（フォーム）で「エラーメッセージの表示」に使えます（後述の `useActionState` の例で活用します）。
- ここを通り抜けたとき（＝両方入力されているとき）だけ、下のDB保存処理に進みます。

> **用語: バリデーション（validation）** … 入力された値が正しいか（必須項目が空でないか等）をチェックすること。不正なデータがDBに保存されるのを防ぐ、フォーム処理の必須ステップです。

Server Component のフォーム（JavaScript 不要）

> **▼ このコードがやること（先に日本語で）**：先ほどの Server Action を使う「書籍追加フォーム」を Server Component として作ります。`<form action={submitBook}>` のように `action` 属性に関数を直接渡せるのが Next.js（React 19）の拡張で、JavaScript が動かない環境でも送信できる堅牢な作りになります（Progressive Enhancement）。`<form action>` に直接渡す関数は「何も返さない」形にする決まりがあるので、結果を返す `createBook` を、何も返さない `submitBook` で包んで渡しています。各 `<input>` の `name` 属性が、サーバー側で値を取り出すときのキー名になる点も重要です。詳細はコード内コメントを参照してください。

```tsx
// 新規追加ページ。"use client" を書かないので Server
// app/books/new/page.tsx（Server Component）

// 同じフォルダの actions.ts から Server Action を取り込む
import { createBook } from "./actions";

// 同期関数でOK（このページ自体は async データ取得をしない）
export default function NewBookPage() {
  // <form action> に直接渡す Server Action は「何も返さない」形にする
  // createBook は結果（error など）を返すので、何も返さない関数で包む
  async function submitBook(formData: FormData) {
    // この関数の中身もサーバーで実行される、という印
    "use server";
    await createBook(formData);
  }

  return (
    <div>
      <h1>書籍を追加</h1>

      {/* `<form action={...}>` に関数を渡せるのが Next.js の拡張 */}
      {/* JavaScript が無効でも動作する（Progressive Enhancement） */}
      {/* 送信時に submitBook（→ createBook）がサーバーで実行される */}
      <form action={submitBook}>
        <div>
          {/* `htmlFor` は HTML の `for` 属性のJSX版（forはJSの予約語） */}
          <label htmlFor="title">タイトル</label>
          <input
            // テキスト入力
            type="text"
            // label と紐付けるための id
            id="title"
            // ※ formData.get で取り出すキー名はこの name 属性
            name="title"
            // 入力必須（ブラウザの標準バリデーション）
            required
          />
        </div>

        <div>
          <label htmlFor="author">著者</label>
          <input
            type="text"
            id="author"
            name="author"
            required
          />
        </div>
```

```
        <div>
          <label htmlFor="publishedYear">出版年</label>
          <input
            // 数値入力欄。上下ボタンが付く
            type="number"
            id="publishedYear"
            name="publishedYear"
          />
        </div>

        {/* submit ボタン */}
        <button type="submit">追加する</button>
      </form>
    </div>
  );
}
```

画面にはこう表示される:「書籍を追加」という見出しの下に、タイトル、著者、出版年の入力フィールドと「追加する」ボタンが表示されます。各フィールドに入力して「追加する」ボタンを押すと、フォームのデータがサーバーに送信され、`createBook` 関数がサーバー上で実行されます。

> **注意（`createBook` をそのまま `action` に渡すと型エラー）：** `<form action={createBook}>` と直接渡したくなりますが、`createBook` は `{ error: ... }` などの結果を**返す**関数なので、`npx tsc --noEmit` で次のエラーになります（「`action` に渡せるのは、何も返さない（`void`）関数だけ」という意味）。

```
app/books/new/page.tsx(18,13): error TS2322: Type '(formData: FormData) => Promise<{
error: string; success?: undefined; } | { success: boolean; error?: undefined; }>' is
not assignable to type 'string | ((formData: FormData) => void | Promise<void>) |
undefined'.
```

> 上のコードのように何も返さない関数で包むか、次の Client Component 版のように `useActionState` を使って結果を受け取るようにします。関数の本体の先頭に `"use server";` と書くと、その関数1つだけを Server Action にできます（ファイルの先頭に書くと、ファイル内の全関数が Server Action になります）。

Client Component のフォーム（ローディング状態の管理）

> **▼ このコードがやること（先に日本語で）：**同じフォームを、送信中の状態やエラー表示まで扱える Client Component 版に作り変えます。React 19 の `useActionState` フックを使うと、Server Action と連携しながら「現在の状態・加工済みの送信関数・送信中フラグ」の3つを受け取れます。送信中はボタンを「追加中…」にしてクリック不可にし、エラーがあれば画面に出す、という親切な UI が作れます。詳細はコード内コメントを参照してください。

```tsx
// クライアント側でローディング状態を扱えるフォーム
// components/BookFormClient.tsx
// useActionState は React のフックなので Client 必須
"use client";

// React 19 で追加された Server Action 連携用フック
// 旧名は useFormState（同じ用途）
import { useActionState } from "react";
// 別ファイルの Server Action を取り込む
import { createBook } from "@/app/books/new/actions";

// フォームの状態（Server Action が返した結果）の型
type FormState = { error?: string; success?: boolean } | null;

// useActionState に渡す関数は「(前回の状態, フォームの中身)」の2つを受け取る形にする
async function submitBook(
  _prevState: FormState,
  formData: FormData
): Promise<FormState> {
  // 中身は Server Action の createBook に任せる
  return createBook(formData);
}

export function BookFormClient() {
  // 戻り値は [現在のstate, ラップ済み formAction, 送信中フラグ]
  // 第2引数の null は state の初期値
  // formAction は <form action={...}> に渡せるよう加工された関数
  const [state, formAction, isPending] = useActionState(submitBook, null);

  return (
    // 元の createBook ではなく、ラップ済みの formAction を渡す
    <form action={formAction}>
      <div>
        <label htmlFor="title">タイトル</label>
        {/* name="title" が formData のキーになる */}
        <input type="text" id="title" name="title" required />
      </div>

      <div>
        <label htmlFor="author">著者</label>
        <input type="text" id="author" name="author" required />
      </div>

      <div>
        <label htmlFor="publishedYear">出版年</label>
        <input type="number" id="publishedYear" name="publishedYear" />
      </div>

      {/* エラーメッセージの表示 */}
```

```
      {/* `?.` はオプショナルチェイニング（state が null でも安全） */}
      {state?.error && (
        // Server Action が返した error メッセージを表示
        <p className="error-message">{state.error}</p>
      )}

      {/* 送信中はボタンを無効化 */}
      {/* `disabled={isPending}` で送信中はクリック不可 */}
      <button type="submit" disabled={isPending}>
        {/* ボタンラベルも切り替え */}
        {isPending ? "追加中..." : "追加する"}
      </button>
    </form>
  );
}
```

画面にはこう表示される: フォームは前の例と同じ見た目ですが、「追加する」ボタンを押すと、ボタンのテキストが「追加中...」に変わり、ボタンがグレーアウトして再クリックできなくなります。バリデーションエラーがあれば、ボタンの上にエラーメッセージが赤字で表示されます。

▼ コードを1つずつ分解して解説

このクライアントフォームの核心は React 19 の `useActionState` フックです。「3つの戻り値」「ラップ済み関数の使い方」「送信中の状態表示」を順に見ていきましょう。

解説1: `useActionState` で Server Action と連携する

```
"use client";

import { useActionState } from "react";
import { createBook } from "@/app/books/new/actions";

type FormState = { error?: string; success?: boolean } | null;

async function submitBook(
  _prevState: FormState,
  formData: FormData
): Promise<FormState> {
  return createBook(formData);
}

export function BookFormClient() {
  const [state, formAction, isPending] = useActionState(submitBook, null);
```

- `useActionState`（ユーズ・アクション・ステート）は、React 19 で追加された「**Server Action とフォームをつなぐ**」フックです（旧名は `useFormState`）。ブラウザ側のフックなので `"use client"` が必須です。

- 第1引数に「送信されたときに呼ぶ関数」（`submitBook`）、第2引数に state の初期値（`null`）を渡します。
- `useActionState` に渡す関数は、**1つ目の引数に「前回の state」、2つ目の引数に「フォームの中身（FormData）」**を受け取る決まりです。`createBook` は FormData の1つだけを受け取る形なので、2つ受け取る `submitBook` で包み、中で `createBook(formData)` を呼んでいます。`_prevState` の先頭の `_` は「受け取るけれど使わない」という目印です。
- 戻り値は3つの値の配列で、分割代入で受け取ります。
- `state`：`submitBook`（つまり `createBook`）が `return` した値（例: `{ error: "..." }`）。
- `formAction`：`<form action={...}>` に渡せるよう**加工済みの送信関数**。
- `isPending`：送信処理中かどうかの `true` / `false`。

> **用語: useActionState** ... Server Action の「実行結果(state)」「送信用の関数」「送信中フラグ」をまとめて受け取れるReactフック。フォームの送信状態を簡単に扱えます。

> **つまずきポイント（`useActionState` に、FormData だけを受け取る関数を渡す）：**
> `useActionState(createBook, null)` と、Server Action をそのまま渡したくなりますが、`useActionState` は渡した関数を「`(前回の state, FormData)`」の順で呼び出します。`createBook(formData)` のように引数が1つの関数を渡すと、`formData` に**前回の state（最初は `null`）**が入ってしまい、`formData.get(...)` で落ちます。TypeScript も型チェックで次のように教えてくれます（`npx tsc --noEmit` の出力の先頭部分）。

```
components/BookFormClient.tsx(16,57): error TS2769: No overload matches this call.
  Overload 1 of 2, '(action: (state: { error: string; success?: undefined; } | {
success: boolean; error?: undefined; } | null) => { error: string; success?: undefined;
} | { success: boolean; error?: undefined; } | Promise<{ error: string; success?:
undefined; } | { ...; } | null> | null, initialState: { ...; } | ... 1 more ... | null,
permalink?: string | undefined): [state: ...]', gave the following error.
    Argument of type '(formData: FormData) => Promise<{ error: string; success?:
undefined; } | { success: boolean; error?: undefined; }>' is not assignable to
parameter of type '(state: { error: string; success?: undefined; } | { success:
boolean; error?: undefined; } | null) => { error: string; success?: undefined; } | {
success: boolean; error?: undefined; } | Promise<{ error: string; success?: undefined;
} | { ...; } | null> | null'.
      Types of parameters 'formData' and 'state' are incompatible.
        Type '{ error: string; success?: undefined; } | { success: boolean; error?:
undefined; } | null' is not assignable to type 'FormData'.
          Type 'null' is not assignable to type 'FormData'.
```

```
// ❌ NG: 1つ目の引数に state が入るのに、FormData を受け取るつもりの関数を渡している
const [state, formAction, isPending] = useActionState(createBook, null);
```

```ts
// ✅ OK: （前回の state, FormData）の2つを受け取る関数で包んでから渡す
async function submitBook(
  _prevState: FormState,
  formData: FormData
): Promise<FormState> {
  return createBook(formData);
}

const [state, formAction, isPending] = useActionState(submitBook, null);
```

> エラー文の最後の `Types of parameters 'formData' and 'state' are incompatible` （引数 formData と state の型が合わない）が決め手です。最初から `useActionState` 用に作るなら、Server Action 自体を `createBook(prevState, formData)` の形で書いてもかまいません。

解説2: 元の関数ではなく「ラップ済みの formAction」を渡す

```tsx
  return (
    <form action={formAction}>
      <div>
        <label htmlFor="title">タイトル</label>
        <input type="text" id="title" name="title" required />
      </div>
      {/* ...他の入力欄... */}
```

- `<form action={formAction}>` のように、`action` 属性には**元の `createBook` ではなく、`useActionState` が返した `formAction`** を渡します。これにより、送信時に state の更新や `isPending` の管理が自動で行われます。
- 各 `<input>` の `name` 属性（`name="title"` など）が、Server Action 側で `formData.get("title")` として値を取り出すときのキーになります。`required` はブラウザ標準の「入力必須」チェックです。

> **用語: action 属性** … Next.js では `<form>` の `action` に「関数」を直接渡せます（通常のHTMLでは送信先URLを書く場所）。これがServer Actionと連携する入口になります。

解説3: エラー表示と「送信中はボタンを無効化」

```
      {state?.error && (
        <p className="error-message">{state.error}</p>
      )}

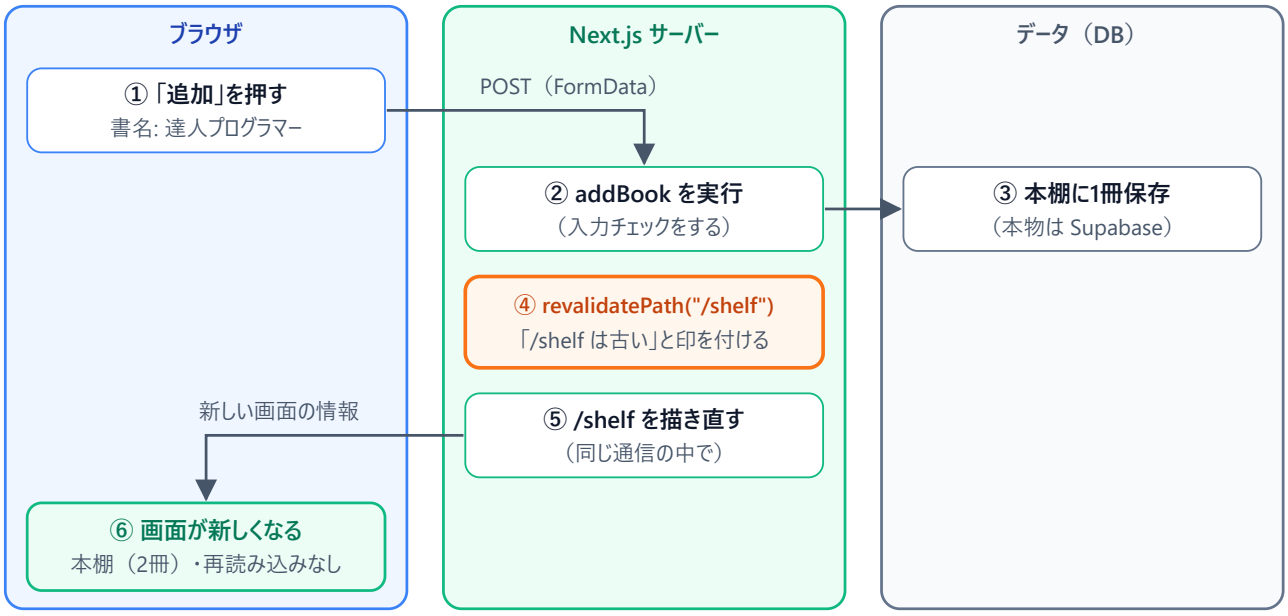
      <button type="submit" disabled={isPending}>
        {isPending ? "追加中..." : "追加する"}
      </button>
```

- `{state?.error && (...)}` … `state?.error` の `?.`（オプショナルチェイニング）は「`state` が `null` でも安全に `error` を読む」書き方です。エラーメッセージがあるときだけ、その右の `<p>` を表示します（`&&` による条件付き表示）。
- `disabled={isPending}` … 送信処理中（`isPending` が `true`）の間、ボタンを**クリック不可（グレーアウト）**にします。これで二重送信を防げます。
- `{isPending ? "追加中..." : "追加する"}` … 三項演算子で、送信中はボタンの文字を「追加中…」に切り替えます。

> **用語: オプショナルチェイニング（`?.`）** … `state?.error` のように書くと、`state` が `null` / `undefined` のときはエラーにならず `undefined` を返します。「値があるか分からないオブジェクト」を安全にたどれます。

送信してから画面が新しくなるまで（revalidatePath）

Server Action でデータを保存したら、一覧の画面にも新しいデータが出てほしいですよね。ここで大事になるのが `revalidatePath`（リバリデート・パス：この URL の画面を作り直して、と伝える関数）です。お店にたとえると、「在庫を補充した店員が、店頭の在庫表示の札を『古い』と書き換えて、新しい札に取り替える」作業にあたります。補充（保存）だけして札を替えないと、お客さん（画面）には古い在庫数が見え続けます。



revalidatePath を忘れると ④⑤ が起きず、保存はされても画面は古いまま
（下の「つまずきポイント」で実験）

図4-10: Server Action でフォームを送ってから、画面が新しくなるまで

▼ このコードがやること（先に日本語で）：書名を入力して「追加」を押すと、本棚（書籍の一覧）に1冊増える小さなページを作ります。データベースの代わりに、サーバーのメモリ上の配列（ `shelf` ）に保存します。保存したあとに `revalidatePath("/shelf")` を呼んで、「 `/shelf` の画面を作り直して」と Next.js に伝えるのがポイントです。

```
// lib/shelf.ts（データベースの代わりに、サーバーのメモリ上の配列を使う）
export const shelf: string[] = ["リーダブルコード"];
```

```ts
// app/shelf/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { shelf } from "@/lib/shelf";

export async function addBook(formData: FormData) {
  // name="title" の入力欄の値を取り出し、前後の空白を取り除く
  const title = String(formData.get("title") ?? "").trim();
  if (!title) {
    return;
  }

  // データを保存する（本物のアプリでは Supabase に INSERT する）
  shelf.push(title);
  console.log(`[addBook] 「${title}」を追加しました（全${shelf.length}冊）`);

  // 「/shelf の画面は古くなった」と Next.js に伝え、作り直してもらう
  revalidatePath("/shelf");
}
```

```tsx
// app/shelf/page.tsx（Server Component）
import { shelf } from "@/lib/shelf";
import { addBook } from "./actions";

export default function ShelfPage() {
  return (
    <div>
      <h1>本棚（{shelf.length}冊）</h1>
      <ul>
        {shelf.map((title) => (
          <li key={title}>{title}</li>
        ))}
      </ul>
      {/* action に Server Action を渡す */}
      <form action={addBook}>
        <input name="title" placeholder="書名" />
        <button type="submit">追加</button>
      </form>
    </div>
  );
}
```

▼ 実行結果（`npm run dev` で `http://localhost:3000/shelf` を開き、「達人プログラマー」と入力して「追加」を押したとき）

ブラウザの表示（`<form>` より前の部分。上が押す前、下が押した後）：

```
<div><h1>本棚（1冊）</h1><ul><li>リーダブルコード</li></ul>
<div><h1>本棚（2冊）</h1><ul><li>リーダブルコード</li><li>達人プログラマー</li></ul>
```

ターミナルの出力（押した後の部分）：

```
[addBook] 「達人プログラマー」を追加しました（全2冊）
[RootLayout] サーバーで描画しました
 POST /shelf 200 in 31ms (next.js: 2ms, application-code: 29ms)
  └─ ƒ addBook({}) in 2ms app/shelf/actions.ts
```

▼ コードを1つずつ分解して解説

解説1: フォーム送信は POST として届き、Server Action が動く

- ターミナルの `POST /shelf` が、図4-10 の①→② です。Server Action は、裏では **POST**（データを送る種類の通信）で呼び出されています。
- `└─ ƒ addBook({}) ...` は、Next.js 16 の開発サーバーが「`addBook` が呼ばれた」ことを表示してくれている行です（`{}` は FormData を簡単に表示したもの）。
- `[addBook] ...（全2冊）` のログがターミナルに出ていることから、保存処理（図4-10 の③）がサーバーで実行されたことが分かります。

解説2: `revalidatePath` のおかげで、同じ通信の中で画面が新しくなる

```
  revalidatePath("/shelf");
```

- ログの次に `[RootLayout] サーバーで描画しました` が出ています。これが図4-10 の⑤で、`revalidatePath` で「古い」と印を付けられた `/shelf` を、Next.js がその場で描き直しています。
- ブラウザの表示は、ページの再読み込みをしていないのに「本棚（2冊）」に変わりました（⑥）。送信と画面の更新が**1回の通信**で済むのが Server Actions の便利なところです。
- `revalidatePath` の引数には、新しくしたい画面の URL を書きます。書籍管理アプリなら、一覧のトップページ `/` や、詳細の `/books/42` などです。

> **ヒント**: 保存したあとに別のページへ移動したいときは、`revalidatePath("/")` のあとに `redirect("/")` を呼びます（`next/navigation` の `redirect`）。順番は「**作り直しの印を付けてから、移動する**」です。`redirect` を先に呼ぶと、そこで処理が止まって後ろの行が実行されません（5.3 の「つまずきポイント」と同じく、`try` の中で呼ばないことにも注意）。

> **つまずきポイント（`revalidatePath` を忘れて、保存したのに画面が古いまま）**：保存処理だけ書いて `revalidatePath` を書き忘れると、データは保存されているのに画面が変わりません。開発モードではページを再読み込みすれば新しくなるので気づきにくく、**本番モード**（`npm run build` → `npm run start`）で初めて「何度読み込み直しても増えない」と困ることになります。`revalidatePath` の行をコメントにして本番モードで試した記録です。

```
// ❌ NG: app/shelf/actions.ts（保存はするが、画面を作り直すよう伝えていない）
  shelf.push(title);
  console.log(`[addBook] 「${title}」を追加しました（全${shelf.length}冊）`);
  // revalidatePath("/shelf");
```

ブラウザの表示（上から「押す前」「押した後」「再読み込みした後」）：

```
<div><h1>本棚（1冊）</h1><ul><li>リーダブルコード</li></ul>
<div><h1>本棚（1冊）</h1><ul><li>リーダブルコード</li></ul>
<div><h1>本棚（1冊）</h1><ul><li>リーダブルコード</li></ul>
```

ターミナルの出力:

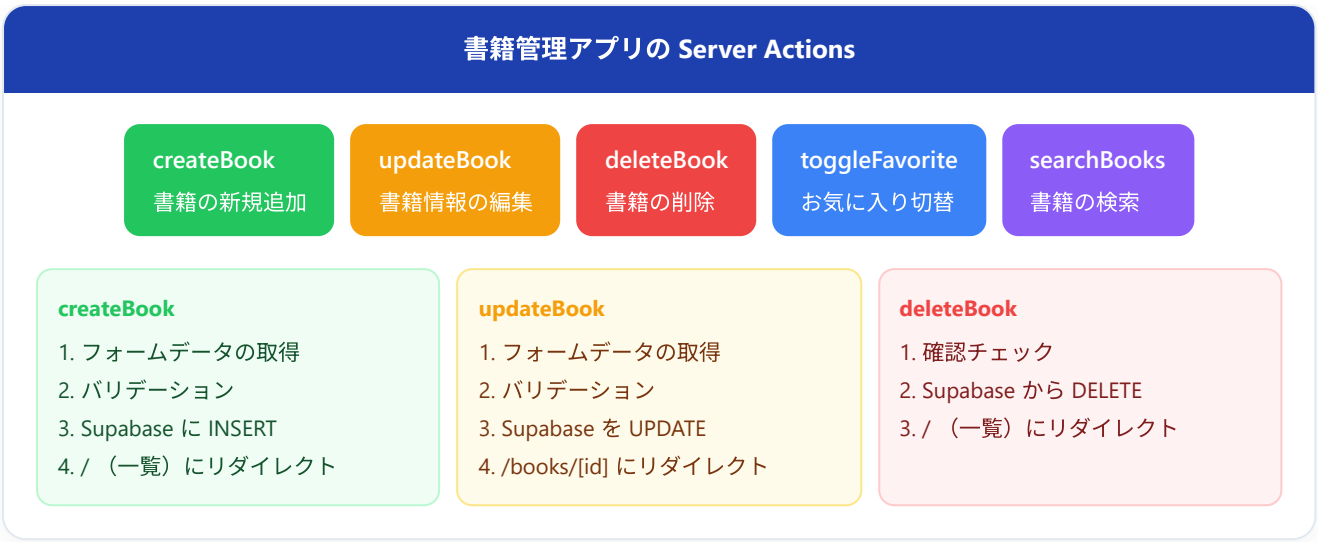
```
[addBook] 「達人プログラマー」を追加しました（全2冊）
```

```
// ✅ OK: 保存したら、その結果を表示している画面の URL を revalidatePath に渡す
  shelf.push(title);
  console.log(`[addBook] 「${title}」を追加しました（全${shelf.length}冊）`);
  revalidatePath("/shelf");
```

> ターミナルには「全2冊」と出ているのに、画面はずっと1冊のままです。これは `/shelf` がビルド時に作り置き（`○ Static`）されていて、誰も「作り直して」と言っていないからです（6.1 のつまずきポイントと同じ仕組み）。OK 版に戻して同じ手順を試すと、押した直後も再読み込み後も「本棚（2冊）」になりました。**データを変えたら `revalidatePath` とセットで覚えましょう。**

7.3 書籍の追加/編集での利用予告

書籍管理アプリの「追加・編集・削除」を Server Actions で作るとしたら、次のような関数になります:



> **予告:** 第7〜8章で、実際のデータベース（Supabase）とつないで、書籍の追加・編集・削除を作ります。本書のアプリでは、学びやすさを優先してブラウザ側（Client Component）から Supabase を呼び出す方法を主に使い、Server Actions で書く方法は参考として紹介します。どちらの方法でも、この章で学んだ「保存したら画面を最新にする（ `revalidatePath` や `router.refresh()` ）」という考え方は同じです。

---

8. 環境変数

8.1 .env.local の設定方法

Next.js では、プロジェクトルートに `.env.local` ファイルを作成して環境変数を設定します。

> **▼ このコードがやること（先に日本語で）:** APIキーやデータベース接続情報などの設定値を、コードに直接書かずに `.env.local` というファイルにまとめて管理します。`KEY="値"` の形で1行に1つ書きます。変数名の先頭に `NEXT_PUBLIC_` を付けるとブラウザにも公開され、付けないとサーバー側だけで使える（＝機密情報を守れる）という違いが最大のポイントです。詳細はコード内コメントを参照してください。

```
# .env.local（プロジェクトルートに作成）
# `#` から始まる行はコメント。シェル/dotenv 共通の記法

# データベース接続情報（サーバーサイドのみ）
# NEXT_PUBLIC_ なしなのでブラウザ用の JavaScript には入らない
# `KEY="VALUE"` の形で1行1変数。`=` の前後にスペースを入れない
DATABASE_URL="postgresql://user:password@localhost:5432/bookapp"

# Supabase 接続情報（サーバーサイドのみ）
# 機密キーには絶対 NEXT_PUBLIC_ を付けない
# Supabase プロジェクトのURL
SUPABASE_URL="https://xxxxx.supabase.co"
# Secret key（旧 service_role key。DB全権限。絶対に公開してはいけない）
SUPABASE_SERVICE_ROLE_KEY="sb_secret_xxxxxxxxxxxxxxxx"

# Supabase 接続情報（クライアントサイドでも使用可能）
# NEXT_PUBLIC_ を付けると、ブラウザ用の JavaScript にも値が埋め込まれる
# 同じURLでも公開してよい想定なら NEXT_PUBLIC_ を付ける
NEXT_PUBLIC_SUPABASE_URL="https://xxxxx.supabase.co"
# Publishable key（旧 anon key。Row Level Security で制限される前提なので公開OK）
NEXT_PUBLIC_SUPABASE_ANON_KEY="sb_publishable_xxxxxxxxxxxxxxxx"
```

> **重要:** `.env.local` は `.gitignore` に含まれているため、Git にコミットされません。APIキーやパスワードなどの機密情報を安全に管理できます。

8.2 NEXT_PUBLIC_ プレフィックス

環境変数のアクセス範囲は、変数名のプレフィックスによって決まります。

NEXT_PUBLIC_ あり	NEXT_PUBLIC_ なし
NEXT_PUBLIC_SUPABASE_URL	SUPABASE_SERVICE_ROLE_KEY
✅ Server Component	✅ Server Component
✅ Client Component	❌ Client Component（ブラウザでは undefined）
⚠️ ブラウザの JS に含まれる	✅ ブラウザ用の JS には入らない
	⚠️ Client で表示すると HTML に出る（下記）

▼ **このコードがやること（先に日本語で）**：Server Component の中で環境変数を読み取る例です。サーバー側では `process.env.変数名` で、`NEXT_PUBLIC_` 付きでも無しでも、すべての環境変数にアクセスできます。機密キーもここでなら安全に使えます（ブラウザには送られない）。`console.log` の出力もブラウザではなくサーバーのターミナルに出る点を意識してください。詳細はコード内コメントを参照してください。

```
// サーバー側ではすべての環境変数にアクセスできる
// Server Component での使用
// app/books/page.tsx

export default async function BooksPage() {
  // ✅ どちらもアクセス可能
  // `process.env` は Node.js 標準の環境変数オブジェクト
  const url = process.env.SUPABASE_URL;
  // 機密キー。Server Component なので安全に取得可能
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  // 公開用キーも同様に取れる
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;

  // "https://xxxxx.supabase.co"
  // サーバーのコンソールに出力（ブラウザDevToolsには出ない）
  console.log(url);
  // "sb_secret_..."
  // ※ 本番ではこれをログに出さないこと
  console.log(serviceKey);
  // "https://xxxxx.supabase.co"
  console.log(publicUrl);

  // 省略表記
  return <div>...</div>;
}
```

▼ **このコードがやること（先に日本語で）**：同じ `process.env` でも、Client Component（ブラウザ側）では `NEXT_PUBLIC_` 付きの変数しか読めない、という違いを確かめる例です。`NEXT_PUBLIC_` なしの機密キーを参照すると、ブラウザでは `undefined`（値なし）になります。ブラウザ用の JavaScript に秘密の値が入らないようにするための仕様です。ただし「だから Client Component で秘密の変数を書いても安全」ではない点に注意してください（このあとの「つまずきポイント」で実験します）。詳細はコード内コメントを参照してください。

```tsx
// ブラウザ側では NEXT_PUBLIC_ 付きしか参照できない
// Client Component での使用
// components/SomeClient.tsx
// Client Component 化
"use client";

export function SomeClient() {
  // ✅ NEXT_PUBLIC_ 付きはアクセス可能
  // ビルド時に値がJSバンドルに埋め込まれる
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  // "https://xxxxx.supabase.co"
  // ブラウザのコンソールに表示される
  console.log(publicUrl);

  // ❌ NEXT_PUBLIC_ なしは、ブラウザでは undefined
  // ブラウザ用の JS（バンドル）には値が入らない
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  // ブラウザのコンソールには undefined と出る
  // ※ ただし、この値を画面に表示すると、サーバーでの最初の描画で
  //    本物の値が HTML に入ってしまう（下の「つまずきポイント」）
  console.log(serviceKey);

  return <div>...</div>;
}
```

使い分けの原則:

環境変数の種類	プレフィックス	使用例
機密情報	なし	`SUPABASE_SERVICE_ROLE_KEY`、`DATABASE_URL`
公開しても安全な情報	`NEXT_PUBLIC_`	`NEXT_PUBLIC_SUPABASE_URL`、`NEXT_PUBLIC_SUPABASE_ANON_KEY`

環境変数は「いつ・どこに」入るのか

`NEXT_PUBLIC_` の正体は、「**ビルドのときに、値をブラウザ用の JavaScript に文字として書き込む**」という印です。たとえるなら、`.env.local` は金庫で、`NEXT_PUBLIC_` は「このメモはチラシに印刷して配ってよい」という付箋です。付箋の付いたメモだけがチラシ（ブラウザ用の JavaScript）に印刷され、付箋の無いメモは金庫（サーバー）から出ません。

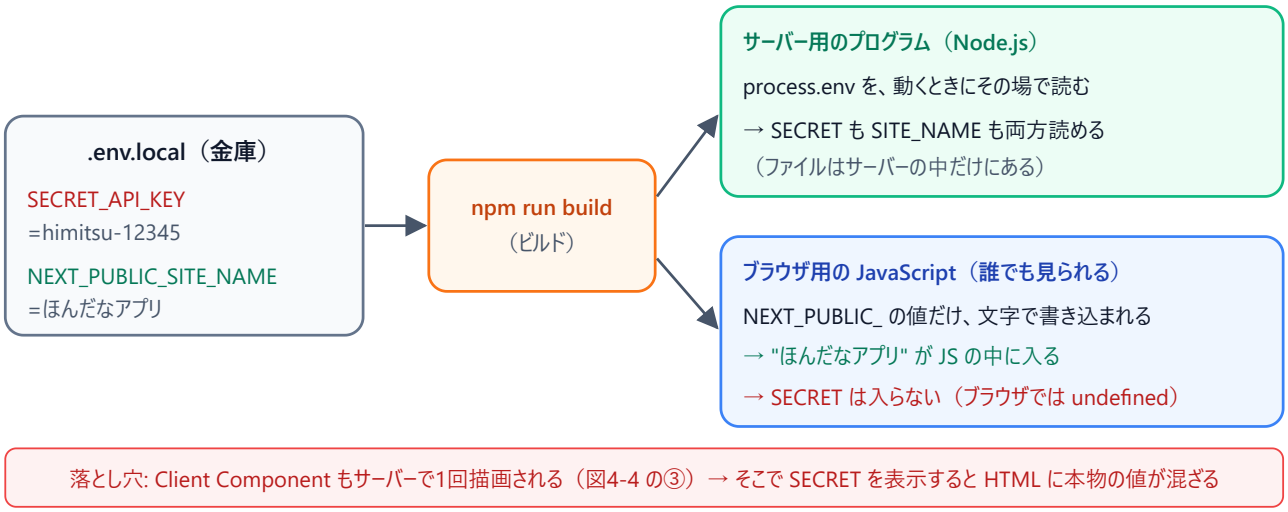


図4-11: 環境変数がどこに入るか（NEXT_PUBLIC_ はビルド時にブラウザ用 JS へ書き込まれる）

環境変数がどこに入ったかを確かめる

▼ **このコードがやること（先に日本語で）**：秘密のキー（ `SECRET_API_KEY` ）と公開してよいサイト名（ `NEXT_PUBLIC_SITE_NAME` ）を `.env.local` に書き、Server Component と Client Component の両方から読んでみます。さらにビルドしてできあがったファイルを検索し、それぞれの値が「ブラウザ用の JavaScript に入ったか」を確かめます。図4-11 が本当かどうかを、自分の目で確認する実験です。

```
# .env.local
SECRET_API_KEY=himitsu-12345
NEXT_PUBLIC_SITE_NAME=ほんだなアプリ
```

```
// app/env-check/page.tsx（Server Component）
import { EnvClient } from "./EnvClient";

export default function EnvCheckPage() {
  // サーバーでは NEXT_PUBLIC_ の有無に関係なく読める
  console.log("[server] SECRET_API_KEY =", process.env.SECRET_API_KEY);
  console.log("[server] NEXT_PUBLIC_SITE_NAME =", process.env.NEXT_PUBLIC_SITE_NAME);

  return (
    <div>
      <h1>環境変数チェック</h1>
      <EnvClient />
    </div>
  );
}
```

```tsx
// app/env-check/EnvClient.tsx（Client Component。実験のため、わざと秘密の値も読んでいる）
"use client";

export function EnvClient() {
  return (
    <ul>
      <li>サイト名: {process.env.NEXT_PUBLIC_SITE_NAME}</li>
      <li>秘密のキー: {String(process.env.SECRET_API_KEY)}</li>
    </ul>
  );
}
```

▼ 実行結果

`npm run dev` で `/env-check` を開いたときのターミナル（Server Component のログ）：

```
[server] SECRET_API_KEY = himitsu-12345
[server] NEXT_PUBLIC_SITE_NAME = ほんだなアプリ
```

`npm run build` のあと、ブラウザ用の JavaScript が置かれる `.next/static` フォルダを `grep -rl`（その文字を含むファイル名を表示するコマンド）で検索した結果です。秘密のキーはどのファイルにも無く（何も表示されない）、サイト名は1つのファイルに入っていました。

```
$ grep -rl "himitsu-12345" .next/static
$ grep -rl "ほんだなアプリ" .next/static
.next/static/chunks/0863qupusboaq.js
```

▼ コードを1つずつ分解して解説

解説1: サーバーでは、どちらの変数も読める

- ターミナルに両方の値が出ています。Server Component はサーバー（Node.js）で動くので、`.env.local` のすべての値を `process.env.〇〇` で読めます。
- 秘密のキーを使ってデータベースや外部サービスにアクセスする処理は、このように**サーバー側**に書きます。

解説2: ブラウザ用の JavaScript には `NEXT_PUBLIC_` の値だけが入る

- `.next/static` は、ブラウザにダウンロードされる JavaScript の置き場所です。そこに「ほんだなアプリ」という**値そのもの**が書き込まれていました。`NEXT_PUBLIC_` 付きの変数は、ビルドのときに `process.env.NEXT_PUBLIC_SITE_NAME` の部分が `"ほんだなアプリ"` という文字に置き換えられるからです。
- 一方「himitsu-12345」はどこにも無く、ブラウザで `process.env.SECRET_API_KEY` を読むと `undefined` になります。ここまでは図4-11 のとおりです。
- ところが、この実験にはもう1つ大事な落とし穴が隠れていました。次の「つまずきポイント」を見てください。

> **つまずきポイント（Client Component で秘密の環境変数を表示して、HTML に漏れる）**：上の `EnvClient` は「ブラウザでは `undefined` になるから安全」に見えますが、そうではありません。図4-4 の③のとおり、Client Component も**最初の1回はサーバーで描画**されます。そのとき `process.env.SECRET_API_KEY` には本物の値が入っているので、**サーバーが作った HTML に秘密の値がそのまま書き込まれます**。ビルドで作られた HTML（ `.next/server/app/env-check.html` ）を検索した実際の結果です。

```
<ul><li>サイト名: <!-- -->ほんだなアプリ</li><li>秘密のキー: <!-- -->himitsu-12345</li></ul>
```

さらにブラウザでは、サーバーの HTML（ `himitsu-12345` ）とブラウザでの描画（ `undefined` ）が食い違うので、開発者ツールの Console に Hydration エラーが出ます（先頭と、食い違った場所の部分を抜粋）。

```
Hydration failed because the server rendered text didn't match the client. As a result
this tree will be regenerated on the client. This can happen if a SSR-ed Client
Component used:
...
                        <EnvClient>
                          <ul>
                            <li>
                            <li>
+                             undefined
-                             himitsu-12345
```

画面の表示は最終的に「秘密のキー: undefined」に置き換わりますが、HTML を見れば誰でも `himitsu-12345` を読めてしまいます。

```tsx
// ❌ NG: Client Component の中で、NEXT_PUBLIC_ の無い秘密の変数を読む
"use client";

export function EnvClient() {
  return <li>秘密のキー: {String(process.env.SECRET_API_KEY)}</li>;
}
```

```tsx
// ✅ OK: app/env-ok/page.tsx（Server Component）
import { EnvClient } from "./EnvClient";

export default function EnvOkPage() {
  // 秘密のキーはサーバーの中で使うだけにする
  // Client Component には「設定されているかどうか」という結果だけを渡す
  const hasKey = Boolean(process.env.SECRET_API_KEY);

  return (
    <EnvClient
      siteName={process.env.NEXT_PUBLIC_SITE_NAME ?? ""}
      hasKey={hasKey}
    />
  );
}
```

```tsx
// ✅ OK: app/env-ok/EnvClient.tsx（Client Component は props で受け取って表示するだけ）
"use client";

export function EnvClient({
  siteName,
  hasKey,
}: {
  siteName: string;
  hasKey: boolean;
}) {
  return (
    <ul>
      <li>サイト名: {siteName}</li>
      <li>秘密のキー: {hasKey ? "設定済み" : "未設定"}</li>
    </ul>
  );
}
```

OK 版を `npm run dev` で開き、届いた HTML を検索した結果です。秘密の値は入っていません。

```
<ul><li>サイト名: <!-- -->ほんだなアプリ</li><li>秘密のキー: <!-- -->設定済み</li></ul>
```

> **ヒント（`server-only` で「うっかり」を防ぐ）**：秘密の値を使う関数を `lib/secret.ts` のようなファイルにまとめ、ファイルの先頭に `import "server-only";` と書いておくと、そのファイルを Client Component から import した時点でエラーになり、うっかり漏らすのを防げます（実際に試すと `'server-only' cannot be imported from a Client Component module` というエラーが出ました）。Next.js では `server-only` を別途インストールしなくても使えます。

つまずきポイント（ NEXT_PUBLIC_ の値を変えたのに、画面が古い値のまま）： NEXT_PUBLIC_ の値は**ビルドのとき**に JavaScript に書き込まれます（図4-11）。そのため、 .env.local を書き換えても、すでに起動しているサーバーやビルド済みのファイルには反映されません。開発中は npm run dev を一度止めて起動し直す、本番では**もう一度ビルドする**（Vercel なら再デプロイする）必要があります。また、変数名を文字列で組み立てて読む書き方は、置き換えの対象になりません。

```
// ❌ NG: 変数名を組み立てて読むと、ビルド時の置き換えが行われない（ブラウザでは undefined）
const name = "NEXT_PUBLIC_SITE_NAME";
const siteName = process.env[name];
```

```
// ✅ OK: process.env.NEXT_PUBLIC_〇〇 と、名前をそのまま書く
const siteName = process.env.NEXT_PUBLIC_SITE_NAME;
```

Next.js の公式ドキュメントにも「 process.env[varName] のような動的な参照は埋め込まれない」と書かれています。 NEXT_PUBLIC_ の変数は、必ず process.env. の後ろに名前をそのまま書きましょう。

8.3 Supabase の接続情報の管理

書籍管理アプリでは、Supabase を使います。環境変数の設定例は以下の通りです。

▼ このコードがやること（先に日本語で）：書籍管理アプリで実際に使う Supabase（データベースサービス）の接続情報を .env.local に設定する具体例です。URL と匿名キーは公開してよいので NEXT_PUBLIC_ を付け、全権限を持つサービスロールキーは絶対に公開しないため NEXT_PUBLIC_ を付けません。この付け分けがそのままセキュリティの境界になります。詳細はコード内コメントを参照してください。

注意（Supabase の API キーの名前）：2026年時点の Supabase ダッシュボードでは、API キーが **Publishable key**（ sb_publishable_ で始まる。公開してよい）と **Secret key**（ sb_secret_ で始まる。絶対に秘密）という名前で表示されます。以前の **anon key・service_role key**（ eyJ... で始まる長い文字列）は、2026年末に廃止予定の旧方式です。本書では変数名は NEXT_PUBLIC_SUPABASE_ANON_KEY のまま、値に Publishable key を貼り付けます（第5・6章と同じ）。大切なのは、 .env.local の**変数名**と、コードの process.env.〇〇 の名前が一字一句そろっていることです。

```
# .env.local
# 開発機ローカル用の環境変数ファイル

# Supabase プロジェクト URL（公開OK）
# ブラウザにも露出してOKな値。NEXT_PUBLIC_ 付きで Client/Server 両方で使える
NEXT_PUBLIC_SUPABASE_URL="https://abcdefghijklm.supabase.co"

# Supabase の Publishable key（公開OK。旧 anon key でも可）
# Row Level Security（RLS）で読める範囲が絞られている前提なので公開してよい
NEXT_PUBLIC_SUPABASE_ANON_KEY="sb_publishable_xxxxxxxxxxxxxxx"

# Supabase の Secret key（旧 service_role key。絶対に公開してはいけない）
# NEXT_PUBLIC_ なし＝Server 限定
SUPABASE_SERVICE_ROLE_KEY="sb_secret_xxxxxxxxxxxxxxxx"
```

▼ このコードがやること（先に日本語で）：ブラウザ側（Client Component）から Supabase に接続するための「クライアント」を作る関数を定義します。公開してよい `NEXT_PUBLIC_` 付きの URL と匿名キーだけを使うのがポイントです。`!`（non-null assertion）は「この値は undefined ではないと TypeScript に約束する」記号で、設定漏れがあると実行時にエラーになります。詳細はコード内コメントを参照してください。

```
// ブラウザ側で動く Supabase クライアント
// lib/supabase/client.ts
// Client Component 用の Supabase クライアント

// `@supabase/ssr` パッケージの import
import { createBrowserClient } from "@supabase/ssr";

// クライアントを生成する関数（毎回新しいインスタンスを作る）
export function createClient() {
  return createBrowserClient(
    // `!` は TypeScript の non-null assertion（「undefinedではないと断言」）
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    // 設定漏れがあるとここで undefined になり実行時エラーになる
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

▼ このコードがやること（先に日本語で）：サーバー側（Server Component）から Supabase に接続するクライアントを作ります。クライアント版との違いは、ログイン状態を保つために Cookie を読み書きする仕組み（`cookies()` を使ったアダプタ）を渡している点です。`cookies()` が Promise を返すため関数を `async` にして `await` する必要があります。やや複雑ですが、いまは「サーバー用は Cookie 連携が要る」とだけ掴めば十分です。詳細はコード内コメントを参照してください。

```ts
// サーバー側で動く Supabase クライアント
// lib/supabase/server.ts
// Server Component 用の Supabase クライアント

// サーバー用ファクトリ関数
import { createServerClient } from "@supabase/ssr";
// Next.js が提供する、リクエストの Cookie 読み書きAPI
import { cookies } from "next/headers";

// async にする必要があるのは cookies() が Promise を返すため
export async function createClient() {
  // 現在のリクエストの Cookie ストアを取得
  const cookieStore = await cookies();

  return createServerClient(
    // URL
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    // 匿名キー
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      // Cookie 入出力のアダプタを渡す
      cookies: {
        // 全 Cookie を返す関数
        getAll() {
          return cookieStore.getAll();
        },
        // Supabase がセッション更新時に呼ぶ関数
        setAll(cookiesToSet) {
          try {
            // 配列を1つずつ処理
            cookiesToSet.forEach(({ name, value, options }) =>
              // Next.js の cookie ストアに書き込み
              cookieStore.set(name, value, options)
            );
          } catch {
            // Server Component からの呼び出し時は
            // Server Component から cookie を書き込もうとすると例外が出るが
            // cookie のセットができないが、問題ない
            // 実害がないため握りつぶしてよい
          }
        },
      },
    }
  );
}
```

▼ コードを1つずつ分解して解説

このサーバー用 Supabase クライアントは、クライアント版と違って「Cookie 連携」が必要なため少し複雑です。「なぜ `async` なのか」「`cookies()` の取得」「Cookie アダプタ」を順に見ていきましょう。

解説1: `cookies()` を `await` するために関数を `async` にする

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
```

- `createServerClient` は「**サーバー側で動く Supabase クライアントを作る**」関数です。`@supabase/ssr` パッケージから取り込みます。
- `cookies`（クッキーズ）は Next.js が提供する「**現在のリクエストの Cookie を読み書きする**」関数です。`next/headers` から取り込みます。
- `createClient` 関数が `async` なのは、`await cookies()` を呼ぶためです。`cookies()` は Promise を返すので、`await` で中身（Cookieストア）を取り出してから使います。

> **用語: Cookie（クッキー）** … ブラウザとサーバーの間で受け渡しされる小さなデータ。ログイン状態（セッション）を保持するのに使われ、Supabaseはこれを使ってユーザーを識別します。

解説2: URL・匿名キーと一緒に「Cookie の入出力アダプタ」を渡す

```
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          // ... (Cookie の書き込み処理) ...
        },
      },
    }
  );
```

- 第1・第2引数は、接続先のURLと匿名キーです。`process.env.NEXT_PUBLIC_...` で環境変数から読み込んでいます。末尾の `!`（non-null assertion）は「この値は `undefined` ではないと断言する」記号で、設定漏れがあると実行時にエラーになります。
- 第3引数の `cookies` オブジェクトが、クライアント版にはない**サーバー版だけの追加設定**です。Supabase に「Cookieを読むときはこれ、書くときはこれを使って」と、読み書きの方法（アダプタ）を渡しています。
- `getAll()`：今のリクエストの全Cookieを返す関数。
- `setAll(...)`：Supabase がセッションを更新するとき、新しいCookieを書き込む関数。

> **用語: アダプタ（adapter）** ... 「形の違う2つの仕組みをつなぐ橋渡し役」のこと。ここでは「Supabaseが期待するCookie操作」と「Next.jsのCookie API」をつなぐ役割を果たします。

解説3: 書き込み時の例外を `try/catch` で握りつぶす理由

```
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Server Component からの呼び出し時は
            // cookie のセットができないが、問題ない
          }
        },
```

- `cookiesToSet.forEach(...)` は、Supabaseが渡してきた複数のCookieを1つずつ取り出して、`cookieStore.set(...)` で書き込んでいます。`({ name, value, options }) =>` は分割代入で、各Cookieの名前・値・オプションを取り出しています。
- これを `try/catch` で囲んでいるのは、**Server Component から Cookie を書き込もうとすると Next.js が例外を出す**ためです。
- ただし、これは実害がないので、`catch {}` の中を空にして**意図的に握りつぶしています**（エラーを無視しています）。コメントの通り「セットできないが問題ない」状況です。

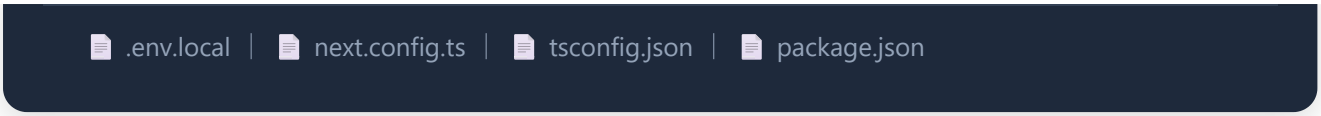
> **用語: 例外を握りつぶす** ... `try/catch` でエラーを受け止めつつ、`catch` の中で何もしないこと。「起きても問題ないと分かっているエラー」に限って使う手法で、むやみに多用するのは禁物です。

> **次章で詳しく解説:** Supabase のセットアップと実際の接続方法は、第5章「Supabase 入門」で詳しく扱います。ここでは環境変数の管理方法だけ理解しておいてください。

9. プロジェクト構成のベストプラクティス

9.1 ディレクトリ構成例





> **注意（第6章で作る実際の構成）**：上の図は、Next.js でよく使われるフォルダの分け方の例です。第6章から実際に作る書籍管理アプリでは、`src/` フォルダは使わずに `app/`・`components/`・`lib/`・`types/` をプロジェクト直下に置き、書籍一覧はトップページの `app/page.tsx` にします（`app/books/` の下は `new/` と `[id]/`・`[id]/edit/` だけ）。正確なファイル一覧は第6章の「4.1 ディレクトリ構成」を見てください。

9.2 各ディレクトリの役割

ディレクトリ	役割	含まれるもの
`app/`	**ルーティングとページ**	page.tsx, layout.tsx, loading.tsx, error.tsx, Server Actions
`components/`	**再利用可能な UI コンポーネント**	ボタン、カード、フォーム、ヘッダー等
`lib/`	**ビジネスロジックとユーティリティ**	DB接続、ヘルパー関数、外部API呼び出し
`types/`	**TypeScript 型定義**	アプリ全体で使う型（Book, User 等）
`public/`	**静的ファイル**	画像、フォント、favicon 等

9.3 書籍管理アプリのファイル構成

以下は、書籍管理アプリの最終的なファイル構成の全体像です。

> **▼ このコードがやること（先に日本語で）**：アプリ全体で使い回す「書籍データの形」を TypeScript の型として1か所にまとめて定義します。型を決めておくと、`book.title` のように書いたときにエディタが補完してくれたり、入力ミスを事前に警告してくれます。`string | null`（文字列か空のどちらか）や `?`（省略可能）といった、型を柔軟に表す書き方も学べます。詳細はコード内コメントを参照してください。

```ts
// 型定義の置き場
// types/book.ts
// アプリ全体で使う書籍の型定義（第6章で実際に作る型と同じ形）

// 読書ステータス。この3つの文字列のどれか（ユニオン型）
// want_to_read＝読みたい、reading＝読書中、completed＝読了
export type BookStatus = "reading" | "completed" | "want_to_read";

// 名前付き export。`import { Book } from "@/types/book"` で使う
export type Book = {
  // データベース上の主キー（UUID 文字列）
  id: string;
  // 書籍タイトル（必須）
  title: string;
  // 著者名（必須）
  author: string;
  // 出版社（未入力なら null）。`string | null` は型のユニオン
  publisher: string | null;
  // 出版日（"2024-01-15" のような文字列。未入力なら null）
  // スネークケースは Supabase の列名と合わせるため
  published_date: string | null;
  // 評価（1〜5。未評価なら null）
  rating: number | null;
  // 読書ステータス
  status: BookStatus;
  // メモ・感想（未入力なら null）
  memo: string | null;
  // 作成日時（ISO 8601 文字列）
  created_at: string;
  // 更新日時
  updated_at: string;
};

// 新規登録のときに送るデータの型（id や日時はデータベースが付けるので不要）
export type BookInsert = {
  title: string;
  author: string;
  // `?` で「省略可能」のオプショナルプロパティ
  publisher?: string | null;
  published_date?: string | null;
  rating?: number | null;
  status?: BookStatus;
  memo?: string | null;
};
```

▼ **このコードがやること（先に日本語で）**：アプリのあちこちで使う「便利な小さな関数（ユーティリティ）」を1か所にまとめます。日付文字列を「2024年1月15日」の形に整える `formatDate` と、複数のクラス名を空白区切りでつなげる `cn` の2つを定義します。`...classes`（レストパラメータ）で引数を何個でも受け取る書き方や、`filter(Boolean)` で空の値を除く書き方が学べます。詳細はコード内コメントを参照してください。

```
// 汎用ヘルパー置き場
// lib/utils.ts
// 汎用ユーティリティ関数

/**
 * 日付文字列を日本語のフォーマットに変換する
 * （`@example` は JSDoc の例示タグ。IDE で吹き出しに表示される）
 * @example formatDate("2024-01-15") → "2024年1月15日"
 */
// 引数も戻り値も string と明示
export function formatDate(dateString: string): string {
  // 文字列を Date オブジェクトに変換
  const date = new Date(dateString);
  // ロケール指定で日本語フォーマット
  return date.toLocaleDateString("ja-JP", {
    // 年は数字（2024）
    year: "numeric",
    // 月は「1月」のような長い表記
    month: "long",
    // 日は数字
    day: "numeric",
  });
}

/**
 * クラス名を結合する（falsy な値は除外）
 * @example cn("base", isActive && "active", "extra") → "base active extra"
 */
// `...classes` はレストパラメータ。可変長の引数を配列として受け取る
// 型は「string か false か undefined か null の配列」
export function cn(...classes: (string | false | undefined | null)[]): string {
  // `filter(Boolean)` で false/undefined/null/"" を除外
  // `join(" ")` で半角スペース区切りに連結
  return classes.filter(Boolean).join(" ");
}
```

▼ **このコードがやること（先に日本語で）**：色や種類を切り替えられる、使い回し可能な汎用ボタン部品を作ります。`variant`（primary/secondary/danger）や `disabled` などを props で受け取り、種類に応じて CSS クラスを組み立てます。`variant = "primary"` のように引数にデフォルト値を持たせる書き方や、props の形を型（`ButtonProps`）で定義する書き方が学べます。詳細はコード内コメントを参照してください。

```tsx
// 再利用可能なボタンコンポーネント
// components/ui/Button.tsx
// 汎用ボタンコンポーネント

// props の型定義
type ButtonProps = {
  // ボタン内の表示内容（テキストやアイコン）
  children: React.ReactNode;
  // ボタンの種類。`?` で省略可能、`|` でユニオン型
  variant?: "primary" | "secondary" | "danger";
  // 無効化フラグ
  disabled?: boolean;
  // HTML の button 要素の type
  type?: "button" | "submit";
  // クリック時のハンドラ（任意）
  onClick?: () => void;
};

// 分割代入で props を取り出す
export function Button({
  children,
  // デフォルト値の指定。指定がなければ "primary"
  variant = "primary",
  disabled = false,
  // デフォルトは "button"（submit を防ぐ）
  type = "button",
  onClick,
}: ButtonProps) {
  // 全ボタン共通のクラス
  const baseClass = "btn";
  // 種類別のクラス（例: "btn-primary"）
  const variantClass = `btn-${variant}`;

  return (
    <button
      // HTML 属性に渡す
      type={type}
      // テンプレートリテラルで2つのクラスを連結
      className={`${baseClass} ${variantClass}`}
      disabled={disabled}
      onClick={onClick}
    >
      {/* ボタンの中身を表示 */}
      {children}
    </button>
  );
}
```

パスエイリアスの設定

`@/` というパスエイリアスを使うと、ディレクトリの深さに関係なく、プロジェクトルートからの絶対パスで import できます。Next.js のプロジェクト作成時に自動で設定されます。

```
// ❌ 相対パスでの import（ディレクトリが深いと地獄）
// `../` の数を間違えると即エラー
import { BookCard } from "../../../components/books/BookCard";
// ファイルを移動するたびに書き直しが必要
import { formatDate } from "../../../lib/utils";

// ✅ パスエイリアスでの import（常にわかりやすい）
// `@/` がプロジェクトルートを指す
import { BookCard } from "@/components/books/BookCard";
// どんなに深い階層からでも同じ書き方でOK
import { formatDate } from "@/lib/utils";
```

この設定は `tsconfig.json`（TypeScript のコンパイル設定ファイル）に記述されています:

```
{
  // TypeScript コンパイラのオプション
  "compilerOptions": {
    // パスエイリアスのマッピング
    "paths": {
      // `@/` を「プロジェクトルートからの相対パス」に展開
      "@/*": ["./*"]
    }
  }
}
```

発展: アプリでは使っていない重要なNext.js機能

ここまでで、このチュートリアルの書籍管理アプリを作るのに必要な機能はひととおり学びました。ですが、Next.js には**実際の本番アプリでよく使われるのに、このアプリでは出番がなかった**重要な機能がまだあります。この「発展」セクションでは、それらを**独立した最小サンプル**で1つずつ紹介します。「いつ使うか」「なぜ便利か」を中心に、初心者でも読めるように解説していきます。

> **読み方のヒント:** ここで紹介する機能は、いきなり全部覚える必要はありません。「こういう機能がある」という存在だけ頭の片隅に置いておき、必要になったときに戻ってくれば十分です。すべて App Router（ `app/` フォルダ）前提のコードです。

generateMetadata と generateStaticParams（ページごとのタイトル設定とビルド時のページ生成）

> ▼ **このコードがやること（先に日本語で）**：書籍ごとに違う詳細ページ（ `/books/1` , `/books/2` …）で、**ブラウザのタブやSNS共有に出るタイトルを「その書籍のタイトル」に変える**仕組みと、**どの書籍ページを事前に用意しておくかを Next.js に教える**仕組みの2つを作ります。前者は検索結果やSNSで「この記事は何の話か」を正しく見せるため（SEO対策）に重要で、後者はアクセス前にページを作っておくことで表示を最速にするためのものです。詳細はコード内コメントを参照してください。

```tsx
// 動的ルート。/books/1, /books/2, ... に対応
// app/books/[id]/page.tsx
// [id] に入った値ごとに別ページになる
// URL: /books/1, /books/2, ...

// タイトルなどの型を読み込む（実行時には消える型だけのimport）
import type { Metadata } from "next";

// ─────────────────────────────────────────────
// (A) 書籍データを取得する仮の関数（本来はDBやAPIから取る）
// ─────────────────────────────────────────────
// id を受け取って書籍1冊分のデータを返す
async function getBook(id: string) {
  // 実際にはここで fetch やデータベース問い合わせを書く
  return { id, title: `サンプル書籍 ${id}`, author: "山田太郎" };
}

// ─────────────────────────────────────────────
// (B) generateMetadata: ページごとに <title> などを動的に作る
// ─────────────────────────────────────────────
// この名前で export すると、Next.js がページ表示前に呼んで <head> を組み立てる。
// 予約名の関数。async にできる
export async function generateMetadata({
  // page.tsx と同じく動的セグメントの値が入る
  params,
}: {
  // Next.js 15+ では params は Promise
  params: Promise<{ id: string }>;
// 戻り値は Metadata 型（title など）
}): Promise<Metadata> {
  // await で id を取り出す（文字列）
  const { id } = await params;
  // その id の書籍データを取得
  const book = await getBook(id);
  return {
    // ブラウザタブ・検索結果に出るタイトル
    title: `${book.title} | 書籍管理アプリ`,
    // SNS共有や検索結果の説明文
    description: `${book.author} 著「${book.title}」の詳細ページ`,
  };
}

// ─────────────────────────────────────────────
// (C) generateStaticParams: ビルド時に作っておくページの一覧
// ─────────────────────────────────────────────
// この名前で export すると、Next.js は「ここで返した id のページ」を
// ビルド時にあらかじめHTML化しておく（＝アクセスが最速になる）。
// 予約名の関数
export async function generateStaticParams() {
```

```
  // 事前生成したい書籍IDの一覧（本来はDBから取得）
  const ids = ["1", "2", "3"];
  // [{ id: "1" }, { id: "2" }, { id: "3" }] を返す
  return ids.map((id) => ({ id }));
}

// ──────────────────────────────────────────────
// (D) ページ本体
// ──────────────────────────────────────────────
export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  // ページ本体でも params から id を取り出す
  const { id } = await params;
  // 書籍データを取得
  const book = await getBook(id);
  return (
    <div>
      {/* 画面に出る見出し */}
      <h1>{book.title}</h1>
      <p>著者: {book.author}</p>
    </div>
  );
}
```

画面にはこう表示される: `/books/1` を開くと本文に「サンプル書籍 1」と表示され、**さらにブラウザのタブには「サンプル書籍 1 | 書籍管理アプリ」**と出ます。`/books/2` なら本文もタブも「2」に変わります。`generateStaticParams` で `"1"〜"3"` を返しているので、この3ページはビルド時に作られ、表示が最速になります。

▼ コードを1つずつ分解して解説

このコードには「`generateMetadata` でタイトルを動的に作る」「`generateStaticParams` で事前生成リストを返す」という2つの新しい仕組みが入っています。順番に見ていきましょう。

解説1: `generateMetadata` でページごとに違うタイトルを付ける

```
export async function generateMetadata({
  params,
}: {
  params: Promise<{ id: string }>;
}): Promise<Metadata> {
  const { id } = await params;
  const book = await getBook(id);
  return {
    title: `${book.title} | 書籍管理アプリ`,
    description: `${book.author} 著「${book.title}」の詳細ページ`,
  };
}
```

- これまで（`layout.tsx`）では `export const metadata = { ... }` という**固定値**でタイトルを付けていました。でも書籍詳細ページは「URLごとに中身が違う」ので、タイトルも本ごとに変えたいですよね。そこで使うのが `generateMetadata`（**ジェネレート・メタデータ**）という決まった名前の関数です。
- この関数は `page.tsx` と同じく `params`（URLの `[id]` の値）を受け取れます。`await params` で `id` を取り出し、その本のデータを取得して、**そのデータを使ったタイトル**を `return` で返します。
- 返したタイトルや説明文を、Next.js が自動で `<head>` の中に入れてくれます。検索エンジンやSNS（X・Facebookなど）はこの `<head>` を読んで「このページは何か」を判断するので、**SEO（検索エンジン対策）やSNS共有の見栄え**に直結します。

> **いつ使う？ なぜ便利？** ブログ記事・商品ページ・プロフィールページなど「URLごとに内容が変わるページ」では、`generateMetadata` でタイトルや説明文を中身に合わせて変えるのが基本です。これをしないと全ページが同じタイトルになり、検索結果で見分けがつかず、クリックされにくくなります。

> **用語: SEO（エスイーオー）** … Search Engine Optimization の略。Google などの検索結果で自分のページが上位に・分かりやすく表示されるよう整えること。タイトルと説明文はその一番の基本です。

解説2: `generateStaticParams` で「事前に作っておくページ」を伝える

```
export async function generateStaticParams() {
  const ids = ["1", "2", "3"];
  return ids.map((id) => ({ id }));
}
```

- 動的ルート（`[id]`）は、本来「アクセスが来たそのとき」にページを組み立てます。でも `generateStaticParams`（ジェネレート・スタティック・パラムス）という名前の関数を置くと、**ビルド時（公開前の準備段階）にまとめてHTMLを作っておきます。**

- 返す形は `[{ id: "1" }, { id: "2" }, { id: "3" }]` のような「`params` のオブジェクトの配列」です。`ids.map((id) => ({ id }))` はこの形を作っているだけです（`{ id }` は `{ id: id }` の省略形）。
- こうしておくと、`/books/1` 〜 `/books/3` は完成済みHTMLとして返るので、**表示が最速**になりサーバー負荷も減ります（第1章で学んだ SSG の仕組みです）。

> **いつ使う？ なぜ便利？**「ページ数があらかじめ分かっていて、頻繁には変わらない」もの（公開済みのブログ記事一覧、商品カタログなど）に向いています。アクセスを待たずに作っておくぶん、ユーザーは待たされません。逆に「数が膨大」「毎回内容が変わる」ページには向きません。

ISR（一定時間ごとにページを自動で作り直す）

> **▼ このコードがやること（先に日本語で）**:「ページを事前に作っておく（最速・SSG）」の良さを保ちつつ、**一定時間ごとに自動で最新版に作り直す**設定です。たった1行 `export const revalidate = 60;` を書くだけで「このページは60秒たったら次のアクセスのタイミングで作り直してね」と Next.js に伝えられます。常に最新でなくてもいいけれど、たまには更新したいページにぴったりです。詳細はコード内コメントを参照してください。

```tsx
// ニュース一覧ページの例
// app/news/page.tsx
// URL: /news

// ──────────────────────────────────────────────
// この1行が ISR の本体。
// 「このページは最大60秒は使い回し、60秒を過ぎたら次のアクセス時に裏で作り直す」
// という意味。数字（秒数）は自由に変えられる。
// ──────────────────────────────────────────────
// 60秒ごとに作り直す（予約名の export）
export const revalidate = 60;

// ニュースを取得する仮の関数
async function getNews() {
  // 実際にはここで外部APIやDBから最新ニュースを取る
  // 「ページを作った時刻」を確認用に作る
  const now = new Date().toLocaleTimeString("ja-JP");
  return [{ title: "新刊が入荷しました", builtAt: now }];
}

// ページ本体（Server Component）
export default async function NewsPage() {
  // データ取得
  const news = await getNews();

  return (
    <div>
      <h1>お知らせ</h1>
      <ul>
        {/* 取得した記事を一覧表示 */}
        {news.map((item, i) => (
          <li key={i}>{item.title}（生成時刻: {item.builtAt}）</li>
        ))}
      </ul>
    </div>
  );
}
```

画面にはこう表示される:「お知らせ」見出しの下にニュース項目が並びます。「生成時刻」の部分に注目すると、ページを何度リロードしても**60秒間は同じ時刻**のまま（=作り置きを使い回している）で、60秒を過ぎてからアクセスすると、裏で作り直しが始まり、その次のアクセスから**新しい時刻に更新**されます。これが「一定時間ごとに作り直す」動きです。

▼ **実行結果**（`npm run build` → `npm run start` で起動し、`curl` で時間をあけて4回開いたときの「生成時刻」。ビルドは 20:50:31 に行い、最初のアクセスはその約2分後）

```
  1秒後: 生成時刻: 20:50:31）
 31秒後: 生成時刻: 20:52:25）
 66秒後: 生成時刻: 20:52:25）
 73秒後: 生成時刻: 20:53:31）
```

- 1回目は、ビルド時に作った 20:50:31 の版がそのまま返りました。すでに60秒以上たっていたので、このアクセスをきっかけに裏で作り直しが始まります。
- 2回目（31秒後）には、作り直された 20:52:25 の版が返っています。
- 3回目（66秒後）は、20:52:25 から60秒を過ぎていたので、古い版をすぐ返しつつ裏で作り直し、4回目（73秒後）で新しい 20:53:31 の版になりました。6.1 節の図4-9 は、この記録を図にしたものです。

`npm run build` の結果にも、`/news` が60秒（1m）ごとに作り直されることが表示されます（関係する行だけ抜粋）。

```
Route (app)                Revalidate  Expire
├ ○ /news                         1m      1y
```

▼ コードを1つずつ分解して解説

ISR は新しい関数を覚える必要はなく、「`revalidate` という決まった名前の数字を export するだけ」というのがポイントです。

解説1: `export const revalidate = 60` の1行がすべて

```ts
export const revalidate = 60;
```

- **ISR**（アイエスアール）は「一度作ったページを使い回しつつ、決めた秒数ごとに裏でこっそり作り直す」仕組みです。これを有効にするのが、この `revalidate`（リバリデート＝再検証）という**決まった名前**の値を export する1行です。
- 数字は「何秒間そのページを使い回すか」を表します。`60` なら「60秒は作り置きを返す。60秒を過ぎた後の最初のアクセスをきっかけに、裏で新しいページを作って差し替える」という動きになります。
- 重要なのは、**作り直しはユーザーを待たせない**点です。秒数が過ぎても、そのアクセスにはまず古いページをすぐ返し、作り直しはその裏で進みます。次のアクセスから新しいページに切り替わります。

> **いつ使う？ なぜ便利？**「最新であってほしいけれど、1秒単位の鮮度までは要らない」ページ（ニュース、ランキング、在庫数の目安など）に最適です。アクセスのたびに毎回作り直す SSR より速く・安く、完全に作り置きの SSG より新しい、という「いいとこ取り」ができます。

> **用語: ISR（Incremental Static Regeneration）** …「少しずつ（インクリメンタルに）静的ページを再生成する」方式。第1章の表で出てきた SSG と SSR の中間にあたる方法だと考えると分かりやすいです。

> **参考（Cache Components を「Yes」にした場合の書き方）**：6.1 節の注意のとおり、`next.config.ts` に `cacheComponents: true` があるプロジェクトでは `export const revalidate = 60` は使えず、ビルドが次のエラーで止まります（実際の出力）。

```
Error: Route segment config "revalidate" is not compatible with
`nextConfig.cacheComponents`. Please remove it.
```

その場合は、ページの関数の先頭に `"use cache"` を書き、`cacheLife(...)` で「どれくらいの間隔で作り直すか」を指定します。`"minutes"` は「1分たったら、次のアクセスのときに裏で作り直す」という設定で、`revalidate = 60` とほぼ同じ意味になります。

```tsx
// app/news/page.tsx（Cache Components 版。cacheComponents: true のときだけ使える）
import { cacheLife } from "next/cache";

async function getNews() {
  // 「ページを作った時刻」を確認用に作る
  const now = new Date().toLocaleTimeString("ja-JP");
  return [{ title: "新刊が入荷しました", builtAt: now }];
}

export default async function NewsPage() {
  // このページの結果をキャッシュ（作り置き）する
  "use cache";
  // "minutes" は「1分たったら次のアクセスで裏で作り直す」という設定
  cacheLife("minutes");

  const news = await getNews();

  return (
    <div>
      <h1>お知らせ</h1>
      <ul>
        {news.map((item, i) => (
          <li key={i}>
            {item.title}（生成時刻: {item.builtAt}）
          </li>
        ))}
      </ul>
    </div>
  );
}
```

`cacheComponents: true` でビルドしたときの結果（関係する行だけ抜粋）です。`revalidate = 60` のときと同じく、1分（1m）ごとに作り直されることが分かります。

```
Route (app)              Revalidate  Expire
├ ○ /news                        1m      1h
```

Route Handlers（自分のアプリの中にAPIを作る）

> **▼ このコードがやること（先に日本語で）**：Next.js のアプリの中に、**他のプログラムがデータをやり取りするための「窓口（API）」**を作ります。`app/api/.../route.ts` という決まった場所にファイルを置くと、「データを取りに来たとき（GET）」と「データを送ってきたとき（POST）」の処理を書けます。スマホアプリや外部サービスと連携したり、ブラウザ側のJavaScriptからデータだけを取りに行きたいときに使います。詳細はコード内コメントを参照してください。

```ts
// 予約名 route.ts。URLは /api/books になる
// app/api/books/route.ts
// page.tsx ではなく route.ts を置くのがポイント
// URL: /api/books

// JSONなどを返すための Next.js 標準ヘルパー
import { NextResponse } from "next/server";

// 仮の書籍データ（本来はDBに保存する）
const books = [
  { id: 1, title: "吾輩は猫である" },
  { id: 2, title: "坊っちゃん" },
];

// ──────────────────────────────────────────────
// (1) GET: データを「取りに来た」ときの処理
// ──────────────────────────────────────────────
// 関数名を GET にすると、HTTPのGETリクエストに対応する。
export async function GET() {
  // NextResponse.json(...) でデータをJSON形式にして返す
  // /api/books に GET すると books 一覧が返る
  return NextResponse.json(books);
}

// ──────────────────────────────────────────────
// (2) POST: データを「送ってきた」ときの処理
// ──────────────────────────────────────────────
// 関数名を POST にすると、HTTPのPOSTリクエストに対応する。
// 送られてきた内容は引数 request に入る
export async function POST(request: Request) {
  // 送信された本文（JSON）を取り出す
  const body = await request.json();
  // 新しい書籍を組み立てる
  const newBook = { id: books.length + 1, title: body.title };
  // 一覧に追加（本来はDBに保存）
  books.push(newBook);
  // 201 は「新しく作成しました」を表すHTTPステータスコード
  // 作成した書籍を返す
  return NextResponse.json(newBook, { status: 201 });
}
```

画面にはこう表示される: これは画面（ページ）ではなく**データの窓口**です。ブラウザで `http://localhost:3000/api/books` を開くと、`[{"id":1,"title":"吾輩は猫である"},{"id":2,"title":"坊っちゃん"}]` のような**JSONデータがそのまま表示**されます。POST のほうは、ブラウザのJavaScriptや外部ツールから「タイトルを送る」と新しい書籍が追加され、追加後のデータが返ってきます。

▼ コードを1つずつ分解して解説

このファイルには「`route.ts` という置き場所のルール」「`GET` / `POST` という関数名のルール」「`NextResponse.json` で返すルール」の3点が詰まっています。

解説1: `route.ts` を置くと、その場所が「APIの窓口」になる

```
// app/api/books/route.ts
// URL: /api/books
```

- これまで作ってきたページは `page.tsx` という名前でした。それに対し、**`route.ts`（ルート・ティーエス）** という名前のファイルを置くと、そこは「画面を出す場所」ではなく「**データをやり取りする窓口（API）**」になります。
- 場所はフォルダ構成のままURLになります。`app/api/books/route.ts` なら URL は `/api/books` です（`api` というフォルダ名は必須ではありませんが、APIだと分かるよう慣習的にこう置きます）。
- このように Next.js だけで API も作れるので、**別途サーバー（Express など）を立てなくてよい**のが大きな利点です。

> **用語: API（エーピーアイ）** … プログラム同士がデータをやり取りするための「窓口・受付」のこと。人が見る画面（HTML）ではなく、機械が読むためのデータ（多くはJSON）を返します。

> **用語: JSON（ジェイソン）** … `{"title":"坊っちゃん"}` のような「キーと値」でデータを表す、プログラム間でよく使われる共通フォーマット。

解説2: 関数名 `GET` / `POST` がそのまま「リクエストの種類」に対応する

```
export async function GET() {
  return NextResponse.json(books);
}

export async function POST(request: Request) {
  const body = await request.json();
  const newBook = { id: books.length + 1, title: body.title };
  books.push(newBook);
  return NextResponse.json(newBook, { status: 201 });
}
```

- Webのデータのやり取りには「種類」があります。代表的なのが **GET（取りに行く）** と **POST（送り込む）** です。`route.ts` では、**関数の名前をそのまま `GET` や `POST` にする**ことで、それぞれの種類に対応する処理を書けます。
- `GET` は引数なしでもよく、`NextResponse.json(books)` のように**返したいデータをJSONにして返す**だけです。

- `POST` では、送られてきた内容が引数 `request` に入っています。`await request.json()` で**送信された本文を取り出し**、新しいデータを作って返しています。`{ status: 201 }` は「新しく作成しました」を意味する番号（HTTPステータスコード）です。

> **いつ使う？ なぜ便利？** スマホアプリや他のWebサービスに自分のデータを提供したいとき、ブラウザ側のJavaScriptから一部のデータだけを取りに行きたいとき、Webhook（外部サービスからの通知）を受け取りたいときなどに使います。なお、このアプリのようにフォーム送信をサーバーで処理するだけなら、第7章の **Server Actions** のほうが手軽な場合も多いです。用途で使い分けます。

Proxy（旧 Middleware：アクセス前に割り込んで認証チェックする）

> **▼ このコードがやること（先に日本語で）**：ページが表示される**前に**「割り込み」を入れて、「この人はログイン済みか？」をチェックします。もしログインしていなければ、目的のページを見せずに**ログイン画面へ自動的に送り返し（リダイレクト）**ます。会員専用ページや管理画面を「ログインしていない人に見せない」ための、見張り番のような仕組みです。詳細はコード内コメントを参照してください。

```ts
// 置き場所が重要。プロジェクトの一番上（app/ と同じ階層）に置く
// proxy.ts（Next.js 15 までは middleware.ts という名前だった）
// ※ app/ の中ではなく、ルート直下に1つだけ置く

// リダイレクトや「通過OK」を返すためのヘルパー
import { NextResponse } from "next/server";
// 届いたリクエストの型（型だけのimport）
import type { NextRequest } from "next/server";

// ──────────────────────────────────────────────
// proxy: ページに届く前に必ず通る「関所」
// 予約名 proxy で export する必要がある（15 までは middleware）。
// ──────────────────────────────────────────────
// request にアクセス内容（URLやCookieなど）が入る
export function proxy(request: NextRequest) {
  // Cookie から「ログインの証（トークン）」を読む
  // ログイン時に保存しておいた印を確認
  const token = request.cookies.get("token");

  // ログインの証が無い=未ログインなら、ログイン画面へ送り返す
  if (!token) {
    // リダイレクト先のURLを組み立てる
    const loginUrl = new URL("/login", request.url);
    // ログイン画面へ強制移動させる
    return NextResponse.redirect(loginUrl);
  }

  // ログイン済みなら、そのまま目的のページへ通す
  // 「通過OK」を返す（何も止めない）
  return NextResponse.next();
}

// ──────────────────────────────────────────────
// config.matcher: この見張りを「どのURLに効かせるか」
// ──────────────────────────────────────────────
export const config = {
  // /mypage と、その下のすべてのページ（/mypage/settings など）にだけ適用する
  matcher: ["/mypage/:path*"],
};
```

画面にはこう表示される: ログインしていない状態で `/mypage` を開こうとすると、`/mypage` の中身は一切見えないまま、**一瞬で `/login`（ログイン画面）に飛ばされ**ます。逆にログイン済み（Cookie にトークンがある）なら、何ごともなく `/mypage` がそのまま表示されます。

▼ 実行結果（`npm run build` → `npm run start` で起動し、`curl -i` で `/mypage` を開いたときの応答の見出し。2回目は `-b "token=abc"` でログインの証の Cookie を付けて送ったもの）

```
$ curl -i http://localhost:3000/mypage
HTTP/1.1 307 Temporary Redirect
location: /login

$ curl -i -b "token=abc" http://localhost:3000/mypage
HTTP/1.1 200 OK
```

Cookie が無いときは `307`（転送）で `/login` へ送り返され、Cookie があるときは `200 OK` で `/mypage` の中身（`<h1>マイページ</h1>`）が返りました。`npm run build` の結果の最後には、`ƒ Proxy (Middleware)` という行が表示され、proxy.ts が組み込まれたことが分かります。

▼ コードを1つずつ分解して解説

Proxy には「`proxy.ts` の置き場所」「`proxy` という関数名」「`config.matcher` で対象を絞る」という3つの約束があります（5.3 節の図4-8 も参照）。

解説1: `proxy.ts` は「ページの手前にある関所」

```
// proxy.ts （プロジェクトのルート直下に置く）
export function proxy(request: NextRequest) {
  const token = request.cookies.get("token");
  if (!token) {
    const loginUrl = new URL("/login", request.url);
    return NextResponse.redirect(loginUrl);
  }
  return NextResponse.next();
}
```

- **Proxy（プロキシ：代理人）** は、ユーザーがページにたどり着く**直前に必ず通る「関所」**です。`proxy.ts` という名前のファイルを**プロジェクトの一番上（`app/` フォルダと同じ階層）に1つだけ**置きます。Next.js 15 までは同じ役割のファイルを `middleware.ts`（ミドルウェア）と呼んでいました。
- 中では `request`（届いたアクセス情報）を調べられます。ここでは `request.cookies.get("token")` で「ログインの証」を探しています。
- 証が**無ければ** `NextResponse.redirect(...)` で**ログイン画面へ送り返し、有れば** `NextResponse.next()` で**そのまま通します**。`next()` は「邪魔せず通過させる」という意味です。

> **用語: リダイレクト** … アクセスしてきた人を、別のURLへ自動的に送り直すこと。「この扉は今開けられないので、あちらの受付へどうぞ」と案内するイメージです。

> **用語: Cookie（クッキー）／トークン** … Cookieはブラウザに小さなデータを保存しておく仕組み。ログインすると「あなたはログイン済みです」という印（トークン）をCookieに入れておき、次回以降そのトークンの有無でログイン状態を判断します。

解説2: `config.matcher` で「どのページに効かせるか」を絞る

```
export const config = {
  matcher: ["/mypage/:path*"],
};
```

- Proxy は放っておくと**すべてのアクセスに割り込んで**しまい、画像やトップページにまで関所がかかって無駄が出ます。そこで `config` の `matcher`（マッチャー＝対象を選ぶもの）で、**「効かせたいURLだけ」を指定**します。
- `"/mypage/:path*"` は「`/mypage` とその下のすべてのページ」という意味です。これで会員専用ページにだけログインチェックがかかり、トップページやログイン画面はチェックなしで開けます。

> **いつ使う？ なぜ便利？**「ログインしていない人に会員ページ・管理画面を見せたくない」というのが代表例です。各ページに毎回チェックを書くより、入口でまとめて見張るほうが書き漏れがなく安全です。ほかにも、言語ごとのページ振り分けや、地域によるアクセス制限などにも使われます。

---

next/image の詳細（画像を自動で最適化する）

> **▼ このコードがやること（先に日本語で）**：普通の `<img>` タグの代わりに Next.js の `<Image>` を使い、**画像を自動で軽く・速く・きれいに**表示します。`fill`（親要素いっぱいに広げる）、`preload`（最優先で読み込む。Next.js 15 までの `priority`）、`sizes`（画面幅ごとの表示サイズ指定）、`placeholder`（読み込み中にぼかしを出す）といった便利なオプションの使い方を1つにまとめた例です。画像が多いページの表示速度を大きく改善できます。詳細はコード内コメントを参照してください。

```tsx
// どのページでも使える例として page.tsx に書く
// app/page.tsx

// Next.js 専用の画像コンポーネントを読み込む
import Image from "next/image";

export default function HomePage() {
  return (
    <div>
      {/* ──── (1) サイズが分かっている画像: width / height を指定 ──── */}
      <Image
        // public フォルダ内の画像（/ から始める）
        src="/cover.jpg"
        // 画像の説明文（読み上げ・SEO・表示失敗時に必須）
        alt="本の表紙"
        // 元画像の幅（ガタつき防止のため必須）
        width={300}
        // 元画像の高さ
        height={450}
        // このページで「最初に見える重要画像」を最優先で読み込む
        preload
      />

      {/* ──── (2) サイズが可変な画像: fill で親要素いっぱいに広げる ──── */}
      <div style={{ position: "relative", width: "100%", height: "300px" }}>
        {/* fill を使うときは、親要素に position: relative とサイズが必要 */}
        <Image
          src="/banner.jpg"
          alt="キャンペーンバナー"
          // width/height の代わりに「親要素を埋める」
          fill
          // 画面幅ごとに必要な画像サイズを指定（後述）
          sizes="(max-width: 768px) 100vw, 50vw"
          // 読み込み中はぼかし画像を表示
          placeholder="blur"
          // ぼかしに使う極小画像データ
          blurDataURL="... "
          // はみ出した部分を切り取って枠に合わせる
          style={{ objectFit: "cover" }}
        />
      </div>
    </div>
  );
}
```

画面にはこう表示される: 見た目は普通の画像と同じですが、裏側では Next.js が「画面サイズに合った小さめの画像」「対応ブラウザには軽い形式（WebPなど）」を自動で配って、読み込みを速くしています。`placeholder="blur"` を

付けた画像は、**読み込みが終わるまでぼんやりした状態で表示**され、完了するとくっきり切り替わるので、待ち時間が気になりにくくなります。

▼ コードを1つずつ分解して解説

`<Image>` には便利なオプションがいくつもあります。よく使う `width/height`・`preload`・`fill`・`sizes`・`placeholder` を順に見ていきましょう。

解説1: `width` / `height` と `preload`

```
<Image
  src="/cover.jpg"
  alt="本の表紙"
  width={300}
  height={450}
  preload
/>
```

- `next/image` の `<Image>` は、普通の `<img>` の代わりに使うと **Next.js が画像を自動で最適化**してくれる部品です。`src` は画像の場所、`alt` は画像の説明文（読み上げソフトや、表示に失敗したときに使われるので必ず書きます）。
- `width` と `height` は**元画像の縦横の大きさ**です。これを指定しておくと、画像が読み込まれる前から「ここに300×450の場所を空けておく」ことができ、**読み込み後に他の文字や要素がガタッとずれるのを防げます。**
- `preload`（プリロード＝先に読み込む）を付けると、その画像を**最優先で読み込み**ます。ページを開いてすぐ目に入る大きな画像（記事のトップ画像など）に付けると、表示が体感で速くなります。逆に、下のほうにあって最初は見えない画像には付けません（付けすぎると逆効果）。

> **注意（Next.js 16 で `priority` → `preload`）**: Next.js 15 までは、この指定は `priority`（プライオリティ＝優先）という名前でした。Next.js 16 で `priority` は非推奨になり、`preload` に置き換わりました。古い記事の `priority` は `preload` と読み替えてください。公式ドキュメントでは、多くの場合は `loading="eager"`（すぐ読み込む）や `fetchPriority="high"`（優先度を上げる）でも十分だと案内されています。

> **用語: alt（オルト）テキスト** … 画像の内容を文字で説明したもの。目の不自由な人向けの読み上げや、画像が表示できなかったときの代わりの表示に使われ、SEOにも効きます。

解説2: `fill` で「親要素いっぱい」に広げる

```
<div style={{ position: "relative", width: "100%", height: "300px" }}>
  <Image
    src="/banner.jpg"
    alt="キャンペーンバナー"
    fill
    style={{ objectFit: "cover" }}
  />
</div>
```

- 画像の正確なサイズが分からない・画面幅に合わせて伸び縮みさせたい、という場合は `width/height` の代わりに `fill`（**フィル＝満たす**）を使います。これは「**親要素の大きさいっぱいに画像を広げる**」オプションです。
- `fill` を使うときの約束として、**親要素に `position: relative`（位置の基準にする指定）とサイズ（高さなど）が必要**です。これがないと画像が正しく広がりません。
- `objectFit: "cover"` は「枠からはみ出す部分は切り取って、枠をきれいに埋める」指定です。これで縦横比が違う画像でも崩れずに表示できます。

解説3: `sizes` で「画面幅ごとに最適な画像」を配る

```
sizes="(max-width: 768px) 100vw, 50vw"
```

- `sizes`（サイズ）は、「**画面の幅ごとに、この画像が実際どれくらいの大きさで表示されるか**」を Next.js に教えるオプションです。これを伝えると、Next.js は**スマホには小さい画像、PCには大きい画像**といったように、ちょうどよいサイズの画像を選んで配れます。
- `(max-width: 768px) 100vw, 50vw` は「画面幅が768px以下（スマホ）なら画面幅いっぱい（100vw）、それ以外（PC）なら画面の半分（50vw）の大きさで表示する」という意味です。`vw` は「画面幅に対する割合」を表す単位です。
- `fill` を使うときは、適切な画像を選ぶために `sizes` を一緒に指定するのが推奨です。スマホで巨大な画像をわざわざダウンロードせずに済むので、**通信量の節約と高速化**につながります。

解説4: `placeholder="blur"` で読み込み中にぼかしを出す

```
placeholder="blur"
blurDataURL="... "
```

- `placeholder="blur"`（プレースホルダー＝仮置き、blur＝ぼかし）を付けると、**画像の読み込みが終わるまでの間、ぼんやりした仮の画像**を表示します。完了すると本物にスッと切り替わります。
- 「真っ白な空白」が一瞬出るより、ぼかしでも何か見えているほうが、ユーザーは「ちゃんと読み込まれている」と感じて待ち時間が気になりにくくなります。

- `blurDataURL` は、そのぼかしに使う**ごく小さな画像データ**です（ここでは省略して `...` と書いています）。`public` フォルダ内の画像をインポートして使う場合は、Next.js がこのぼかし用データを自動生成してくれるので自分で用意しなくて済みます。

> **いつ使う？ なぜ便利？ （next/image 全体）** 画像はWebページの表示が遅くなる最大の原因の1つです。`<img>` をそのまま使うと「大きすぎる画像をそのまま配ってしまう」「読み込み中にレイアウトがずれる」といった問題が起きがちですが、`next/image` を使えばサイズ調整・形式変換・遅延読み込みなどを Next.js が自動でやってくれます。**画像を多く使うページほど効果が大きい**ので、Next.js では原則 `<Image>` を使うのがおすすめです。

---

10. よくあるエラーと対処法

エラー1: "useState" は Server Component で使用できない

`npm run dev` 中にそのページを開いたときの、ターミナルの実際の出力です（Next.js 16.4.1。ファイル名は実験用のもの）。

```
× ./app/ng-usestate/page.tsx:1:10
Error: You're importing a module that depends on `useState` into a React Server
Component module. This API is only available in Client Components. To fix, mark the
file (or its parent) with the `"use client"` directive.
    Learn more: https://nextjs.org/docs/app/api-reference/directives/use-client
> 1 | import { useState } from "react";
    |          ^^^^^^^^
  2 |
  3 | export default function Counter() {
  4 |   const [count, setCount] = useState(0);

Ecmascript file had an error
```

エラー文の要点は「`useState` を使うモジュールを React Server Component に import している。この API は Client Component でしか使えない。直すにはファイル（またはその親）に `"use client"` を付けて」です。このエラーは「組み立て（コンパイル）の失敗」なので、直すまでは**ほかのページも開けなくなる**ことがあります（実験でも、関係ないページが 500 エラーになりました）。

原因: Server Component（デフォルト）で `useState` や `useEffect` を使おうとしている。

対処法: ファイルの先頭に `"use client"` を追加する。

```
// "use client" がないので Server Component 扱い
// ❌ エラーになる
// React のフックを取り込み
import { useState } from "react";

export default function Counter() {
  // ← ここでエラー
  // Server Component では useState を呼べない
  const [count, setCount] = useState(0);
  // onClick も Server Component では使えない
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}

// ✅ 修正版
// ← これを追加
// ファイル先頭のディレクティブで Client Component 化
"use client";

import { useState } from "react";

export default function Counter() {
  // OK: Client では useState が使える
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

エラー2: "async/await" は Client Component で使用できない

ブラウザの開発者ツールの Console に、次のエラーが出ます（実際の出力。`<BooksPage>` の部分はコンポーネント名）。

```
<BooksPage> is an async Client Component. Only Server Components can be async at the moment. This error is often caused by accidentally adding `'use client'` to a module that was originally written for the server.
```

実験では、このエラーと一緒に `fetch` が何度も繰り返され、Console が `Failed to load resource` で埋まりました。「関数に `async` を付けた Client Component は、描画のたびに処理をやり直してしまう」ためです。

原因: `"use client"` を付けたコンポーネントで `async` 関数としてエクスポートしている。

対処法: データ取得を Server Component に移動するか、`useEffect` を使う。

```
// ❌ エラーになる
// Client Component なのに
"use client";

// 関数本体に async を付けている
export default async function BooksPage() {
  // ← async は Client Component で不可
  // App Router の Client では関数を async にできない
  const books = await fetch("/api/books");
  return <div>...</div>;
}

// ✅ 修正版 1: Server Component にする（"use client" を削除）
// "use client" がなければ Server Component
export default async function BooksPage() {
  // Server なら async でも OK
  const books = await fetch("https://api.example.com/books");
  return <div>...</div>;
}

// ✅ 修正版 2: Client Component のまま useEffect を使う
"use client";

import { useState, useEffect } from "react";

// 関数本体は同期のままにする
export default function BooksPage() {
  // 結果を state に保持
  const [books, setBooks] = useState([]);

  // マウント後にデータ取得
  useEffect(() => {
    fetch("/api/books")
      // Promise チェーン
      .then((res) => res.json())
      // 取れた配列を state に格納
      .then(setBooks);
  // 依存配列が空なので初回マウント時に1度だけ実行
  }, []);

  return <div>...</div>;
}
```

エラー3: "next/router" と "next/navigation" の混同

```
Error: NextRouter was not mounted.
```

原因: App Router で `next/router` を import している。App Router では `next/navigation` を使う。

```
// ❌ App Router では使えない
// 旧 Pages Router 用の useRouter
import { useRouter } from "next/router";

// ✅ App Router ではこちらを使う
// 新しい App Router 用。push/replace/back/refresh などを持つ
import { useRouter } from "next/navigation";
```

エラー4: metadata と "use client" の共存

`npm run build` の実際の出力です（関係する部分だけ抜粋）。

```
./app/ng-meta/page.tsx:3:14
Error: You are attempting to export "metadata" from a component marked with "use client", which is disallowed. "metadata" must be resolved on the server before the page component is rendered. Keep your page as a Server Component and move Client Component logic to a separate file. Read more: https://nextjs.org/docs/app/api-reference/functions/generate-metadata#why-generatemetadata-is-server-component-only
```

最後の文「ページは Server Component のままにして、Client Component の処理は別のファイルに移して」が、そのまま直し方です（3.3 の「つまずきポイント」と同じ考え方）。

原因: `"use client"` のファイルで `metadata` をエクスポートしている。メタデータは Server Component でのみエクスポート可能。

```tsx
// ❌ エラーになる
// Client Component なのに
"use client";

// ← Client Component では不可
// metadata は Server Component の機能なので Client では宣言できない
export const metadata = { title: "書籍一覧" };

// ✅ 修正版: metadata は Server Component（page.tsx や layout.tsx）に置く
// app/books/page.tsx（Server Component）
// この行でブラウザタブのタイトルが「書籍一覧」になる
export const metadata = { title: "書籍一覧" };

export default function BooksPage() {
  return <div>...</div>;
}
```

> **補足: `generateMetadata`**：動的にメタデータを決めたいとき（例: 書籍IDから書籍タイトルを取って `<title>` に入れる）は、`generateMetadata` という名前で async 関数を export します。引数として `params` を受け取れます。同様に「ビルド時に動的セグメントの値を事前列挙したい」場合は `generateStaticParams` を export します。

エラー5: 環境変数が undefined になる

原因: Client Component で `NEXT_PUBLIC_` プレフィックスのない環境変数を使っている。

```tsx
// ❌ Client Component では undefined になる
"use client";

// undefined
// NEXT_PUBLIC_ なしの環境変数はブラウザバンドルに含まれない
const apiKey = process.env.API_SECRET_KEY;

// ✅ NEXT_PUBLIC_ を付ける（ただし機密情報には使わない！）
// 値が取得できる
// 公開してよい値だけに NEXT_PUBLIC_ を付ける
const publicUrl = process.env.NEXT_PUBLIC_API_URL;
```

> **注意:** 機密情報に `NEXT_PUBLIC_` を付けてはいけません。秘密の値は Server Component・Server Action・Route Handler の中だけで使ってください。また、`NEXT_PUBLIC_` の無い秘密の変数を Client Component の中で**画面に表示**すると、ブラウザでは `undefined` でも、サーバーでの最初の描画で本物の値が HTML に入ってしまいます（8.2 の「つまずきポイント」で実験しました）。`NEXT_PUBLIC_` の値を変えたのに反映されないときは、開発サーバーの再起動か、もう一度ビルドが必要です（同じく 8.2）。

エラー6: "Hydration failed because the server rendered text didn't match the client"（Hydration エラー）

React 19 では、ブラウザの開発者ツールの Console に次のエラーが出ます。`Math.random()`（実行するたびに違う数を返す）を Client Component で表示して試したときの実際の出力です（先頭と、食い違った場所の部分を抜粋）。`+` がブラウザで描画した値、`-` がサーバーの HTML の値です。

```
Hydration failed because the server rendered text didn't match the client. As a result
this tree will be regenerated on the client. This can happen if a SSR-ed Client
Component used:

- A server/client branch `if (typeof window !== 'undefined')`.
- Variable input such as `Date.now()` or `Math.random()` which changes each time it's
called.
- Date formatting in a user's locale which doesn't match the server.
- External changing data without sending a snapshot of it along with the HTML.
- Invalid HTML tag nesting.
...
                        <NgHydrationPage>
                          <NowText>
                            <p>
+                             0.033854490369345513
-                             0.399743962157744
```

エラー文の箇条書きが、そのまま「よくある原因」の一覧になっています。「`typeof window` で分岐している」「`Date.now()` や `Math.random()` のように毎回変わる値」「利用者の地域設定で変わる日付の書式」などです。

原因: Client Component の「サーバーでの最初の描画（図4-4 の③）」と「ブラウザでの描画（⑥）」で、違う内容になっている。日時のフォーマットやランダム値でよく発生する。なお、Hydration（ハイドレーション）は Client Component にだけ起きるので、`"use client"` の無い Server Component で時刻を表示しても、このエラーにはなりません（その場合は 6.1 の「時刻が止まる」問題に注意）。

```tsx
// ❌ サーバーとクライアントで結果が異なる可能性がある（Client Component）
"use client";

export default function DateDisplay() {
  // サーバーとブラウザで実行時刻が違えば表示も違う
  // → Hydration エラーになる
  return <p>{new Date().toLocaleString()}</p>;
}

// ✅ 修正版 1: suppressHydrationWarning を使う（Client Component のまま）
"use client";

export default function DateDisplay() {
  // React の Hydration 不一致警告を抑制する属性
  // 中身が動的でもよい場合の応急処置
  return <p suppressHydrationWarning>{new Date().toLocaleString()}</p>;
}

// ✅ 修正版 2: Client Component にして useEffect で設定
// Client 化
"use client";

import { useState, useEffect } from "react";

export default function DateDisplay() {
  // 初期値は空文字（サーバーでも同じ値）
  const [dateStr, setDateStr] = useState("");

  // ハイドレーション後＝ブラウザ側でのみ走る
  useEffect(() => {
    // ここで日付を設定 → 再描画
    setDateStr(new Date().toLocaleString("ja-JP"));
  // 初回1回だけ
  }, []);

  // サーバー描画時は空、ブラウザで時刻に置き換わる
  return <p>{dateStr}</p>;
}
```

エラー7: 本文で扱ったそのほかのエラー（早見表）

この章の本文の「つまずきポイント」で、実際の出力と直し方を紹介したエラーです。エラー文の一部で検索して、該当する節に戻ってください。

エラー文（の一部）	何が起きているか	戻る節
`used params.id. params is a Promise and must be unwrapped with await`	`params` を `await` せずに読んだ	2.4
`ReferenceError: window is not defined`	Client Component の描画中に `window` を読んだ（サーバーでの描画で失敗）	3.2
`Event handlers cannot be passed to Client Component props` / `Functions cannot be passed directly to Client Components`	Server Component から Client Component に関数を渡した	3.5
`Only plain objects, and a few built-ins, can be passed to Client Components`	クラスから作ったものを props で渡した	3.5
（エラーは出ないが）`catch で捕まえたもの: Error: NEXT_REDIRECT`	`redirect()` を `try` の中で呼んだ	5.3
`Types of parameters 'formData' and 'state' are incompatible`	`useActionState` に FormData だけを受け取る関数を渡した	7.2
（エラーは出ないが）本番で一覧や時刻が更新されない	ページがビルド時に作り置きされた／`revalidatePath` を忘れた	6.1・7.2
`Next.js encountered uncached or runtime data during prerendering`	Cache Components を Yes にしたプロジェクトで、`loading.tsx` の無いページが `params` などを使った	6.1 の注意

> **補足（この見出しにあった古い警告について）**：以前の版では「`layout.tsx` で `cache: "no-store"` の `fetch` を使うと警告が出る」と書いていましたが、Next.js 16.4.1 で実際に試したところ、そのような警告は出ませんでした。`layout.tsx` で毎回データを取得すると、その下のすべてのページが毎回作り直しになる（6.1 の `ƒ`）点には注意してください。

エラーの調査方法（一般的なアドバイス）

1. **ターミナルのエラーメッセージを読む:** Next.js のエラーメッセージは非常に詳しく、多くの場合、解決策まで提示してくれます。
2. **ブラウザのコンソールを確認する:** 開発者ツール（F12）のConsoleタブにもエラーが表示されます。
3. **Server Component と Client Component の境界を確認する:** 多くのエラーは、この2つの境界での誤りが原因です。
4. **Next.js の公式ドキュメントを参照する:** https://nextjs.org/docs には詳細な説明とトラブルシューティングがあります。

まとめ

この章では Next.js の基礎を広く学びました。

トピック	学んだこと
Next.js とは	React のフルスタックフレームワーク。SSR/SSG/CSR に対応
App Router	ファイルベースルーティング。page.tsx, layout.tsx 等の特殊ファイル
Server/Client Components	デフォルトは Server。インタラクティブ性が必要なら "use client"
レイアウト	ルートレイアウトとネストされたレイアウトで共通UIを効率的に管理
ナビゲーション	Link コンポーネント（宣言的）と useRouter フック（命令的）
データフェッチ	Server Component なら async/await で直接。Client は useEffect
Server Actions	"use server" でサーバー処理を直接呼び出し。フォーム処理に最適
環境変数	NEXT_PUBLIC_ の有無でアクセス範囲が変わる
プロジェクト構成	app/, components/, lib/, types/ の4層構造
1回のアクセスの流れ	サーバーで HTML を作る（Client Component も1回描画）→ 表示 → ハイドレーションで動き出す（図4-4）
props の境界	Server から Client へ渡せるのは文字にできるものだけ。関数は渡さない（図4-5）
キャッシュ	本番ではページがビルド時に作り置きされることがある。`connection()`・`revalidate`・`revalidatePath` で更新を制御（図4-9・図4-10）
Next.js 16 の変更点	`params` は必ず `await`、`middleware.ts` → `proxy.ts`、`priority` → `preload`、`error.tsx` の `retry`

> **次の章では:** Supabase のセットアップを行い、実際のデータベースと接続します。書籍管理アプリのバックエンドを構築し、この章で学んだ Server Components や Server Actions を使ってデータの読み書きを実装していきます。

確認問題

以下の問いに答えて、理解度を確認してみましょう。

1. **Server Component と Client Component の違い** を3つ挙げてください。
2. 以下のコンポーネントは Server / Client のどちらにすべきですか？ - データベースから書籍一覧を取得して表示するコンポーネント - 検索バー（ユーザーの入力をリアルタイムに反映） - 書籍カードの静的な表示（クリックイベントなし）
3. `NEXT_PUBLIC_API_KEY` と `API_SECRET_KEY` はそれぞれどこで使用できますか？

4. `layout.tsx` と `page.tsx` の違いは何ですか？
5. Server Actions を使う利点を2つ挙げてください。

確認問題の解答と解説

まず自分の言葉で答えてから読んでください。

問1の解答（Server Component と Client Component の違い）

- Server Component はサーバーで実行され、Client Component はブラウザでも実行される（最初の1回はサーバーでも描画される。図4-4 の③）
- Server Component はデータベースや秘密のキーに直接アクセスできる。Client Component はできない（書いてもブラウザでは使えず、秘密は漏れる危険がある）
- Client Component は `useState`・`useEffect`・`onClick` などが使える。Server Component は使えない

解説: 3つとも「どこで動くか」から説明できます。サーバーで動くから DB に近く秘密を守れる、ブラウザで動くからクリックや入力に反応できる、という関係です。「Client Component はブラウザ**だけ**で動く」と答えた人は、3.1.1 の実験（ターミナルに「実行場所: サーバー」と出た）をもう一度見てください。

問2の解答（Server / Client の判断）

- データベースから書籍一覧を取得して表示 → **Server Component**（DB に直接アクセスでき、データ入りの HTML がすぐ届く）
- 検索バー（入力をリアルタイムに反映）→ **Client Component**（`useState` と `onChange` が必要）
- 書籍カードの静的な表示 → **Server Component**（動きが無いので、ブラウザに JavaScript を送る必要が無い）

解説: 判断の基準は「その部品に、利用者の操作に反応する動きがあるか」です。動きが無いものは Server Component のまま、動きがある部品だけを小さく切り出して `"use client"` を付けます（3.3 の「つまずきポイント」）。

問3の解答（環境変数のアクセス範囲）

- `NEXT_PUBLIC_API_KEY` → Server Component と Client Component の両方で使える。ただし**ビルドのときにブラウザ用の JavaScript に値が書き込まれ、誰でも見られる**
- `API_SECRET_KEY` → サーバー（Server Component・Server Action・Route Handler）だけで使う。Client Component ではブラウザで `undefined` になる

解説: 名前に `API_KEY` と付いていても、`NEXT_PUBLIC_` を付けた時点で「公開してよい値」として扱われます（図4-11）。秘密のキーに `NEXT_PUBLIC_` を付けてはいけません。また、`API_SECRET_KEY` を Client Component の中で画面に表示すると、サーバーでの最初の描画で本物の値が HTML に入ってしまう点にも注意しましょう（8.2 の「つまずきポイント」）。

問4の解答（layout.tsx と page.tsx の違い）

- `layout.tsx` は、そのフォルダ以下の複数のページで共有される外枠（ヘッダー、サイドバーなど）。`{children}` の位置にページが入る。ページを移動しても作り直されず、状態が保たれる
- `page.tsx` は、その URL 固有の中身。ページを移動すると入れ替わる

解説: 図4-6 の「箱の中に箱」を思い出してください。5.1 の実験では、レイアウトの中のカウンターは `<Link>` で移動しても 3 のまま残りました（`<a>` だと 0 に戻る）。

問5の解答（Server Actions の利点）

- API（`route.ts`）を別に作らなくても、フォームから直接サーバーの関数を呼べるので、コードが短くなる
- 関数の中身はサーバーで実行されるので、DB の接続情報などの秘密がブラウザに送られない
- `revalidatePath` と組み合わせると、保存と画面の更新が1回の通信で済む（図4-10）
- `<form action>` に渡せば、JavaScript が動かない環境でも送信できる

解説: 2つ挙げられれば十分です。ただし、Server Action は「誰でも POST で呼び出せる公開の入り口」でもあるので、本物のアプリでは関数の中で必ずログインの確認や入力チェックをします（Next.js の公式ドキュメントでも強く注意されています）。

練習問題（解答・解説つき）

本文の内容を、手を動かして確かめる問題です。まず自分で考えて（できれば紙やエディタに書いて）から、後ろの「解答と解説」を見てください。

問題1: ファイルをどこに置く？（2節）

次の画面を表示するには、`app/` フォルダの下にどんな名前のファイルを作ればよいでしょうか。フォルダも含めて答えてください。

1. `/about` のページ
2. `/books/42/edit`（42 は書籍ごとに変わる）の編集ページ
3. `/docs/a`・`/docs/a/b`・`/docs/a/b/c` のどれでも表示される1つのページ
4. `/books` 以下のページでデータを取得している間に出す「読み込み中」の表示

問題2: 古いサンプルを今の書き方に直す（2.4節）

インターネットで見つけた次のコードを Next.js 16 で使うと、どうなるでしょうか。また、正しく動くように直してください。

```tsx
// app/books/[id]/page.tsx（古い書き方）
export default function BookDetailPage({
  params,
}: {
  params: { id: string };
}) {
  return <h1>書籍 {params.id} の詳細</h1>;
}
```

問題3: ボタンを1つ足したらエラーになった（3節）

次の `app/books/page.tsx` に「いいね」ボタンを足したところ、ページを開くと 500 エラーになりました。データ取得はサーバーで行ったまま、ボタンが動くように直してください。

```tsx
// app/books/page.tsx
export default async function BooksPage() {
  const books = ["リーダブルコード", "達人プログラマー"];
  return (
    <div>
      <h1>書籍一覧（{books.length}冊）</h1>
      <button onClick={() => console.log("いいね")}>いいね</button>
    </div>
  );
}
```

問題4: ログはどこに出る？（3.1.1節）

次の Client Component を Server Component のページに置き、`npm run dev` でページを1回開いてから、ボタンを1回押しました。(a)〜(c) のログは、それぞれ「サーバーのターミナル」「ブラウザの Console」のどちらに出るでしょうか（両方の場合もあります）。

```tsx
"use client";

import { useEffect } from "react";

export function Hello() {
  // (a) 描画のたびに出すログ
  console.log("描画しました");
  useEffect(() => {
    // (b) 画面に出た後に出すログ
    console.log("画面に出ました");
  }, []);
  // (c) ボタンを押したときに出すログ
  return <button onClick={() => console.log("押されました")}>押す</button>;
}
```

問題5: Link か router.push か（5節）

次の場面では、`<Link>` と `router.push` のどちらを使うのがよいでしょうか。理由も答えてください。

1. 画面上部のメニューにある「書籍一覧」「書籍を追加」
2. 登録フォームで「保存」を押し、保存に成功したら、その本の詳細ページへ移動する
3. 一覧に並んだ書籍カードを押すと、その本の詳細ページへ移動する

問題6: 本番だけ一覧が増えない（6.1・7.2節）

次の Server Action で本を追加すると、`npm run dev` では一覧（`/`）を再読み込みすれば増えて見えるのに、`npm run build` → `npm run start` の本番モードでは、何度再読み込みしても増えません。原因と直し方を答えてください（`saveBook` は、データベースに保存する処理のまねをする自作の関数とします）。

```tsx
// app/actions.ts
"use server";

import { saveBook } from "@/lib/books";

export async function addBook(formData: FormData) {
  const title = String(formData.get("title") ?? "").trim();
  if (!title) {
    return;
  }
  await saveBook(title);
}
```

問題7: 環境変数の仕分け（8節）

`.env.local` に次の4つを書きました。

```
NEXT_PUBLIC_SUPABASE_URL="https://abcdefghijklm.supabase.co"
NEXT_PUBLIC_SUPABASE_ANON_KEY="sb_publishable_xxxxxxxxxxxxxxx"
SUPABASE_SERVICE_ROLE_KEY="sb_secret_xxxxxxxxxxxxxxx"
NEXT_PUBLIC_ADMIN_PASSWORD="p@ssw0rd"
```

1. ビルド後、ブラウザ用の JavaScript に値が書き込まれるのはどれですか。
2. この中で、設定のしかたが危ないものはどれですか。どう直しますか。
3. ある Client Component に `<p>{process.env.SUPABASE_SERVICE_ROLE_KEY}</p>` と書いてしまいました。ブラウザの画面には何が出て、何が問題ですか。

解答と解説

問題1の解答

1. `app/about/page.tsx`
2. `app/books/[id]/edit/page.tsx`
3. `app/docs/[...slug]/page.tsx`
4. `app/books/loading.tsx`

解説: URL の区切り（`/`）ごとにフォルダを作り、最後に `page.tsx` を置くのが基本です（図4-3）。変化する部分は `[id]` のように角括弧のフォルダにし、何階層でも受け取りたいときは `[...slug]` にします。`loading.tsx` は、置いたフォルダとその下のページの「読み込み中」表示になるので、`/books` 以下すべてに効かせたいなら `app/books/` に置きます（図4-2）。

問題2の解答

どうなるか: 型チェックも `next build` も通ってしまいますが、画面には id が空のまま「書籍 の詳細」と表示され、開発サーバーのターミナルに `Route "/books/[id]" used `params.id`. `params` is a Promise and must be unwrapped with `await` ...` というエラーが出ます（2.4 の「つまずきポイント」の実験結果）。

```tsx
// app/books/[id]/page.tsx（直したもの）
export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  return <h1>書籍 {id} の詳細</h1>;
}
```

解説: 直すところは3つです。①関数に `async` を付ける、②型を `Promise<{ id: string }>` にする、③ `await params` で封を開けてから `id` を取り出す。`searchParams` も同じように直します。

問題3の解答

原因: `"use client"` の無い `page.tsx` は Server Component です。Server Component の中で `onClick` に関数を渡しているので、ターミナルに `Event handlers cannot be passed to Client Component props.` というエラーが出ます（`<button onClick={function onClick} children=...>` の部分が指し示されます。実際に試して確認したエラー文です）。

```
// app/books/LikeButton.tsx（動きが必要なボタンだけを Client Component にする）
"use client";

export function LikeButton() {
  return <button onClick={() => console.log("いいね")}>いいね</button>;
}
```

```
// app/books/page.tsx（ページは Server Component のまま）
import { LikeButton } from "./LikeButton";

export default async function BooksPage() {
  const books = ["リーダブルコード", "達人プログラマー"];
  return (
    <div>
      <h1>書籍一覧（{books.length}冊）</h1>
      <LikeButton />
    </div>
  );
}
```

解説: `page.tsx` の先頭に `"use client"` を付けても動きますが、ページ全体が Client Component になり、`async` でのデータ取得や `metadata` が使えなくなります（3.3 の「つまずきポイント」）。動きが必要な部品だけを切り出すのが正解です。

問題4の解答

- (a) `描画しました` … **両方**。最初の HTML を作るためにサーバーで1回（ターミナル）、ハイドレーションのためにブラウザで（Console。開発中は Strict Mode で2回）実行されます。
- (b) `画面に出ました` … **ブラウザの Console だけ**。`useEffect` はブラウザで画面に表示された後にだけ実行されます。
- (c) `押されました` … **ブラウザの Console だけ**。クリックはブラウザで起きるからです。

解説: 3.1.1 の実験結果と同じです。「Client Component の関数本体はサーバーでも動く」「`useEffect` とイベントの中はブラウザだけ」の2点が分かっていれば、`window is not defined` のエラー（3.2）も防げます。

問題5の解答

1. `<Link>` … いつでも行ける場所への道案内なので。本物の `<a href>` になり、新しいタブで開いたり、検索エンジンがたどったりできる
2. `router.push` …「保存に成功したら」という**処理の結果のあとで**移動したいので（Server Action で保存しているなら、Server Action の中で `redirect()` を使う方法もある）
3. `<Link>` … 1 と同じく、利用者が選んで進む道案内なので

解説: 迷ったら `<Link>`、「何かが終わってから移動」のときだけ `router.push`（図4-7）。一覧のカードをボタンにして `router.push` で移動させると、右クリックで新しいタブを開けない不便なリンクになってしまいます。

問題6の解答

原因: 保存したあとに `revalidatePath` を呼んでいないので、Next.js は一覧（`/`）の作り置きが古くなったことを知りません。本番モードでは、`/` がビルド時に作り置き（`○ Static`）されているので、何度読み込み直しても作り置きが返ります。開発モードではページを毎回作り直すので、気づきにくいのです（7.2 の「つまずきポイント」の実験と同じ）。

```
// app/actions.ts（直したもの）
"use server";

import { revalidatePath } from "next/cache";
import { saveBook } from "@/lib/books";

export async function addBook(formData: FormData) {
  const title = String(formData.get("title") ?? "").trim();
  if (!title) {
    return;
  }
  await saveBook(title);
  // 一覧の画面が古くなったことを Next.js に伝える
  revalidatePath("/");
}
```

解説:「データを変えたら、そのデータを表示している画面の URL を `revalidatePath` に渡す」がセットです（図4-10）。詳細ページ `/books/[id]` にも表示しているなら、そのパスも作り直します。

問題7の解答

1. `NEXT_PUBLIC_` が付いた3つ（`NEXT_PUBLIC_SUPABASE_URL`・`NEXT_PUBLIC_SUPABASE_ANON_KEY`・`NEXT_PUBLIC_ADMIN_PASSWORD`）。
2. `NEXT_PUBLIC_ADMIN_PASSWORD` が危険です。管理者のパスワードがブラウザ用の JavaScript に書き込まれ、誰でも読めてしまいます。`NEXT_PUBLIC_` を外して `ADMIN_PASSWORD` にし、サーバー（Server Action など）の中だけで使います。すでに公開してしまった値は、パスワード自体を変更します。

3. ブラウザの画面には（ハイドレーションのあと）何も表示されません（ブラウザでは `undefined`）。しかし、サーバーでの最初の描画では本物の Secret key が読めるので、**サーバーから届く HTML に Secret key がそのまま入ってしまいます**。さらに Hydration エラーも出ます。Client Component では秘密の変数を読まず、必要な処理はサーバー側で行います。

解説: `NEXT_PUBLIC_` は「チラシに印刷してよい」という付箋でした（図4-11）。Publishable key（旧 anon key）は Row Level Security で守られている前提なので公開してかまいませんが、Secret key（旧 service_role key）やパスワードは絶対に付箋を付けてはいけません。3 は 8.2 の「つまずきポイント」で実際に確かめた落とし穴です。